UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO

FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

SEGUNDO EXAMEN PARCIAL

“DIAGRAMADOR UML: HERRAMIENTA CASE COLABORATIVA CON MODERADOR IA”

Modelado de diagramas de clases UML 2.5 y generación de backends Spring Boot — Ciclo 1 y Ciclo 2

Grupo: 43

Integrantes:

Angulo Heredia Luis Fernando — Registro: 215001958

Peña Rivero Elizabeth — Registro: 215053745

Materia: Ingeniería de Software I

Sigla: INF422-SB

Docente: Ing. Rolando Martínez

Semestre: 2026-2

Fecha de entrega: martes 10 de noviembre de 2026

Repositorio (GitHub)

Aplicación en producción

Índice

2. Fundamentación teórica 6
2.1. Ingeniería de Software Asistida por Computadora (CASE) 6
2.2. Desarrollo de Software Basado en Componentes (CBSE) 6
2.3. El Software como Producto 7
2.4. Ingeniería Humana (Experiencia del Usuario — UX) 8
3. Arquitectura de software 10
3.1. Propósito y estilos de la arquitectura de software 10
3.2. Transformación y diseño de datos 10
4. Modelado UML 2.5+ 11
4.1. Fundamentos y origen de UML 11
4.2. Del modelo orientado a objetos al modelo relacional: el porqué del mapeo 11
4.3. Notación UML 2.5+ utilizada en el proyecto 12
5. La Inteligencia Artificial en el desarrollo de software 13
6. Todo lo relacionado con Spring Boot 15
7. El Proceso de Desarrollo de Software (PUDS) 18
7.1. Características del Proceso Unificado 18
7.2. Ciclo de vida: ciclos, fases y flujos de trabajo 18
8. Fundamentación teórica del Segundo Parcial 20
8.1 La Ingeniería de Software Asistida por Computadora (CASE) 20
8.2 Aceleración de la productividad: desarrollo basado en componentes y reuso 21
8.3 Arquitectura del software (los 4 tipos) 21
8.4 Propósito del software: “dar larga vida al software” 25
8.5 IA local: diferencia entre entrenar y configurar un modelo 26
8.6 SDD — Specification-Driven Development (Desarrollo Dirigido por Especificaciones) 28
8.7 Estándares UML 29
8.8 Mecanismo de onboarding del usuario (agente de ayuda) 29
1. Perfil 32
1.1. Introducción 32
1.1.1 Introducción al Segundo Parcial 32
1.2 Descripción del problema 32
1.2.1 Descripción del problema — Segundo Parcial 33

1.3 Objetivo General 33
1.3.1 Objetivo General — Segundo Parcial 33
1.4 Objetivos Específicos 33
1.4.1 Objetivos Específicos — Segundo Parcial 34
1.5 Alcance 35
2. Captura De Requisitos 37
2.1 Identificar Actores Y Casos De Uso 37
2.2 Priorizar Casos De Uso 37
2.3 Detallar Caso De Uso 38
2.4 Estructurar Modelo De Caso De Uso 49
3. Análisis 52
3.1 Análisis De Arquitectura 52
3.2. Análisis de caso de uso — Diagramas de Comunicación 53
3.3. Análisis de clase 63
3.4. Análisis de paquete 65
4. Diseño 67
4.1 Diseño de arquitectura 67
4.2 Diseño de datos lógicos 69
4.3 Diseño de casos de uso 73
4.4 Diseño detallado del Segundo Ciclo 83
5. Implementación 88
5.1 Implementación de la arquitectura del sistema 88
5.2 Implementación de la arquitectura de subsistema 88
5.3 Implementación — Segundo Ciclo 88
6. Pruebas 92
6.1 Plan de pruebas 92
6.2 Diseño de pruebas (casos de prueba) 92
6.3 Pruebas de integración 93
6.4 Pruebas de sistema 93
6.5 Evaluación de las pruebas 94
6.6 Pruebas del Segundo Ciclo 94
7. Registro del desarrollo (evidencia del proceso) 96
7.1 Ciclos del Proceso Unificado y forma de trabajo 96

7.2 Línea de tiempo del 1er Parcial (base del proyecto) 99
7.3 Registro del 2do Parcial: plan vs. ejecución real 100
7.4 Evidencias de pruebas funcionales 103
7.5 Registro de defectos encontrados y corregidos 113
7.6 Trazabilidad requisito → código → prueba (insumo para V&V) 115
7.7 Herramientas utilizadas y uso de asistencia de IA 116
7.8 Pendientes registrados (al 18/10/2026) 116
3.1. Un agente asistente del software 118
3.2. Manual de Usuario 119
3.3. Mecanismo de aprendizaje del usuario (Segundo Parcial) 122
3.3.1 Enfoque: la aplicación enseña a usarse a sí misma 122
3.3.2 Nivel 1 — Estado vacío del lienzo 122
3.3.3 Nivel 2 — Recorrido guiado (tour) 123
3.3.4 Nivel 3 — Ayuda bajo demanda 125
3.3.5 Nivel 4 — Retroalimentación mientras se trabaja 126
3.3.6 Aprendizaje en el cliente móvil 127
3.3.7 Cómo aprende un usuario nuevo: recorrido de ejemplo 128
Anexos 129
Anexo A. Evidencia del sistema en funcionamiento 129
Anexo B — Estándar de Codificación 138
Anexo C. Información del grupo y recursos del proyecto 140
Anexo D. Metodología de investigación y de desarrollo 140
Bibliografía 142

PARTE 1

Fundamentación del Proyecto

2. Fundamentación teórica

2.1. Ingeniería de Software Asistida por Computadora (CASE)

La Ingeniería de Software Asistida por Computadora (CASE, por sus siglas en inglés) es el conjunto de herramientas que apoyan el trabajo de análisis, diseño, construcción y mantenimiento de software. Automatizan tareas repetitivas, verifican consistencia y generan artefactos —diagramas, código, documentación— a partir de modelos (Pressman & Maxim, 2015).

2.1.1 Funciones básicas de las herramientas CASE

- **Acelerar el desarrollo** evitando codificación manual innecesaria.
- **Estandarizar** nombres, capas y patrones (entidad → repositorio → servicio → controlador).
- **Reducir errores** con validaciones y plantillas.
- **Mantener trazabilidad** entre requerimientos, modelos y código.
- **Facilitar cambios**: si cambia el modelo, se regenera el código afectado.

2.1.2 Clasificación de herramientas CASE

- **Upper-CASE:** ayudan en fases tempranas (requerimientos, análisis, diseño).
- **Lower-CASE:** ayudan en construcción, pruebas y mantenimiento (generación de código, refactor).
- **Integradas:** cubren de punta a punta y conectan modelos con código.

El proyecto desarrollado corresponde a una herramienta CASE integrada: el mismo lienzo donde el usuario modela el diagrama de clases alimenta directamente al motor que genera el código del backend, sin pasos manuales intermedios.

2.1.3 Ventajas y desventajas

- **Ventajas:** productividad, estandarización del código generado, reducción de errores, documentación automática (exportación XMI).
- **Desventajas:** curva de aprendizaje de la herramienta, rigidez del código generado frente a plantillas, y dependencia tecnológica del framework de salida (en este caso, Spring Boot).

2.2. Desarrollo de Software Basado en Componentes (CBSE)

La Ingeniería de Software Basada en Componentes (CBSE) es un enfoque de desarrollo que construye aplicaciones a partir de componentes reutilizables e independientes, cada uno con una interfaz bien definida que expone qué hace sin exponer cómo lo hace (Szyperski, 2002). El

principio central es definir primero qué se necesita, y luego ensamblar y reutilizar librerías y módulos ya existentes que aceleren la productividad, en lugar de construir todo desde cero.

Este principio se aplicó de dos formas concretas en el proyecto:

- **Reutilización de componentes de terceros:** en vez de construir un motor de lienzo interactivo desde cero, se ensambló la librería React Flow para el modelado visual; TypeORM para la persistencia; Socket.io para la colaboración en tiempo real; y el SDK oficial de Google Gemini para la integración de IA. Cada una es un componente independiente, con una interfaz clara, integrado sin modificar su código interno.
- **Producción de componentes propios reutilizables:** el motor generador no escribe cada archivo Java a mano; produce cada uno de los cuatro artefactos (Entity, Repository, Service, Controller) a partir de la MISMA plantilla parametrizada, aplicada una vez por cada clase del diagrama. Esa plantilla es, en sí misma, un componente de software reutilizado tantas veces como clases tenga el modelo del usuario — exactamente el principio de "producir componentes que se usen varias veces".

Los beneficios observados coinciden con los descritos en la literatura: reducción de costos y tiempo de desarrollo al no construir cada pieza desde cero, escalabilidad (agregar un nuevo tipo de relación no obliga a reescribir el motor completo), y mantenibilidad, dado que una corrección en una plantilla se propaga automáticamente a todo el código futuro generado.

2.3. El Software como Producto

No cualquier software desarrollado puede llamarse, en sentido estricto, un “producto”. Un producto de software es aquel que ha superado la etapa de desarrollo interno y se ofrece —implícita o explícitamente— a un mercado que lo evalúa, lo compara con alternativas existentes y decide si le genera valor suficiente para adoptarlo (Pressman & Maxim, 2015). Esta distinción importa porque cambia el estándar de calidad exigido: un software que solo debe funcionar para el propio equipo que lo construyó se evalúa de forma muy distinta a uno que debe sostenerse frente al juicio de terceros.

2.3.1 Terminado, probado y con garantía

La primera condición es que el software esté completo: no un prototipo ni un producto mínimo viable (MVP), sino una versión que un tercero pueda usar sin que el equipo de desarrollo deba intervenir constantemente para que funcione. Un MVP, por definición, es una versión deliberadamente incompleta construida para validar una hipótesis con el menor esfuerzo posible (Ries, 2011): es una herramienta de aprendizaje, no un producto terminado. Un producto de software, en cambio, ha sido probado lo suficiente como para que el equipo esté dispuesto a respaldarlo con una garantía —la garantía es, en la práctica, la señal de que el desarrollador confía en que el software funcionará según lo prometido, y no solo “la mayoría de las veces” bajo condiciones controladas.

2.3.2 Validado por el mercado, no solo por el cliente

La segunda condición es que el software resuelva una necesidad validada por un mercado, no únicamente por el cliente puntual que lo encargó. Un desarrollo a medida puede satisfacer a un solo cliente sin que eso lo convierta en un producto; un producto, en cambio, debe demostrar que resuelve un problema compartido por un conjunto más amplio de usuarios potenciales. Es ese mercado —no el equipo que lo construyó ni un único comprador— quien finalmente juzga si el software tiene valor real.

2.3.3 Diferenciación: el sello que distingue al producto

La tercera condición reconoce que casi siempre existirá software similar compitiendo por la misma necesidad. Por eso un producto necesita una diferenciación deliberada —una marca, una característica distintiva o una combinación de decisiones de diseño— que lo posicione como superior a las alternativas existentes para el segmento de mercado que busca atender, en lugar de competir únicamente por ser “una opción más”.

2.3.4 Aplicación al proyecto

Aplicando este marco a la herramienta desarrollada, el diferencial que se busca (el “sello”) es la integración de capacidades que, por separado, sí existen en el mercado —canvas de modelado UML colaborativo por un lado, generadores de código a partir de modelos por otro— pero que rara vez conviven en una misma herramienta con foco explícito en la generación de un backend completo, validación lógica automática del modelo y un asistente de IA que edita el diagrama por operaciones puntuales en vez de regenerarlo. En cuanto a estar “terminado, probado y con garantía” en el sentido estricto aquí definido, el proyecto se ubica todavía en la etapa de prototipo funcional verificado técnicamente, no en la de un producto listo para el mercado —una distinción relevante para no sobreestimar su madurez actual frente a la definición desarrollada en esta sección.

2.4. Ingeniería Humana (Experiencia del Usuario — UX)

La Experiencia de Usuario (UX) se centra en cómo las personas interactúan con un sistema, evaluando facilidad de uso, accesibilidad, eficiencia, satisfacción y consistencia. Según la norma ISO 9241-210, la UX es “la respuesta de una persona que resulta del uso o la anticipación del uso de un producto, sistema o servicio”. En una herramienta CASE, donde el usuario pasa la mayor parte del tiempo manipulando un modelo visual, estos principios no son un añadido cosmético: determinan directamente qué tan rápido alguien puede pasar de una idea a un diagrama correcto.

Aplicación al proyecto:

- **Facilidad de uso y curva de aprendizaje:** un recorrido guiado interactivo (tour paso a paso) introduce cada función del lienzo la primera vez que se usa la herramienta, y una leyenda de notación UML permanente en el propio lienzo recuerda el significado de cada símbolo (triángulo hueco, rombos, línea punteada) sin obligar al usuario a salir de la aplicación para consultarlo.

- **Retroalimentación inmediata:** la validación lógica del modelo, el deshacer/rehacer, y el bloqueo visual de una clase que otro usuario está editando en ese momento le devuelven al usuario el estado del sistema sin que tenga que adivinarlo.
- **Reducción de carga cognitiva:** la barra de herramientas agrupa las acciones por función (edición, archivo, herramientas) en lugar de exponer una fila plana de botones sueltos, y las acciones menos frecuentes (importar, exportar, documentación, bitácora) se recogen en un menú desplegable “Archivo” en vez de competir visualmente con las acciones de uso constante.
- **Consistencia:** la misma semántica de multiplicidad y estereotipo se respeta en tres representaciones distintas del mismo modelo — el lienzo interactivo, el archivo XMI exportado y el código Java generado — evitando que el usuario tenga que reconciliar información contradictoria entre ellas.

3. Arquitectura de software

3.1. Propósito y estilos de la arquitectura de software

La arquitectura de software es el diseño estructural de un sistema: define cómo se organizan sus componentes y cómo interactúan entre sí (Fowler, 2002). Su propósito último, tal como lo plantea Martin (2017), no es meramente técnico sino de supervivencia del propio sistema: una buena arquitectura busca que el software viva el mayor tiempo posible, minimizando el costo humano de construirlo y —sobre todo— de mantenerlo y modificarlo a lo largo de los años. Cuánto tiempo "vive" un software depende directamente de su estructura: un sistema con acoplamiento bajo y responsabilidades bien separadas puede evolucionar durante años sin reescrituras completas; uno con una estructura rígida se vuelve, tarde o temprano, más caro de mantener que de reemplazar.

3.1.1 Estilos de arquitectura aplicados en el proyecto

- **Arquitectura en capas:** tanto el backend propio de la herramienta como cada backend Spring Boot generado siguen una separación estricta Controlador → Servicio → Repositorio → Entidad, con responsabilidades no solapadas entre capas.
- **Cliente-servidor:** el lienzo web (cliente) se comunica con el backend NestJS (servidor) vía REST y WebSockets.
- **Arquitectura basada en eventos:** la colaboración en tiempo real entre varios usuarios editando el mismo diagrama se resuelve mediante eventos emitidos y consumidos sobre WebSockets (Socket.io), no mediante sondeo (polling).

3.2. Transformación y diseño de datos

La transformación y diseño de datos define cómo la información se representa, organiza y manipula a lo largo de las tres etapas clásicas del diseño de bases de datos: conceptual, lógico y físico.

- **Diseño conceptual:** el diagrama de clases UML que el usuario dibuja en el lienzo (clases, atributos tipados, relaciones).
- **Diseño lógico:** la resolución automática de cada relación UML a su forma JPA equivalente (uno a uno, uno a muchos, muchos a uno, muchos a muchos), determinando el lado propietario y el lado inverso de cada relación bidireccional.
- **Diseño físico:** la generación de tablas, columnas, claves foráneas y tablas intermedias en PostgreSQL a partir de las anotaciones JPA producidas.

En este proyecto, la transformación de datos es el núcleo mismo de la herramienta: el motor generador convierte cada clase UML en una entidad JPA anotada (@Entity, @Id, @OneToMany, @ManyToOne, @ManyToMany), infiriendo automáticamente el tipo de columna, la visibilidad de los atributos y la multiplicidad de cada relación, sin que el usuario escriba una sola línea de SQL.

4. Modelado UML 2.5+

4.1. Fundamentos y origen de UML

El Lenguaje Unificado de Modelado (UML) es el estándar internacional para el modelado de sistemas de software. Su origen se remonta a la unificación de tres metodologías de modelado orientado a objetos que competían entre sí a inicios de los años noventa: la Técnica de Modelado de Objetos —OMT, Object Modeling Technique— de James Rumbaugh, el método de Grady Booch, y la técnica de Ingeniería de Software Orientada a Objetos —OOSE— de Ivar Jacobson (Rumbaugh, Jacobson, & Booch, 2005). Los tres autores, conocidos posteriormente como "los tres amigos", unieron sus notaciones en un único lenguaje formalizado y estandarizado por el Object Management Group (OMG), cuya especificación vigente se publica y mantiene en omg.org (Object Management Group, 2017).

4.1.1 Objetivos de UML 2.5

- Proporcionar un lenguaje de modelado estándar, expresivo y preciso.
- Facilitar el intercambio de modelos entre herramientas y equipos de desarrollo.
- Soportar el ciclo de vida completo del software, desde el análisis hasta la implementación.

4.1.2 Principales artefactos de UML 2.5

Diagramas estructurales: Clases, Componentes, Despliegue, Paquetes.

Diagramas de comportamiento: Casos de Uso, Secuencia, Comunicación, Actividad, Estado.

4.2. Del modelo orientado a objetos al modelo relacional: el porqué del mapeo

El proyecto se centra específicamente en el diagrama de clases, el artefacto estructural que describe la organización estática del sistema —clases, atributos y relaciones—, y en su transformación automática hacia un modelo relacional. Este paso no es trivial: existe una diferencia estructural conocida como el "desajuste de impedancia objeto-relacional" (object-relational impedance mismatch) entre el paradigma orientado a objetos y el modelo relacional (Fowler, 2002). Un objeto puede tener herencia, referencias directas a otros objetos y colecciones anidadas; una tabla relacional solo puede representar filas planas relacionadas por claves foráneas.

Por eso se mapea de POO a modelo relacional: cada clase se convierte en una tabla, cada atributo en una columna, y cada relación de objetos (asociación, agregación, composición, herencia) debe traducirse a un mecanismo puramente relacional —clave foránea para las relaciones de uno a muchos, tabla intermedia para las de muchos a muchos, o una tabla adicional para la herencia—. Este es precisamente el problema que resuelve el motor

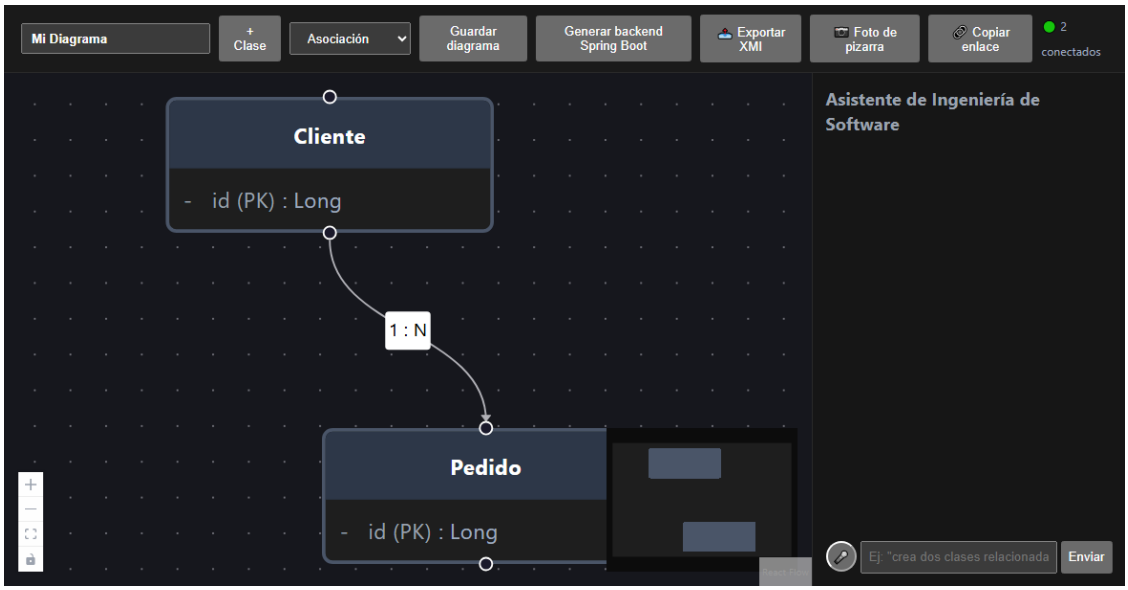
generador del proyecto: automatiza esa traducción, que de hacerse a mano es repetitiva y propensa a errores.

4.3. Notación UML 2.5+ utilizada en el proyecto

El usuario construye el diagrama de clases de forma interactiva sobre el lienzo, definiendo:

- **Clases:** con nombre y atributos tipados (nombre, tipo de dato, visibilidad, y marca de clave primaria).
- **Relaciones:** asociación, agregación, composición, herencia, y las cuatro multiplicidades relacionales (uno a uno, uno a muchos, muchos a uno, muchos a muchos).

El modelo resultante se exporta en formato estándar XMI 2.1 / UML 2.1 —el mismo formato de intercambio que reconocen herramientas profesionales como Enterprise Architect—, preservando clases, atributos tipados, visibilidad, multiplicidad de cada asociación y generalizaciones entre clases.



Anexo A.1 — Captura real del sistema en ejecución: lienzo con las clases Cliente y Pedido conectadas por una relación 1:N. El indicador "1 conectado" (esquina superior derecha) confirma la sesión de colaboración en tiempo real activa.

5. La Inteligencia Artificial en el desarrollo de software

El sistema integra inteligencia artificial directamente dentro del backend de la herramienta CASE, mediante la API de Google Gemini, con el objetivo de aumentar la productividad del usuario durante el modelado. La IA se aplica en tres canales multimodales complementarios: chat conversacional en texto, comandos de voz capturados por el navegador, e interpretación visual de fotografías de diagramas dibujados a mano.

5.1 Personalización del modelo

El modelo de lenguaje no se usa de forma genérica: se le personaliza mediante instrucciones de sistema distintas según la tarea. Para el chat, se le restringe ("enjaula") a responder exclusivamente sobre ingeniería de software y modelado UML; para la interpretación de imágenes, se le instruye explícitamente sobre cómo inferir el tipo de relación UML a partir de la notación dibujada a mano (flecha hueca, rombos, cardinalidades) y se le exige devolver una estructura JSON estricta y verificable por el backend.

5.2 Funciones de IA implementadas

- **Chat conversacional enjaulado:** responde exclusivamente sobre ingeniería de software y este proyecto, rechazando temas fuera de dominio.
- **Comandos de voz (Web Speech API):** el usuario dicta un comando (ej. "crea dos clases relacionadas") que se transcribe en el navegador y se envía al mismo canal de chat.
- **Interpretación de fotografías de pizarra:** un endpoint dedicado recibe una imagen, la envía a Gemini junto con las instrucciones de personalización descritas arriba, y devuelve un modelo estructurado (clases, atributos, relaciones) que se mezcla directamente en el lienzo del usuario.

Estas tres funciones comparten el mismo cliente de IA configurado en el backend, cada una invocada con su propio conjunto de instrucciones — una forma ligera de especialización por tarea que podría evolucionar, en una futura iteración, hacia una arquitectura de agentes más explícitamente separados (uno de conversación, uno de visión, uno de manipulación directa del lienzo).

Anexo A.2 — Transcripción real de una consulta al asistente de IA

Usuario: ¿Qué es una relación de agregación en UML y cuándo debo usarla?

- En el lenguaje de modelado UML, la agregación es un tipo de asociación especial que representa una relación de "todo-parte" (has-a) entre dos clases, donde el acoplamiento entre ellas es débil.
- Independencia de ciclo de vida: la "parte" puede existir de forma independiente al "todo". Si el objeto contenedor se destruye, los objetos contenidos siguen existiendo.

- Representación gráfica: se dibuja con una línea continua que une a las dos clases, con un rombo hueco en el extremo de la clase contenedora.
- Diferencia con la composición: en la composición (rombo lleno) el acoplamiento es fuerte y la parte no puede existir sin el todo (ej. Factura–DetalleFactura); en la agregación (rombo hueco) el acoplamiento es débil (ej. Departamento–Profesor).
- Traducción a Spring Boot/JPA: se suele mapear con @OneToMany o @ManyToMany sin cascade = CascadeType.REMOVE, garantizando que eliminar la entidad principal no borre las entidades asociadas.

(Respuesta real generada por Google Gemini durante una sesión de pruebas del sistema; no editada.)

6. Todo lo relacionado con Spring Boot

El principal artefacto de salida de la herramienta es un backend Spring Boot completo y compilable, generado automáticamente a partir del diagrama de clases, siguiendo la arquitectura en capas Controlador → Servicio → Repositorio → Entidad, con PostgreSQL como base de datos relacional.

6.1 Contenido generado por clase del diagrama

- **Entidad (Entity):** clase Java anotada con @Entity, con sus atributos tipados y las relaciones resueltas como campos JPA en ambos extremos de cada relación bidireccional.
- **Repositorio (Repository):** interfaz que extiende JpaRepository, habilitando las operaciones CRUD estándar sin código adicional.
- **Servicio (Service):** clase con la lógica de negocio básica: buscar, guardar, actualizar y eliminar.
- **Controlador (Controller):** endpoints REST (GET, POST, PUT, DELETE), con soporte CORS habilitado (@CrossOrigin) para que cualquier frontend pueda consumirlos sin configuración adicional.
- **Configuración de serialización:** una clase adicional que resuelve dos problemas reales detectados durante el desarrollo — la recursión infinita al serializar relaciones bidireccionales a JSON, y el error de serialización de los proxies perezosos de Hibernate — mediante anotaciones @JsonIgnoreProperties y el módulo Hibernate6Module.

6.2 Edición del backend generado y pruebas con Postman

El backend generado no es una caja negra: es un proyecto Maven estándar que el desarrollador puede abrir, editar y extender libremente después de generado —agregar validaciones, reglas de negocio adicionales, seguridad, etc.— sin perder la estructura de cuatro capas con la que fue creado.

Se validó el flujo completo de extremo a extremo: se generó un backend a partir de un diagrama de prueba, se compiló con Maven, se ejecutó localmente contra una base de datos PostgreSQL real, y se probó cada endpoint (crear, listar, actualizar, eliminar) mediante peticiones HTTP equivalentes a Postman. Esta verificación práctica permitió detectar y corregir en la propia plantilla generadora los dos problemas de serialización JSON mencionados en 6.1, garantizando que todo backend generado a partir de ese momento los tenga resueltos de forma permanente.

6.3 Librerías y dependencias utilizadas

El backend generado es un proyecto Maven estándar sobre Spring Boot 3.3.4 y Java 17, heredando la gestión de versiones del spring-boot-starter-parent (BOM que fija versiones compatibles entre todas las dependencias, evitando conflictos manuales). Cada librería incluida

responde a una necesidad concreta de la arquitectura en cuatro capas; no se agregó ninguna dependencia superflua.

- **spring-boot-starter-web —** expone la capa de controladores como una API REST. Incluye el servidor embebido Tomcat, el framework Spring MVC y la librería de serialización Jackson, permitiendo que el backend arranque de forma autónoma (java -jar) sin necesidad de desplegar en un servidor de aplicaciones externo.
- **spring-boot-starter-data-jpa —** provee la capa de persistencia mediante JPA (Jakarta Persistence API) con Hibernate como implementación ORM. Es la librería que permite que cada @Entity del diagrama se mapee automáticamente a una tabla, y que los repositorios (JpaRepository) ofrezcan las operaciones CRUD sin escribir SQL.
- **postgresql —** driver JDBC de PostgreSQL. Es el puente entre Hibernate y la base de datos relacional; sin él, la aplicación no podría abrir conexiones al motor PostgreSQL configurado en application.properties.
- **spring-boot-starter-validation —** integra Jakarta Bean Validation (Hibernate Validator). Habilita la validación declarativa de los datos que entran por los endpoints mediante anotaciones (@NotNull, @Size, etc.), reforzando la integridad de la información antes de persistirla.
- **jackson-datatype-hibernate6 —** módulo de Jackson que resuelve un problema real detectado durante el desarrollo: la serialización a JSON de los proxies perezosos (lazy) de Hibernate. Sin este módulo, exponer entidades con relaciones perezosas por la API provocaría errores de serialización. Se registra junto con @JsonIgnoreProperties para evitar además la recursión infinita en relaciones bidireccionales.
- **springdoc-openapi-starter-webmvc-ui (versión 2.6.0) —** genera de forma automática la especificación OpenAPI 3 de todos los controladores y sirve una interfaz Swagger UI navegable en /swagger-ui.html. Permite al evaluador (o a cualquier consumidor del backend) explorar y probar los endpoints desde el navegador sin herramientas externas, complementando las pruebas con Postman.
- **spring-boot-starter-test (ámbito test) —** suite de pruebas que agrupa JUnit 5, Mockito, AssertJ y Spring Test. Queda disponible en el proyecto generado para que el desarrollador añada pruebas unitarias y de integración sobre la lógica de negocio de la capa de servicio.
- **spring-boot-maven-plugin (plugin de build) —** empaqueta la aplicación como un JAR ejecutable auto-contenido (fat JAR) que incluye todas las dependencias y el servidor embebido, de modo que el backend se ejecuta con un único comando y es directamente contenedorizable con Docker.

En conjunto, estas dependencias materializan la arquitectura en capas Controlador → Servicio → Repositorio → Entidad descrita en 6.1: spring-boot-starter-web para la capa de exposición, spring-boot-starter-data-jpa y el driver postgresql para la persistencia, spring-boot-starter-validation y los módulos de Jackson para la integridad y correcta serialización de los datos, y springdoc-openapi para la documentación viva de la API. La selección se mantuvo

deliberadamente mínima y estándar para que el backend generado sea fácil de entender, extender y defender.

6.4 Artefactos adicionales generados

Además del código Java de las cuatro capas, el proyecto generado incluye archivos que facilitan su prueba y despliegue inmediato:

- **postman_collection.json:** colección Postman v2.1 con todos los endpoints CRUD de cada clase, lista para importar en Postman y probar el backend sin configuración manual.
- **src/main/resources/schema.sql** *(Ciclo 1)*: script DDL con la definición de las tablas. En el Ciclo 2 se eliminó del proyecto generado: el esquema lo crea el ORM (Hibernate) a partir de las entidades, y `data.sql` solo inserta datos de prueba en esas tablas.
- **src/main/resources/data.sql:** datos de prueba realistas, ordenados para respetar las claves foráneas (las entidades padre se insertan antes que las hijas), de modo que el sistema arranque con información de ejemplo.

Adicionalmente, la herramienta web permite exportar la colección Postman directamente desde el diagrama, sin necesidad de generar todo el proyecto, y ofrece una vista previa del código generado antes de la descarga.

7. El Proceso de Desarrollo de Software (PUDS)

El Proceso Unificado de Desarrollo de Software (PUDS) es el proceso propuesto por Jacobson, Booch y Rumbaugh (1999), los mismos autores de UML. Define quién hace qué, cuándo y cómo para llevar los requisitos de un usuario hasta un sistema de software, y usa UML como lenguaje para todos sus modelos.

7.1. Características del Proceso Unificado

- **Dirigido por casos de uso:** los casos de uso capturan los requisitos y guían todo lo demás. El análisis, el diseño, la implementación y las pruebas se planifican y se verifican caso de uso por caso de uso.
- **Centrado en la arquitectura:** desde temprano se define la arquitectura (las partes más importantes del sistema y cómo se relacionan), y el resto se construye sobre ella.
- **Iterativo e incremental:** el trabajo se divide en iteraciones; cada iteración recorre los flujos de trabajo y produce un incremento ejecutable del sistema.

7.2. Ciclo de vida: ciclos, fases y flujos de trabajo

La vida de un sistema se organiza en **ciclos**. Cada ciclo termina con una versión del producto entregada al usuario. Dentro de cada ciclo hay **cuatro fases**, y cada fase termina en un hito que decide si se puede pasar a la siguiente:

Fase	Propósito	Hito al terminar
Inicio	Delimitar el alcance, identificar los casos de uso principales y los riesgos críticos, y decidir si el proyecto es viable	Objetivos del ciclo de vida
Elaboración	Capturar la mayoría de los requisitos, definir y probar la arquitectura base, y planificar la construcción	Arquitectura del ciclo de vida
Construcción	Construir el sistema completo sobre la arquitectura, iteración por iteración	Capacidad operativa inicial (versión beta)
Transición	Llevar el producto a los usuarios: despliegue, corrección de defectos y capacitación	Lanzamiento del producto

En cada fase se recorren, con distinta intensidad, los **cinco flujos de trabajo fundamentales**:

Flujo de trabajo	Pregunta que responde	Modelo que produce
Requisitos	¿Qué debe hacer el sistema?	Modelo de casos de uso

Flujo de trabajo	Pregunta que responde	Modelo que produce
Análisis	¿Cómo se estructuran los requisitos, sin pensar aún en la tecnología?	Modelo de análisis (clases de interfaz, control y entidad; diagramas de comunicación)
Diseño	¿Cómo se construye con la tecnología elegida?	Modelo de diseño y de despliegue (clases de diseño, secuencia, subsistemas)
Implementación	¿Qué componentes y código lo realizan?	Modelo de implementación (componentes, código)
Prueba	¿El sistema hace lo que dicen los requisitos?	Modelo de pruebas (casos y procedimientos de prueba)

La intensidad cambia según la fase: en Inicio predominan los requisitos; en Elaboración, el análisis y el diseño de la arquitectura; en Construcción, la implementación y las pruebas; y en Transición, las pruebas y el despliegue.

La Parte 2 aplica este ciclo de vida al proyecto: el **Ciclo 1** corresponde al 1er Parcial y el **Ciclo 2** al 2do Parcial. Cada uno recorre las cuatro fases, y el capítulo 7 de la Parte 2 resume en una tabla qué flujos se trabajaron en cada fase de cada ciclo.

8. Fundamentación teórica del Segundo Parcial

8.1 La Ingeniería de Software Asistida por Computadora (CASE)

Una herramienta **CASE** (*Computer-Aided Software Engineering*) es un software que automatiza o asiste actividades del proceso de desarrollo: análisis, diseño, generación de código, documentación, pruebas y gestión. Su objetivo es aumentar la productividad y la calidad, reduciendo los errores manuales que se producen al pasar de una fase a otra (Sommerville, 2016).

Según la parte del ciclo de vida que cubren, las herramientas CASE se clasifican en:

Tipo	Fases que cubre	Ejemplos
Upper CASE (alto nivel)	Requisitos, análisis y diseño (modelado)	Enterprise Architect, StarUML
Lower CASE (bajo nivel)	Implementación, generación de código, pruebas	Generadores de código, compiladores
I-CASE (integradas)	Cubren ambas, con un repositorio común	Rational Rose, herramientas MDA

Beneficios: coherencia entre diseño e implementación, menor tiempo de desarrollo, documentación siempre actualizada, estandarización (UML) y trabajo en equipo sobre un mismo modelo.

Nuestra aplicación es una herramienta I-CASE, porque integra en un mismo repositorio (el diagrama) las dos mitades del ciclo:

- **Upper CASE:** modelado visual de diagramas de clases UML, validación del modelo, interoperabilidad XMI con Enterprise Architect y documentación generada.
- **Lower CASE:** generación automática de un backend **Spring Boot 3.3.4** completo (entidades JPA, repositorios, servicios, controladores REST, Swagger, `data.sql` con datos de prueba, colección Postman y Docker). Las tablas no salen de un script: las crea el ORM (Hibernate) a partir de las entidades.
- **Asistencia inteligente (2do Parcial):** un moderador de IA grupal que responde dudas teóricas y modifica el diagrama compartido a pedido.

Dónde se ve en el código

- Modelado: `frontend/src/App.tsx`, `frontend/src/components/UmlClassNode.tsx`, `UmlRelationEdge.tsx`
- Validación del modelo: `backend/src/diagrams/validation.util.ts`
- Interoperabilidad XMI (OMG XMI 2.1): `backend/src/diagrams/xmi.util.ts`, `xmi-import.util.ts`

- Generador: `backend/src/generator/generator.service.ts` y `backend/src/generator/templates/`

8.2 Aceleración de la productividad: desarrollo basado en componentes y reuso

El **Desarrollo Basado en Componentes** (CBSE) construye sistemas ensamblando piezas independientes y reutilizables con interfaces bien definidas, en lugar de escribir todo desde cero. El **reuso** se aplica a varios niveles: código, diseño (patrones) y arquitectura (plantillas de aplicación).

Nuestro generador aplica el reuso de tres formas:

1. **Plantillas como componentes de generación.** Cada capa del backend generado sale de una plantilla independiente y probada: `entity`, `repository`, `service`, `controller`, `enum`, `interface`, `data-sql`, `postman`, `openapi-config`, `jackson-config` y `project-files`. Agregar una capa nueva es agregar un componente sin tocar los demás.
2. **Reuso de componentes del ecosistema Spring.** El código generado no reimplementa la persistencia ni la API: se apoya en componentes maduros (Spring Data JPA, Spring Web, springdoc-openapi/Swagger), que es lo que permite que un diagrama se convierta en un backend funcional en segundos.
3. **Arquitectura en capas como plantilla de aplicación.** Todo backend generado repite la misma estructura Controller → Service → Repository → Entity, así que cualquier desarrollador ya sabe dónde está cada cosa.

En el frontend se aplica el mismo principio: la interfaz se compone de **31 componentes React** reutilizables (lienzo, inspector, chat, paneles), y la lógica pura (diferencias, métricas, exportaciones) vive en módulos sin UI.

Dónde se ve en el código

- Plantillas (cada una con su test): `backend/src/generator/templates/*.template.ts` y `*.spec.ts`
- Componentes de UI: `frontend/src/components/` · lógica reutilizable: `frontend/src/lib/`

8.3 Arquitectura del software (los 4 tipos)

El propósito de la arquitectura es que el software **viva el mayor tiempo posible**: que se pueda cambiar, escalar y mantener sin rehacerlo. Como “el software” es demasiado amplio, la arquitectura se diseña desde **cuatro perspectivas**, y cada una puede usar un estilo distinto: backend, frontend, datos y despliegue.

Estilos conocidos para el backend:

Estilo	Idea central

Estilo	Idea central
En capas	Presentación → negocio → datos; cada capa solo usa la de abajo
MVC	Separa el modelo, la vista y el controlador que los coordina
Hexagonal (puertos y adaptadores)	El dominio en el centro; la base de datos, la web o la IA se conectan por puertos (interfaces) con adaptadores intercambiables
Clean Architecture / cebolla	Capas concéntricas; las dependencias apuntan siempre hacia el dominio
DDD	El diseño sigue al dominio del negocio, dividido en contextos delimitados
Microservicios	Cada capacidad del negocio es un servicio independiente con su propia base de datos

Nuestro backend combina **capas** dentro de cada módulo con una idea de **puertos y adaptadores** en el motor de IA: `AiProvider` es el puerto, y `GeminiProvider` y `OllamaProvider` son adaptadores intercambiables.

8.3.1 Arquitectura de backend — monolito modular por responsabilidades

El backend de la herramienta CASE es un **monolito modular en NestJS** (Node.js + TypeScript). Cada módulo encapsula una responsabilidad y expone solo lo necesario mediante inyección de dependencias:

Módulo	Responsabilidad
`auth`	Registro, login y JWT; guard global que protege la API REST
`users`	Usuarios (PostgreSQL)
`diagrams`	Diagramas, autoría granular, bitácora, sesiones continuas, gateway de colaboración en tiempo real
`ai`	Motor IA híbrido (Gemini + Ollama), moderador grupal y gateway del chat
`generator`	Generación del backend Spring Boot en ZIP
`attachments`	Archivos adjuntos en Amazon S3

Dentro de cada módulo se respeta la separación **Controller (HTTP) / Gateway (WebSocket) → Service (lógica) → Schema/Entity (datos)**. En el motor de IA se aplica el **patrón Strategy** (`AiProvider`, con dos implementaciones: `GeminiProvider` y `OllamaProvider`) compuesto por `HybridAiProvider`, que agrega fallback automático y *circuit breaker*.

Evidencia: `backend/src/app.module.ts`, `backend/src/*/…module.ts`, `backend/src/ai/providers/`.

8.3.2 Arquitectura de frontend — componentes, hooks y capa de API

- **Arquitectura de componentes (React + Vite + TypeScript):** cada parte de la UI es un componente independiente (`ChatPanel`, `ClassInspector`, `UmlClassNode`, `HistoryPanel`…).
- **Estado global con Context API:** `AuthContext` (sesión y JWT) y `ToastContext` (avisos).
- **Hooks personalizados** para efectos secundarios: `useSpeechRecognition` (dictado por voz).
- **Capa de acceso a datos separada:** `api/client.ts` (REST con Axios e interceptor JWT) y `api/socket.ts` (Socket.io con el JWT en el *handshake*). Los componentes no hablan directamente con el servidor.
- **Lienzo con React Flow**, que representa clases como nodos y relaciones como aristas.

Evidencia: `frontend/src/components/`, `frontend/src/context/`, `frontend/src/hooks/`, `frontend/src/api/`.

8.3.3 Arquitectura de datos — persistencia políglota

En el 2do Parcial adoptamos **persistencia políglota**: cada dato se guarda en el motor que mejor se adapta a su forma.

Motor	Colecciones / tablas	Por qué
MongoDB (NoSQL documental)	`diagrams`, `diagram_versions`, `diagram_sessions`, `diagram_activity_logs`, `group_chat_messages`	El modelo UML es un **árbol** (clases → atributos, relaciones) con la autoría **embebida** en cada elemento; las sesiones y la bitácora crecen con el uso. Un documento evita *joins* y se guarda tal cual.
PostgreSQL (relacional)	`users`, `attachments`	Datos relacionales con restricciones (correo único), estables y ya probados en el 1er Parcial.

Decisiones clave:

- **Autoría granular:** cada clase, atributo y relación guarda `author`, `lastModifiedBy` y `source` (MANUAL, AI, XMI_IMPORT). La calcula el servidor comparando el modelo anterior con el nuevo, así que **el cliente no puede falsificarla**.
- **Bitácora estructurada:** una entrada por cambio (`CLASS_CREATED`, `ATTRIBUTE_ADDED`, `RELATION_DELETED`…), que permite a la IA responder *"¿qué hizo Marco?"* con datos reales.

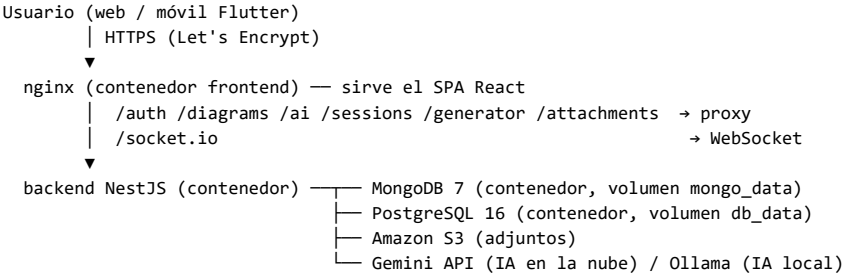
- **Historial de versiones:** cada sesión de edición deja una foto completa del modelo (`diagram_versions`), y cualquiera se puede **restaurar**. Restaurar crea una versión nueva: el historial nunca se reescribe. Un documento por versión es natural en MongoDB, porque el modelo se guarda entero tal cual.
- **Migración sin pérdida:** un script idempotente copió los datos del 1er Parcial conservando los mismos identificadores (UUID).

Otros modelos de datos que se evaluaron:

Modelo	Para qué sirve	¿Lo usamos?
Relacional (PostgreSQL)	Datos con estructura fija, restricciones y transacciones	Sí: usuarios y adjuntos, y la base que genera el backend Spring Boot
NoSQL documental (MongoDB)	Árboles de datos que se leen y guardan enteros	Sí: diagramas, sesiones, bitácora y chat
NoSQL clave-valor (DynamoDB, Redis)	Lecturas muy rápidas por clave	No
Orientado a objetos	Guardar objetos tal cual, sin mapeo	No
Vectorial	Cada registro tiene un vector (*embedding*) generado por un modelo de IA; se buscan los vectores más cercanos para aproximar la **intención** de una consulta (búsqueda semántica, RAG)	No por ahora; sería útil para que la IA busque en chats y diagramas antiguos por significado y no por palabras exactas

Evidencia: `backend/src/diagrams/schemas/`, `backend/src/diagrams/authorship.util.ts`, `backend/src/scripts/migrate-pg-to-mongo.ts`.

8.3.4 Arquitectura de despliegue — contenedores en la nube



- **Docker Compose** orquesta cuatro contenedores separados con *healthchecks* y reinicio automático; los datos viven en volúmenes persistentes.
- **Servidor en la nube** (instancia AWS EC2 de 1 GB) con dominio **DuckDNS** (`diagramasw1pracial100.duckdns.org`) y certificado **SSL/TLS de Let's Encrypt**; nginx redirige HTTP → HTTPS.

- **Despliegue por compilación local:** el `dist/` se compila en la PC y se sube por SCP, porque compilar en el servidor de 1 GB agotaba la memoria.
- **Plan B local:** `docker-compose.local.yml` levanta la misma pila en una laptop con IA local, para defender sin internet.

Evidencia: `docker-compose.prod.yml`, `docker-compose.local.yml`, `frontend/nginx.conf`, `deploy/deploy-local-build.ps1`.

8.3.5 Monolito o microservicios: por qué elegimos un monolito modular

	Monolito	**Microservicios**
A favor	Fácil de desplegar y probar; transacciones y reportes simples (una sola base de datos)	Cada servicio escala por separado y usa los recursos que necesita; se despliegan de forma independiente
En contra	Escala como un todo; una actualización baja todo el sistema	Mucho más complejo de coordinar (ver abajo)

Lo que se complica con microservicios, y cómo se resuelve:

- **Transacciones y *rollback*:** si un flujo pasa por tres servicios y el tercero falla, no hay un *rollback* único. Se usa el **patrón Saga**: cada paso tiene una acción de compensación que deshace lo hecho.
- **Reportes y *joins*:** cada servicio tiene su base de datos y no se puede hacer *join* entre ellas. Se replica la información a una base de **solo lectura** para consultas (CQRS) o a un ***data warehouse***.
- **Comunicación de cambios:** si inventario modifica un producto, ventas debe enterarse. Se publican **eventos** en un *broker* de mensajes y cada servicio interesado actualiza su réplica.
- **Frontend:** puede haber un solo frontend o **microfrontends**; todos llegan a los servicios a través de un **API Gateway**.

Nuestra decisión: el proyecto no tiene todavía una carga que justifique microservicios. Por eso es un **monolito modular**: módulos independientes (`auth`, `diagrams`, `ai`, `generator`…) que se comunican por interfaces y podrían separarse en servicios si algún día hiciera falta. El motor de IA sería el primer candidato, porque es el que más recursos consume.

8.4 Propósito del software: “dar larga vida al software”

Un software vive mientras pueda **cambiar a bajo costo**. La mayor parte del costo total ocurre después de la primera versión, en mantenimiento y evolución (Sommerville, 2016). Para alargar la vida del sistema aplicamos:

1. **Estándares de programación verificados por herramientas:**

- TypeScript con tipado estricto (backend con `strict: true`; frontend con `noUnusedLocals`, `noUnusedParameters` y `noFallthroughCasesInSwitch`).
- Linter **oxlint** (reglas de React Hooks incluidas) y formateador **Prettier**; en Flutter, `flutter_lints`.
- El estándar de codificación del equipo está documentado en el Anexo B.

2. **Diseño modular y extensible:** módulos con una sola responsabilidad, plantillas independientes en el generador y contratos explícitos (`AiProvider`). Agregar un tercer proveedor de IA o una nueva capa generada no obliga a modificar el resto.
3. **Pruebas automatizadas: 152 pruebas unitarias** (Vitest) cubren el generador, la validación, XMI, la autoría, el motor híbrido y el prompt del moderador. Fallan si un cambio rompe un comportamiento existente.
4. **Evolución sin romper lo anterior:** el 2do Parcial agregó MongoDB, IA grupal y el visor móvil **manteniendo intactas** las funciones del 1er Parcial (verificado con pruebas de regresión: XMI, generador, locks).
5. **Configuración externa:** URL, claves y proveedores de IA se definen con variables de entorno (`.env.example` documentado), no en el código.

Evidencia: `backend/tsconfig.json`, `.prettierrc`, `oxlint.json`, `docs/estandar_de_codificacion.md`, `backend/src/**/*.spec.ts`.

8.5 IA local: diferencia entre entrenar y configurar un modelo

	Entrenar un modelo	**Configurar un modelo (lo que hicimos)**
Qué se hace	Ajustar millones de parámetros con grandes volúmenes de datos	Usar un modelo **ya preentrenado** y guiarlo con instrucciones
Requiere	Datasets, GPU/TPU, días o semanas de cómputo	Un *runtime* (Ollama) y un buen *System Prompt*
Resultado	Un modelo nuevo (pesos nuevos)	El mismo modelo con un **rol** y un **contexto** definidos

Nosotros configuramos, no entrenamos. Usamos **qwen2.5:1.5b-instruct** (Alibaba) ejecutado localmente con **Ollama**, y **Gemini** (Google AI Studio) en la nube. Lo configuramos así:

- **System Prompt de rol:** *"Eres el Moderador y Desarrollador Senior del equipo… dirígete siempre por su nombre a quien pregunta… solo modifica el lienzo si te lo piden explícitamente"*.
- **Contexto dual inyectado en cada pregunta:**
  - **Grupal:** el estado del diagrama con su autoría, la bitácora de quién hizo qué y el chat público de la sala.
  - **Individual:** el hilo de preguntas previas de quien pregunta.

- **Ejemplos (few-shot)** y un mini glosario UML para el modelo pequeño.
- **Parámetros de inferencia:** temperatura 0 (respuestas deterministas) y salida en JSON con un contrato fijo de operaciones.
- **Modelo propio en Ollama: `moderador-uml`.** Lo construimos con un *Modelfile* (`FROM qwen2.5:1.5b-instruct`) que fija el rol del moderador, el glosario y los parámetros (temperatura 0, ventana de 4.096 tokens, límite de 256 tokens de respuesta). El Modelfile se **genera desde el código** (`npm run ollama:modelo`), así que el modelo y el backend nunca quedan desincronizados. Si el modelo no está creado, el backend usa qwen2.5 base.
- **Validación determinista:** el servidor descarta operaciones inválidas y las que no responden a un pedido explícito. **La IA nunca toca el diagrama sin control.**

Proceso para trabajar con una IA local (el que seguimos):

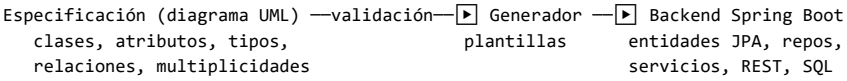
1. **Elegir el modelo según el hardware:** el tamaño (cantidad de parámetros) y la cuantización deciden cuánta memoria y velocidad necesita. Con una laptop sin GPU elegimos un modelo de 1.500 millones de parámetros.
2. **Instalar un *runtime*** (Ollama) y descargar el modelo (`ollama pull`).
3. **Personalizarlo:** de menor a mayor esfuerzo, con un *System Prompt* y ejemplos, con un **Modelfile** que guarda ese rol y los parámetros como un modelo propio, con **RAG** (darle documentos o datos en cada consulta), o con ***fine-tuning*** (por ejemplo LoRA), que sí cambia los pesos. Nosotros llegamos hasta el Modelfile y le damos el contexto del diagrama en cada consulta.
4. **Integrarlo** por su API HTTP (`/api/generate`) con un contrato fijo de respuesta en JSON.
5. **Medir y ajustar:** tiempo de respuesta, calidad y límites conocidos (sin internet arma solo 2 clases de un proyecto completo).

Motor híbrido (local + nube): ambos proveedores comparten el mismo contrato. Por defecto responde Gemini, y **Ollama toma el relevo automáticamente si no hay internet**; con `AI_HYBRID_ORDER=local-first` se invierte el orden para la demostración. Lo medimos en la laptop del equipo (CPU): el prompt completo tardaba 36–79 s en el modelo local; el *prompt lite*, con caché de prefijo, baja a 5–18 s.

Dónde se ve en el código: `backend/src/ai/moderator/moderator-prompt.ts`, `backend/src/ai/providers/hybrid.provider.ts`, `ollama.provider.ts`, `backend/src/ai/moderator/intent.ts`.

8.6 SDD — Specification-Driven Development (Desarrollo Dirigido por Especificaciones)

En SDD la **especificación es el artefacto central**: el código se deriva de ella, y no al revés. En nuestra herramienta, **el diagrama de clases UML es la especificación** y el generador la transforma directamente en un backend ejecutable:



Se relaciona con el **Proceso Unificado (PUDS)** de Jacobson, Booch y Rumbaugh (1999), que es dirigido por modelos: el modelo de diseño guía la implementación.

- **Ventajas:** el proceso es claro y estructurado, el código es coherente con el diseño, la generación es inmediata y el modelo sirve como documentación viva.
- **Desventaja (la "ventana" del SDD):** *si se olvidó algo en la especificación, todo lo generado lo arrastra.* Una relación mal declarada produce claves foráneas incorrectas en todas las capas.
- **Cómo mitigamos esa ventana:** el validador revisa el modelo **antes** de generar (clases sin nombre o sin clave primaria, clases o atributos duplicados, relaciones hacia clases inexistentes, atributos desnormalizados y claves foráneas escritas a mano), la IA moderadora señala problemas de modelado, y la bitácora con autoría permite rastrear quién introdujo cada elemento.

SDD al desarrollar con IA (cómo construimos el proyecto): usamos un asistente de IA integrado al entorno de desarrollo, pero no a puro *prompt*. Antes de pedir código escribimos la especificación, y esa especificación es la **fuente de verdad** (*source of truth*) que la IA lee en cada tarea:

Paso	Artefacto del proyecto
Especificación: qué se pide y con qué reglas	`Contexto_Examen_Requisitos.md`, `CONTEXTO_CLAUDE_2DO_PARCIAL.md` y los modelos UML de la Parte 2
Plan técnico	`PLAN_EVOLUCION_2DO_PARCIAL.md`, `PLAN_SEMANA1_MONGODB.md` (la migración se diseñó antes de escribir código)
Tareas	Cronograma de 4 semanas y checklist del plan
Implementación	Commits pequeños, uno por tarea (Conventional Commits)
Verificación	Pruebas automatizadas y la trazabilidad requisito → código → prueba (capítulo 7 de la Parte 2)

Así se reduce el ida y vuelta con la IA y el tiempo de revisar lo que genera: si el código no coincide con la especificación, se corrige el código, no la especificación.

Dónde se ve en el código: `backend/src/diagrams/validation.util.ts`, `backend/src/generator/`.

8.7 Estándares UML

- **UML** es el lenguaje estándar de modelado de software, mantenido por el **OMG** (*Object Management Group*, omg.org). La versión vigente es **UML 2.5.1** (Object Management Group, 2017).
- Nuestro diagrama de clases soporta la notación UML 2.x:
    - Clases con atributos y operaciones, visibilidad (`+ - # ~`) y estereotipos («`enum`», «`abstract`», «`interface`»).
    - Relaciones de asociación, agregación, composición, herencia y dependencia, con multiplicidades 1:1, 1:N, N:1 y N:M y nombres de rol.
- **Intercambio estándar:** los diagramas se exportan e importan en **XMI** (*XML Metadata Interchange*, también del OMG), compatible con Enterprise Architect.
- **Reglas de modelado — OMT:** la *Object Modeling Technique* de Rumbaugh et al. (1991) es la base del modelo de clases de UML. De sus reglas de modelado salen las que aplica nuestro validador:
    - toda clase tiene nombre único e identifica a sus objetos (clave primaria);
    - una clase no repite atributos;
    - las asociaciones solo unen clases existentes;
    - los vínculos entre clases se modelan como **relaciones** y no como atributos sueltos (por eso se avisa de las claves foráneas escritas a mano y de los atributos desnormalizados).
- **Coherencia entre diagramas:** los modelos no son independientes. Cada objeto de un diagrama de secuencia es instancia de una clase del diagrama de clases. Si el objeto `b` envía el mensaje `m()` al objeto `a`, entonces `m()` es una **operación de la clase de a** (la que recibe), porque es un servicio que `a` ofrece y `b` utiliza. En el código, `m()` es un método de esa clase. Ejemplo del proyecto: `ChatGateway` envía `answer()` a `ModeratorService`, así que `answer()` está implementada en `ModeratorService`.

Dónde se ve en el código: `backend/src/diagrams/uml.types.ts`, `frontend/src/components/UmlLegend.tsx`, `backend/src/diagrams/xmi.util.ts`.

8.8 Mecanismo de onboarding del usuario (agente de ayuda)

La aplicación **enseña a usarse a sí misma**, sin documentación externa:

1. **Recorrido guiado (Tour):** la primera vez que el usuario entra, un tour de 7 pasos resalta cada parte de la interfaz. Los pasos son:
    1. dar nombre al diagrama;

2. agregar una clase;
3. elegir el tipo de relación;
4. validar el modelo;
5. generar el backend Spring Boot;
6. exportar a XMI;
7. usar el asistente de IA.

Se puede repetir desde el botón **“Recorrido”** de la barra.

2. **Ayuda contextual:**
    - Un **estado vacío** del lienzo con las primeras acciones sugeridas.
    - Una **leyenda UML** que explica la notación.
    - Un modal de **atajos de teclado**.
    - Una **paleta de comandos** (`Ctrl+K`).
    - **Plantillas** de diagramas de ejemplo.
3. **Agente de ayuda en línea (IA):** el Moderador IA del chat público responde dudas sobre cómo modelar y sobre conceptos de ingeniería de software, dirigiéndose por su nombre a quien pregunta. Como el chat es público, lo que aprende un integrante queda visible para todo el equipo.

Dónde se ve en el código: `frontend/src/components/Tour.tsx` (pasos en `TOUR_STEPS`, `App.tsx`), `CanvasEmptyState.tsx`, `UmlLegend.tsx`, `ShortcutsModal.tsx`, `CommandPaletteModal.tsx`, `TemplatesPanel.tsx`, `ChatPanel.tsx`.

PARTE 2

Proceso de Desarrollo de Software (PUDS)

1. Perfil

1.1. Introducción

El presente proyecto desarrolla una herramienta CASE web para el modelado colaborativo de diagramas de clases UML 2.5+ y la generación automática de backends Spring Boot. El caso de uso real es una empresa de desarrollo de software que necesita acelerar la construcción de sistemas de información con arquitectura en capas, evitando la codificación manual repetitiva de las capas Entity, Repository, Service y Controller a partir de un diagrama de clases ya validado por el equipo.

El sistema combina un lienzo de modelado colaborativo en tiempo real, un motor generador de código que transforma el modelo UML en un proyecto Spring Boot compilable, y un asistente de inteligencia artificial multimodal (texto, voz, imagen) que ayuda al usuario a construir el diagrama.

1.1.1 Introducción al Segundo Parcial

El segundo ciclo de desarrollo convierte la herramienta CASE en un **espacio de trabajo grupal moderado por inteligencia artificial**. En el primer ciclo, el asistente de IA era privado: cada usuario conversaba con él a solas. En este ciclo, la IA actúa como un **docente en el aula o un desarrollador senior** que acompaña a todo el equipo. Incorpora cinco capacidades:

1. **Chat grupal público:** todos los integrantes ven cada pregunta y cada respuesta.
2. **Moderador IA que responde por nombre:** *“Luis, aquí tu respuesta…”*, con contexto **grupal** (el estado del diagrama y quién hizo qué) e **individual** (las preguntas previas de cada integrante).
3. **Autoría granular persistida en MongoDB:** quién creó y quién modificó cada clase, atributo y relación, y desde qué fuente (manual, IA o XMI).
4. **Sesiones continuas** que se pausan y se reanudan otro día exactamente donde quedaron.
5. **Visor móvil en Flutter** que sigue el diagrama y el chat en tiempo real.

El motor de IA pasa a ser **híbrido**: Google Gemini en la nube y Ollama local como respaldo automático sin internet.

1.2 Descripción del problema

Los equipos de desarrollo dedican una parte significativa de su tiempo a escribir código repetitivo y mecánico: entidades JPA, repositorios, servicios CRUD y controladores REST que se derivan directamente de un diagrama de clases ya acordado. Este trabajo manual es propenso a errores de transcripción, inconsistente entre desarrolladores, y no deja un artefacto de diseño reutilizable una vez escrito el código. Además, cuando el modelado se hace en

equipo, la falta de herramientas colaborativas en tiempo real obliga a los desarrolladores a coordinar cambios de forma asíncrona, generando conflictos y demoras.

1.2.1 Descripción del problema — Segundo Parcial

En el sistema del primer ciclo, la IA ayudaba a un solo usuario por vez y no sabía quién preguntaba. Por eso respondía de forma genérica, y lo que aprendía un integrante no lo veían los demás. Además:

- **Autoría:** no se registraba quién había creado cada clase o atributo; la bitácora era un texto libre por guardado.
- **Continuidad:** el trabajo no podía retomarse con su contexto (conversación, presencia y bloqueos).
- **Móvil:** quien seguía el trabajo desde el celular no veía los cambios en vivo.
- **Conectividad:** la IA dependía de un único proveedor en la nube.

1.3 Objetivo General

Desarrollar una herramienta CASE web que permita a un equipo de desarrollo modelar diagramas de clases UML de forma colaborativa y generar automáticamente un backend Spring Boot funcional y compilable a partir de ese modelo.

1.3.1 Objetivo General — Segundo Parcial

Extender la herramienta CASE con un **Moderador IA grupal**. El moderador debe:

- responder en un chat público, nombrando a quien pregunta;
- usar el contexto grupal e individual del equipo;
- modificar el diagrama solo a pedido explícito.

El ciclo incluye además persistencia en MongoDB con autoría granular y sesiones continuas, un motor de IA híbrido (nube y local) y un visor móvil en tiempo real.

1.4 Objetivos Específicos

- Implementar un lienzo de modelado interactivo en React con la librería React Flow.
- Desarrollar un motor generador de código en NestJS que transforme el modelo UML en las cuatro capas de un proyecto Spring Boot.
- Integrar colaboración en tiempo real mediante WebSockets (Socket.io) para edición simultánea de diagramas.
- Conectar un asistente de inteligencia artificial (Google Gemini) para chat conversacional, dictado por voz e interpretación de fotografías de diagramas dibujados a mano.
- Implementar la importación y exportación del diagrama en formato estándar XMI 2.1 para interoperar con herramientas UML externas como Enterprise Architect.

- Incorporar autenticación de usuarios (registro e inicio de sesión) para identificar con nombre real a quien edita cada clase.
- Registrar una bitácora de cambios por diagrama que indique quién modificó qué y cuándo.
- Permitir adjuntar documentos e imágenes a un diagrama, respaldados en almacenamiento en la nube (Amazon S3).
- Soportar la declaración de clases abstractas e interfaces, reflejadas tanto en el diagrama como en el código Java generado.
- Habilitar la selección múltiple de clases y la exportación del diagrama como imagen (PNG/SVG).
- Desplegar la aplicación en un servidor accesible públicamente (AWS), verificando su funcionamiento fuera de un entorno local.
- Desarrollar un cliente móvil en Flutter que consuma el backend generado, con interacción por voz y operación sin conexión (offline) que sincroniza los cambios al reconectar.
- Habilitar el uso de modelos de inteligencia artificial locales, tanto en escritorio como en el dispositivo móvil, para asistir al usuario incluso sin conexión a internet.

1.4.1 Objetivos Específicos — Segundo Parcial

- Migrar la persistencia de los diagramas a **MongoDB**, conservando los datos del primer ciclo, con **autoría granular** por clase, atributo y relación, y una **bitácora estructurada** por cambio.
- Implementar **sesiones continuas** (ACTIVE / PAUSED) que restauren el chat y la actividad al volver a la sala.
- Identificar a cada integrante en la sala mediante **JWT en el handshake de Socket.io**.
- Construir un **chat grupal público** sobre WebSockets (`chat:send`, `chat:new-message`, `ai:status`).
- Diseñar el **System Prompt del Moderador / Desarrollador Senior**: rol, contexto dual y regla de nombrar a quien pregunta.
- Implementar un **motor de IA híbrido** (Gemini + Ollama) con respaldo automático, *circuit breaker* y orden configurable.
- Garantizar en el servidor que **la IA solo modifique el lienzo ante un pedido explícito**.
- Mostrar la **autoría en vivo** en el inspector de clases.
- Desarrollar el **visor UML en vivo** en Flutter, de solo lectura, con chat y **notificaciones locales**.
- Documentar los modelos del segundo ciclo en **UML 2.5.1** y mantener la trazabilidad entre modelo, código y prueba.

1.5 Alcance

Módulo de Diagramas

Gestiona la creación, edición, persistencia y carga compartida de diagramas UML.

Módulo Generador

Transforma el modelo UML en un proyecto Spring Boot completo (Entity, Repository, Service, Controller) y en un archivo XMI. El proyecto generado incluye además una colección Postman, el script de datos de prueba (data.sql; desde el Ciclo 2 el esquema lo crea el ORM), y la documentación Swagger de la API.

Módulo de Colaboración

Sincroniza los cambios del diagrama en tiempo real entre todos los usuarios conectados a la misma sesión de edición. Muestra la presencia de los usuarios conectados y aplica control de exclusión mediante bloqueo por clase: mientras un usuario edita una clase, esta queda bloqueada para los demás y se libera al terminar la edición.

Módulo de Inteligencia Artificial

Provee el chat conversacional, el reconocimiento de voz y la interpretación de fotografías dibujadas a mano. Puede operar con modelos de IA en la nube (Google Gemini) o con modelos locales, en escritorio y en el dispositivo móvil, para funcionar sin conexión a internet.

Módulo Cliente Móvil

Aplicación desarrollada en Flutter que consume el backend generado para probar sus endpoints. Ofrece interacción por voz, un asistente de inteligencia artificial local y operación offline que encola las acciones realizadas sin conexión y las sincroniza al recuperar la conectividad.

Módulo de Chat Grupal y Moderador IA (Segundo Ciclo)

Chat público de la sala del diagrama: todos ven cada mensaje. Las preguntas a la IA las responde el Moderador IA dirigiéndose por nombre a quien pregunta, con contexto grupal e individual, y muestra qué proveedor respondió (🏠 Ollama Local / ☁ Gemini Cloud). Si se le pide explícitamente, aplica cambios al diagrama, que quedan registrados como “vía IA”.

Módulo de Autoría, Bitácora y Sesiones (Segundo Ciclo)

Persistencia en MongoDB. Incluye:

- **autoría embebida** en cada clase, atributo y relación;
- **bitácora estructurada** con una entrada por cambio;
- **sesiones continuas** que guardan participantes y bloqueos, y se reanudan al volver.

Módulo de Motor de IA Híbrido (Segundo Ciclo)

Combina Google Gemini (nube) y Ollama con qwen2.5 (local) detrás de un mismo contrato. Por defecto responde Gemini, y Ollama toma el relevo sin internet. El orden puede invertirse para la demostración.

Módulo Visor Móvil en Tiempo Real (Segundo Ciclo)

Pantalla Flutter de solo lectura. Muestra en vivo el diagrama (con autoría y bloqueos) y el chat del equipo, y emite notificaciones locales de mensajes y cambios.

2. Captura De Requisitos

2.1 Identificar Actores Y Casos De Uso

En el primer ciclo, a diferencia de un sistema con roles y autenticación, la herramienta no distinguía perfiles de acceso: cualquier persona que abre la aplicación puede modelar. Se identifican tres actores:

- **Usuario:** actor principal. Modela el diagrama de clases, lo guarda, lo comparte, genera el backend y lo exporta.
- **Usuario Colaborador:** una instancia del mismo rol Usuario, conectada simultáneamente a la misma sesión de edición mediante un enlace compartido.
- **Asistente de IA:** actor secundario (sistema externo, Google Gemini) que procesa las consultas de texto, voz e imagen del Usuario.

2.1.1 Actores del Segundo Ciclo

Desde el segundo ciclo, la herramienta **sí identifica a cada usuario** (registro e inicio de sesión con JWT), porque el Moderador IA debe saber a quién responde y la autoría debe registrar quién hizo cada cambio. Los actores del ciclo 2 son:

- **Integrante del equipo:** el Usuario del ciclo 1, ahora identificado. Además de modelar, conversa en el chat grupal, consulta al Moderador IA, le pide cambios y revisa la autoría y la bitácora.
- **Integrante (móvil):** el mismo integrante, que sigue la sala desde el celular en modo de solo lectura.
- **Moderador IA:** actor secundario (Google Gemini o Ollama local) que responde en el chat grupal.

2.2 Priorizar Casos De Uso

NRO	CASO DE USO	ACTOR	ESTADO	PRIORIDAD	RIESGO
CU1	Crear Clase UML	Usuario	Aprobado	Alta	Crítico
CU2	Editar Clase UML	Usuario	Aprobado	Alta	Normal
CU3	Eliminar Clase UML	Usuario	Aprobado	Media	Normal
CU4	Crear Relación entre Clases	Usuario	Aprobado	Alta	Crítico
CU5	Guardar Diagrama	Usuario	Aprobado	Alta	Crítico
CU6	Cargar Diagrama Compartido	Usuario	Aprobado	Alta	Normal
CU7	Colaborar en Tiempo Real	Usuario / Usuario	Aprobado	Alta	Crítico

NRO	CASO DE USO	ACTOR	ESTADO	PRIORIDAD	RIESGO
		Colaborador			
CU8	Generar Backend Spring Boot	Usuario	Aprobado	Alta	Crítico
CU9	Exportar Diagrama a XMI	Usuario	Aprobado	Media	Normal
CU10	Consultar Asistente de IA	Usuario / IA	Aprobado	Media	Normal
CU11	Dictar Comando por Voz	Usuario	Aprobado	Baja	Normal
CU12	Interpretar Foto de Pizarra	Usuario / IA	Aprobado	Media	Normal

Casos de uso del Segundo Ciclo

NRO	CASO DE USO	ACTOR	ESTADO	PRIORIDAD	RIESGO
CU13	Iniciar Sesión	Integrante / Integrante (móvil)	Aprobado	Alta	Crítico
CU14	Enviar Mensaje al Chat Grupal	Integrante	Aprobado	Alta	Crítico
CU15	Consultar al Moderador IA	Integrante / Moderador IA	Aprobado	Alta	Crítico
CU16	Solicitar Cambio del Diagrama a la IA	Integrante / Moderador IA	Aprobado	Alta	Crítico
CU17	Consultar Autoría y Bitácora	Integrante	Aprobado	Alta	Normal
CU18	Reanudar Sesión de Trabajo	Integrante	Aprobado	Media	Normal
CU19	Visualizar Diagrama en Vivo (Móvil)	Integrante (móvil)	Aprobado	Alta	Normal
CU20	Recibir Notificaciones en el Móvil	Integrante (móvil)	Aprobado	Media	Normal

2.3 Detallar Caso De Uso

CU1 — Crear Clase UML

Propósito	Permitir al usuario agregar una nueva clase UML al lienzo de modelado.

Propósito	Permitir al usuario agregar una nueva clase UML al lienzo de modelado.
Resumen	El usuario hace clic en el botón "+ Clase". El sistema crea un nodo con un atributo identificador por defecto y lo posiciona en el lienzo.
Actores	Usuario
Actor Iniciador	Usuario

Flujo Principal:

1. El usuario hace clic en el botón "+ Clase" de la barra de herramientas.
2. El sistema genera un identificador único para la nueva clase.
3. El sistema crea un nodo "NuevaClase" con el atributo "id" marcado como clave primaria.
4. El nodo se posiciona en el lienzo en una ubicación libre.
5. El usuario puede arrastrar el nodo a la posición deseada.

Precondición: El usuario debe tener el lienzo abierto en el navegador.

CU2 — Editar Clase UML

Propósito	Permitir al usuario modificar el nombre y los atributos de una clase existente.
Resumen	El usuario hace doble clic sobre una clase. El sistema abre el panel inspector donde puede renombrar la clase y agregar, editar o eliminar atributos.
Actores	Usuario
Actor Iniciador	Usuario

Flujo Principal:

1. El usuario hace doble clic sobre el nodo de la clase.
2. El sistema abre el panel inspector con el nombre y los atributos actuales.
3. El usuario modifica el nombre, agrega un atributo, o cambia el tipo/visibilidad/clave primaria de uno existente.
4. El sistema actualiza el nodo en el lienzo en tiempo real con cada cambio.
5. El usuario cierra el inspector.

Precondición: Debe existir al menos una clase en el lienzo.

CU3 — Eliminar Clase UML

Propósito	Permitir al usuario eliminar una clase y sus relaciones asociadas del diagrama.
Resumen	Desde el panel inspector, el usuario solicita eliminar la clase. El sistema la remueve del lienzo junto con todas las relaciones que la involucraban.

Propósito	**Permitir al usuario eliminar una clase y sus relaciones asociadas del diagrama.**
Actores	Usuario
Actor Iniciador	Usuario

Flujo Principal:

1. El usuario abre el inspector de la clase a eliminar.
2. Hace clic en "Eliminar clase".
3. El sistema remueve el nodo del lienzo.
4. El sistema remueve todas las relaciones (aristas) conectadas a esa clase.
5. El panel inspector se cierra.

Precondición: La clase debe existir en el lienzo.

CU4 — Crear Relación entre Clases

Propósito	**Permitir al usuario conectar dos clases mediante una relación UML tipada.**
Resumen	El usuario selecciona un tipo de relación en la barra de herramientas y arrastra una conexión entre los conectores de dos clases.
Actores	Usuario
Actor Iniciador	Usuario

Flujo Principal:

1. El usuario selecciona el tipo de relación (asociación, agregación, composición, herencia, 1:1, 1:N, N:1, N:M) en el selector de la barra de herramientas.
2. El usuario arrastra desde el conector inferior de la clase origen hasta el conector superior de la clase destino.
3. El sistema crea la arista con una etiqueta correspondiente al tipo de relación seleccionado.
4. La relación queda visible conectando ambas clases.

Precondición: Deben existir al menos dos clases en el lienzo.

CU5 — Guardar Diagrama

Propósito	**Persistir el estado actual del diagrama en la base de datos.**
Resumen	El usuario hace clic en "Guardar diagrama". El sistema envía el modelo completo al backend, que lo almacena, y actualiza la URL con un identificador único para compartir.
Actores	Usuario

Propósito	Persistir el estado actual del diagrama en la base de datos.
Actor Iniciador	Usuario

Flujo Principal:

1. El usuario hace clic en "Guardar diagrama".
2. El sistema construye el modelo UML a partir del estado del lienzo (clases con posición, atributos, y relaciones).
3. Si es la primera vez, el sistema envía una petición de creación (POST); si el diagrama ya existía, envía una actualización (PATCH).
4. El backend persiste el modelo en PostgreSQL.
5. El sistema actualiza la URL del navegador con el identificador del diagrama guardado.

Precondición: El lienzo debe tener al menos el nombre del diagrama definido.

CU6 — Cargar Diagrama Compartido

Propósito	Permitir que un usuario abra un diagrama existente a partir de un enlace compartido.
Resumen	El usuario abre una URL con el identificador de un diagrama. El sistema recupera el modelo desde el backend y reconstruye el lienzo con la misma disposición de clases y relaciones.
Actores	Usuario
Actor Iniciador	Usuario

Flujo Principal:

1. El usuario abre una URL con el parámetro "?diagram=<id>".
2. El sistema solicita el diagrama al backend mediante ese identificador.
3. El backend responde con el modelo completo (clases, posiciones, relaciones).
4. El sistema reconstruye los nodos y aristas del lienzo respetando las posiciones guardadas.
5. El usuario puede continuar editando el diagrama desde ese estado.

Precondición: El identificador del diagrama debe existir en la base de datos.

CU7 — Colaborar en Tiempo Real

Propósito	Permitir que varios usuarios editen simultáneamente el mismo diagrama con sincronización instantánea.
Resumen	Al abrir un diagrama guardado, el sistema une al usuario a una sala de WebSocket asociada a ese diagrama. Los cambios de cualquier participante se retransmiten

Propósito	**Permitir que varios usuarios editen simultáneamente el mismo diagrama con sincronización instantánea.**
	automáticamente a los demás.
Actores	Usuario, Usuario Colaborador
Actor Iniciador	Usuario

Flujo Principal:

1. El usuario abre un diagrama guardado (con identificador asignado).
2. El sistema emite el evento "join-diagram" al servidor de WebSockets.
3. El servidor agrega al cliente a la sala correspondiente y notifica la cantidad de usuarios conectados.
4. Cuando el Usuario Colaborador modifica el diagrama, el cambio se emite al servidor.
5. El servidor retransmite el cambio a todos los demás clientes conectados a esa sala.
6. El lienzo del Usuario se actualiza automáticamente sin necesidad de recargar.

Precondición: El diagrama debe estar guardado (tener un identificador asignado).

CU8 — Generar Backend Spring Boot

Propósito	**Producir un proyecto Spring Boot completo y compilable a partir del modelo UML actual.**
Resumen	El usuario hace clic en "Generar backend Spring Boot". El sistema transforma cada clase en sus cuatro capas y entrega un archivo comprimido descargable.
Actores	Usuario
Actor Iniciador	Usuario

Flujo Principal:

1. El usuario hace clic en "Generar backend Spring Boot" (si el diagrama no está guardado, el sistema lo guarda automáticamente primero).
2. El sistema solicita al backend la generación del proyecto para ese diagrama.
3. El motor generador produce, por cada clase, los archivos Entity, Repository, Service y Controller, resolviendo las relaciones JPA en ambos extremos.
4. El sistema agrega los archivos de configuración del proyecto (pom.xml, application.properties, clase principal, configuración de serialización JSON).
5. El sistema empaqueta todo en un archivo .zip y lo descarga en el navegador del usuario.

Precondición: El diagrama debe tener al menos una clase.

CU9 — Exportar Diagrama a XMI

Propósito	**Exportar el modelo UML a formato estándar XMI para su uso en herramientas externas.**
Resumen	El usuario hace clic en "Exportar XMI". El sistema genera un archivo XML válido según el estándar XMI 2.1/UML 2.1.
Actores	Usuario
Actor Iniciador	Usuario

Flujo Principal:

1. El usuario hace clic en "Exportar XMI" (si el diagrama no está guardado, el sistema lo guarda automáticamente primero).
2. El sistema solicita al backend la exportación XMI de ese diagrama.
3. El backend construye el documento XMI: declara los tipos de dato usados, cada clase como packagedElement, cada atributo con su visibilidad, y cada relación como asociación con sus multiplicidades (o generalización, en herencia).
4. El sistema descarga el archivo .xmi en el navegador del usuario.

Precondición: El diagrama debe tener al menos una clase.

CU10 — Consultar Asistente de IA

Propósito	**Permitir al usuario resolver dudas de ingeniería de software mediante un chat conversacional.**
Resumen	El usuario escribe una pregunta en el panel de chat. El sistema la envía al modelo de IA con instrucciones que restringen sus respuestas al dominio de ingeniería de software.
Actores	Usuario, Asistente de IA
Actor Iniciador	Usuario

Flujo Principal:

1. El usuario escribe un mensaje en el campo de texto del panel de chat y hace clic en "Enviar".
2. El sistema envía el mensaje al backend.
3. El backend invoca al modelo de IA (Gemini) con instrucciones de sistema que restringen el dominio de respuesta.
4. El modelo genera una respuesta de texto.
5. El sistema muestra la respuesta en el panel de chat.

Precondición: Ninguna; el chat está disponible en todo momento.

CU11 — Dictar Comando por Voz

Propósito	**Permitir al usuario enviar un mensaje al asistente de IA mediante voz en lugar de escribirlo.**
Resumen	El usuario activa el micrófono del panel de chat. El navegador transcribe el habla a texto y lo envía automáticamente.
Actores	Usuario
Actor Iniciador	Usuario

Flujo Principal:

1. El usuario hace clic en el botón del micrófono.
2. El navegador solicita permiso de acceso al micrófono (si no se otorgó previamente).
3. El sistema activa el reconocimiento de voz nativo del navegador (Web Speech API) y muestra el estado "Escuchando…".
4. El usuario dicta su instrucción.
5. El navegador transcribe el audio a texto al finalizar la frase.
6. El sistema envía automáticamente el texto transcrito al mismo flujo de CU10 (Consultar Asistente de IA).

Precondición: El navegador debe soportar la Web Speech API (Chrome o Edge) y el usuario debe otorgar permiso de micrófono.

CU12 — Interpretar Foto de Pizarra

Propósito	**Permitir al usuario incorporar al lienzo un diagrama fotografiado en una pizarra o cuaderno.**
Resumen	El usuario sube o toma una fotografía de un diagrama dibujado a mano. El sistema la envía a la IA de visión, que devuelve un modelo estructurado y lo agrega al lienzo existente.
Actores	Usuario, Asistente de IA
Actor Iniciador	Usuario

Flujo Principal:

1. El usuario hace clic en "Foto de pizarra" y selecciona o captura una imagen.
2. El sistema codifica la imagen en base64 y la envía al backend.
3. El backend envía la imagen al modelo de IA junto con instrucciones para interpretar la notación UML dibujada a mano.
4. El modelo de IA devuelve las clases, atributos y relaciones detectadas en formato JSON.
5. El sistema genera nuevos identificadores para evitar colisiones con las clases existentes, y agrega las nuevas clases y relaciones al lienzo.

Precondición: Ninguna; si la imagen no contiene un diagrama reconocible, el sistema informa el error sin modificar el lienzo.

Casos de uso del Segundo Ciclo

CU13 — Iniciar Sesión

Propósito	**Identificar al integrante, para que la IA sepa a quién responde y la autoría registre quién hizo cada cambio.**
Resumen	El integrante ingresa su correo y contraseña. El sistema valida las credenciales y emite un JWT, que acompaña cada petición REST y el *handshake* del WebSocket.
Actores	Integrante, Integrante (móvil)
Actor Iniciador	Integrante

Flujo Principal:

1. El integrante abre la herramienta y completa correo y contraseña.
2. El sistema busca el usuario por correo y compara la contraseña con su hash bcrypt.
3. El sistema genera un JWT con el identificador y el nombre del usuario.
4. El cliente guarda el token y abre la herramienta.
5. Al entrar a un diagrama, el socket envía el JWT en el *handshake* y el servidor identifica al integrante en la sala.

Precondición: El integrante debe estar registrado.

CU14 — Enviar Mensaje al Chat Grupal

Propósito	**Permitir que el equipo converse en un canal público asociado al diagrama.**
Resumen	El integrante escribe un mensaje sin activar la IA. El sistema lo guarda y lo difunde a todos los conectados a la sala, en la web y en el móvil.
Actores	Integrante
Actor Iniciador	Integrante

Flujo Principal:

1. El integrante desactiva el botón de IA del panel de chat y escribe su mensaje.
2. El sistema guarda el diagrama actual (para que la IA vea el lienzo vigente si se le consulta luego).
3. El cliente emite `chat:send` con `askAi: false`.
4. El servidor identifica al autor por su JWT y guarda el mensaje en `group_chat_messages`.
5. El servidor emite `chat:new-message` a toda la sala.

6. Todos los integrantes ven el mensaje con nombre, avatar y hora.

Precondición: El integrante inició sesión (CU13) y está en la sala de un diagrama.

CU15 — Consultar al Moderador IA

Propósito	**Resolver dudas teóricas o sobre el diagrama del equipo con un mentor que responde por nombre y con contexto.**
Resumen	El integrante pregunta con la IA activada. La pregunta se publica en la sala y el Moderador IA responde dirigiéndose a quien preguntó. Usa el estado del diagrama con su autoría, la bitácora, la conversación de la sala y el hilo individual del integrante.
Actores	Integrante, Moderador IA
Actor Iniciador	Integrante

Flujo Principal:

1. El integrante escribe o dicta su pregunta con la IA activada.
2. El sistema publica la pregunta en la sala (incluye CU14).
3. El sistema emite `ai:status` y todos ven *"La IA está pensando la respuesta para …"*.
4. El servidor reúne el contexto dual: diagrama con autoría, bitácora, chat de la sala e hilo del integrante.
5. El motor híbrido consulta a Gemini y, si no hay internet o falla, a Ollama local.
6. El sistema garantiza que la respuesta empiece con el nombre del integrante y la guarda con el proveedor usado.
7. El sistema difunde la respuesta a toda la sala con el badge del proveedor.

Flujo alternativo (sin internet): en el paso 5, Gemini falla y responde Ollama (punto de extensión *Responder con IA local*).

Precondición: El integrante inició sesión y está en la sala.

CU16 — Solicitar Cambio del Diagrama a la IA

Propósito	**Permitir que el Moderador IA, como desarrollador senior, aplique cambios al diagrama compartido cuando se le piden explícitamente.**
Resumen	El integrante pide un cambio (*"agrega la clase Producto y relaciónala muchos a muchos con Pedido"*). La IA propone operaciones y el servidor las valida. Solo el cliente de quien pidió el cambio las aplica, y se guardan con autoría "vía IA".
Actores	Integrante, Moderador IA
Actor Iniciador	Integrante

Flujo Principal:

1. El integrante escribe un pedido con un verbo de cambio (crea, agrega, elimina, relaciona…).

2. Se ejecuta CU15 y la IA devuelve una respuesta y una lista de operaciones.

3. El servidor descarta operaciones inválidas (clases inexistentes o ya existentes) y verifica que el mensaje sea un pedido explícito.

4. La respuesta llega a toda la sala; el cliente de quien pidió el cambio aplica las operaciones al lienzo.

5. El cambio se sincroniza a todos y se guarda con `source: AI`.

6. La autoría registra *"Creada por Luis · vía IA"* y la bitácora agrega la entrada *"(vía IA)"*.

Precondición: El integrante inició sesión y está en la sala.

CU17 — Consultar Autoría y Bitácora

Propósito	**Saber quién creó o modificó cada elemento del diagrama y cuándo.**
Resumen	El inspector de clases muestra la autoría actualizada en vivo. El panel Historial lista la bitácora estructurada del diagrama.
Actores	Integrante
Actor Iniciador	Integrante

Flujo Principal:

1. El integrante abre una clase con doble clic.

2. El inspector muestra *"Creada por …"*, *"Última edición …"*, la fuente ("vía IA" o importado de XMI) y el autor de cada atributo.

3. El integrante abre el panel Historial.

4. El sistema devuelve las últimas 50 entradas de la bitácora: usuario, acción y resumen.

5. Tras cada guardado, el servidor emite `diagram:authorship` y el inspector se actualiza sin recargar.

Precondición: El diagrama fue guardado al menos una vez.

CU18 — Reanudar Sesión de Trabajo

Propósito	**Continuar el trabajo otro día exactamente donde quedó.**
Resumen	Al entrar a la sala, el sistema reanuda la misma sesión (de PAUSED a ACTIVE) y envía el chat y la actividad recientes.
Actores	Integrante
Actor Iniciador	Integrante

Flujo Principal:

1. El integrante abre un diagrama.
2. El cliente emite `join-diagram` con su JWT.
3. El servidor busca la última sesión no cerrada: si estaba PAUSED la reactiva y, si no existe, la crea.
4. El servidor envía `session:init-state` con la sesión, la bitácora reciente y el chat.
5. El panel de chat muestra la conversación anterior.
6. Cuando sale el último integrante, la sesión vuelve a PAUSED.

Precondición: El integrante inició sesión (CU13).

CU19 — Visualizar Diagrama en Vivo (Móvil)

Propósito	**Seguir desde el celular, en solo lectura, el diagrama y el chat del equipo.**
Resumen	El integrante abre el visor UML de la app Flutter, elige un diagrama y lo ve actualizarse en vivo, con la autoría de cada clase y el chat con la IA.
Actores	Integrante (móvil)
Actor Iniciador	Integrante (móvil)

Flujo Principal:

1. El integrante toca “Visor UML en vivo” e inicia sesión (CU13).
2. El sistema lista los diagramas del equipo y el integrante elige uno.
3. El visor descarga el diagrama y se une a la sala por WebSocket.
4. Cuando alguien cambia el diagrama en la web, el visor lo redibuja y resalta las clases modificadas.
5. El integrante toca una clase para ver su autoría o cambia a la pestaña Chat.

Precondición: La app está instalada y el integrante tiene una cuenta.

CU20 — Recibir Notificaciones en el Móvil

Propósito	**Avisar al integrante de la actividad del equipo aunque no esté mirando la pantalla.**
Resumen	El visor muestra notificaciones locales cuando llega un mensaje de otro integrante, una respuesta de la IA o un cambio en el diagrama.
Actores	Integrante (móvil)
Actor Iniciador	Sistema (evento de la sala)

Flujo Principal:

1. Llega un `chat:new-message` de otro integrante o del Moderador IA, y el visor muestra la notificación *“Moderador IA → Luis”*.

2. Llega un `diagram-update` con el nombre de quien editó, y el visor notifica *“Luis modificó Pedido”*, como máximo un aviso cada 10 segundos.

Precondición: El visor está abierto en un diagrama (CU19) y el integrante concedió el permiso de notificaciones.

2.4 Estructurar Modelo De Caso De Uso

El diagrama general de casos de uso muestra los doce casos de uso del sistema, todos accesibles por el actor Usuario (sin restricción de rol), con el caso Colaborar en Tiempo Real compartido con el actor Usuario Colaborador, y los casos Consultar Asistente de IA e Interpretar Foto de Pizarra extendidos hacia el actor secundario Asistente de IA.

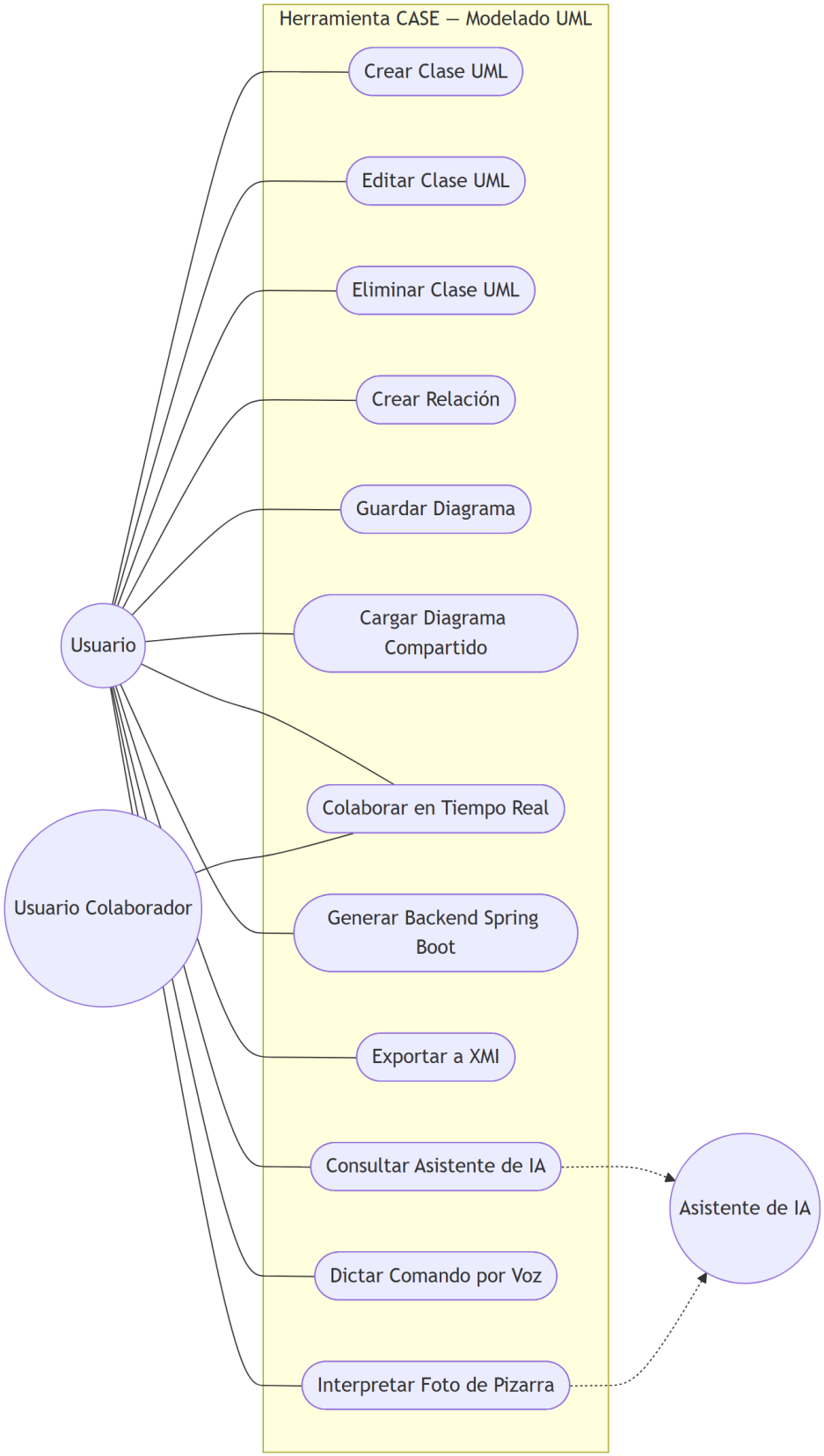

Herramienta CASE — Modelado UML
Crear Clase UML
Editar Clase UML
Eliminar Clase UML
Crear Relación
Guardar Diagrama
Cargar Diagrama Compartido
Colaborar en Tiempo Real
Generar Backend Spring Boot
Exportar a XMI
Consultar Asistente de IA
Dictar Comando por Voz
Interpretar Foto de Pizarra
Usuario
Usuario Colaborador
Asistente de IA

Figura 1. Diagrama General de Casos de Uso — Vista completa del sistema

Para el segundo ciclo, el modelo de casos de uso se amplía con dos paquetes nuevos:

- **Sala grupal con Moderador IA:** CU13–CU18.
- **Visor móvil:** CU19–CU20.

Entre los casos de uso se establecen estas relaciones:

- CU15 **incluye** a CU14 (toda consulta se publica en el chat).
- CU16 **extiende** a CU15.
- *Responder con IA local* **extiende** a CU15 (sin internet).
- CU18 y CU19 **incluyen** a CU13.
- CU20 **extiende** a CU19.

El actor secundario Moderador IA participa en CU15 y CU16.

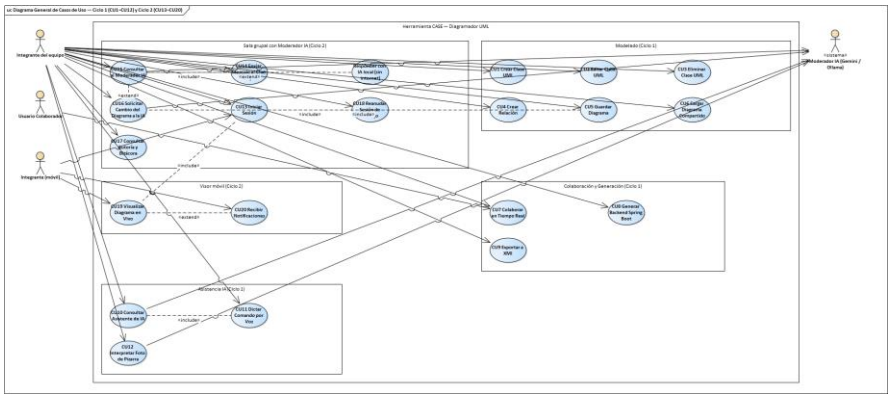


Figura — Diagrama General de Casos de Uso, Ciclos 1 y 2 (UML 2.5.1)

3. Análisis

3.1 Análisis De Arquitectura

3.1.1 Identificar Paquetes

El sistema se organiza en tres paquetes principales dentro del backend de la herramienta (diagrams, generator, ai), consumidos por dos módulos del frontend (el lienzo y el panel de chat). Cada paquete agrupa los casos de uso relacionados y define un subsistema con responsabilidades bien delimitadas.

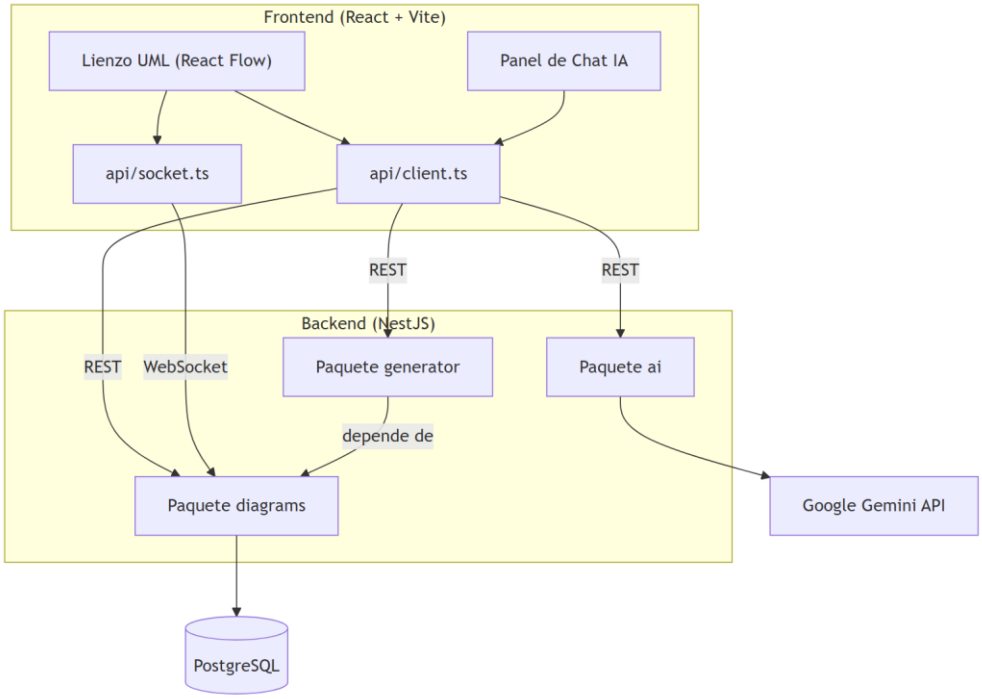


Figura 2. Diagrama de Paquetes del Sistema

En el segundo ciclo se agregan los paquetes **chat grupal** (frontend) y **live** (visor de la app móvil). También se amplían los paquetes **diagrams** (autoría, bitácora y sesiones) y **ai** (motor híbrido, moderador y gateway del chat). Las dependencias siguen siendo unidireccionales: **ai** depende de **diagrams** para leer el modelo y la bitácora, y ambos dependen de **auth** para identificar al usuario.

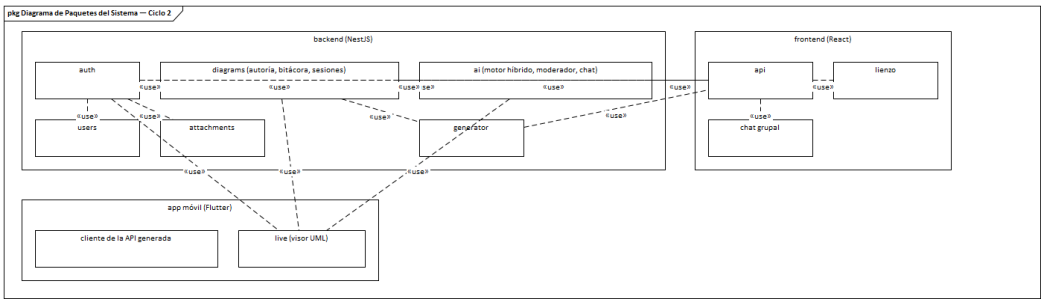


Figura — Diagrama de Paquetes del Sistema, Ciclo 2

3.2. Análisis de caso de uso — Diagramas de Comunicación

Los diagramas de comunicación muestran las interacciones entre objetos boundary (interfaz), control (lógica) y entity (datos) siguiendo el patrón ECB de PUDS. Los mensajes están numerados para indicar el orden de las interacciones.

CU1 — Crear Clase UML

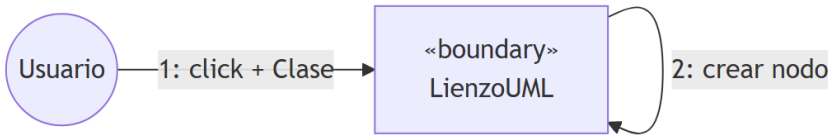


Figura — Diagrama de Comunicación CU1

CU2 — Editar Clase UML

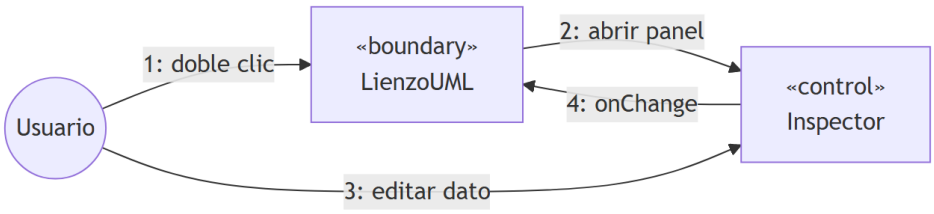


Figura — Diagrama de Comunicación CU2

CU3 — Eliminar Clase UML



Figura — Diagrama de Comunicación CU3

CU4 — Crear Relación entre Clases

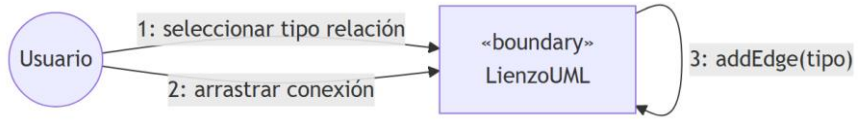


Figura — Diagrama de Comunicación CU4

CU5 — Guardar Diagrama

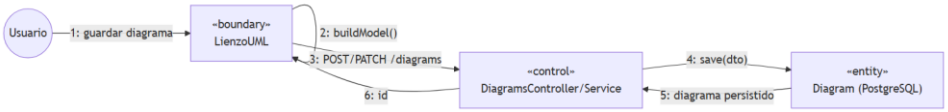


Figura — Diagrama de Comunicación CU5

CU6 — Cargar Diagrama Compartido

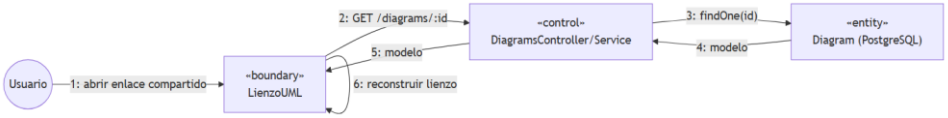


Figura — Diagrama de Comunicación CU6

CU7 — Colaborar en Tiempo Real

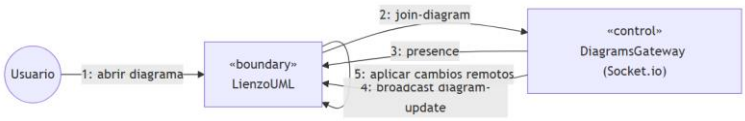


Figura — Diagrama de Comunicación CU7

CU8 — Generar Backend Spring Boot

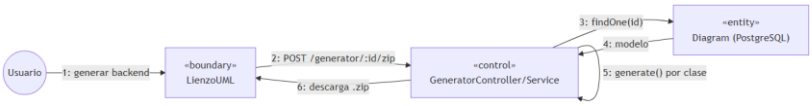


Figura — Diagrama de Comunicación CU8

CU9 — Exportar Diagrama a XMI

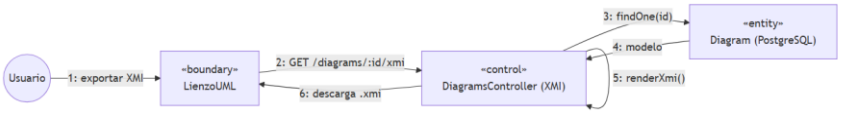


Figura — Diagrama de Comunicación CU9

CU10 — Consultar Asistente de IA



Figura — Diagrama de Comunicación CU10

CU11 — Dictar Comando por Voz

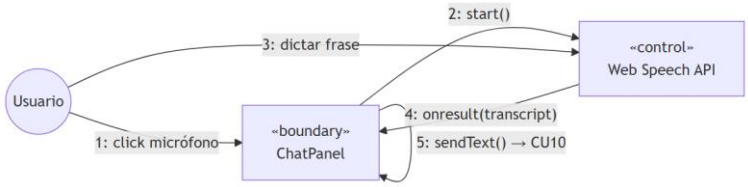


Figura — Diagrama de Comunicación CU11

CU12 — Interpretar Foto de Pizarra

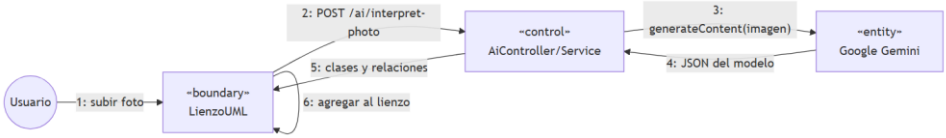


Figura — Diagrama de Comunicación CU12

CU13 — Iniciar Sesión

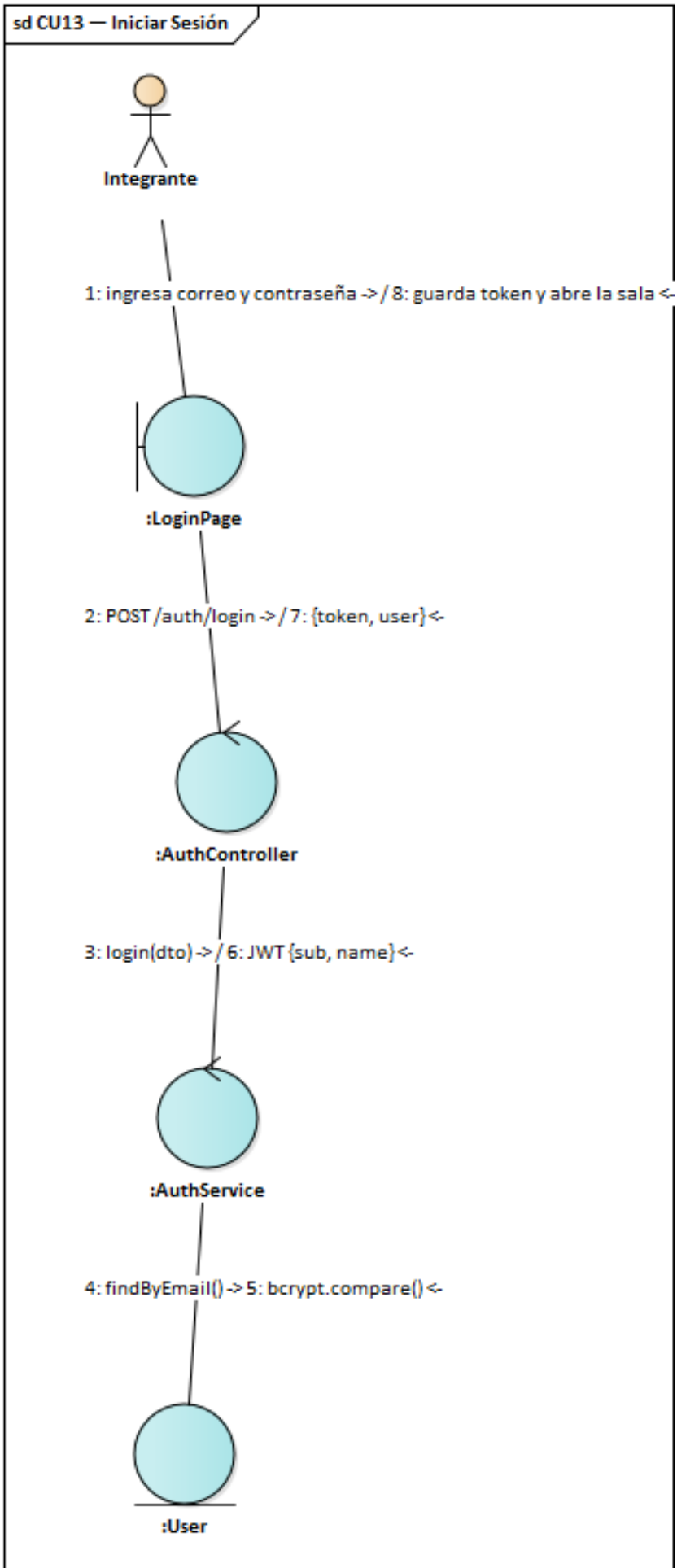

sd CU13 — Iniciar Sesión
Integrante
1: ingresa correo y contraseña -> / 8: guarda token y abre la sala <-
:LoginPage
2: POST /auth/login -> / 7: {token, user} <-
:AuthController
3: login(dto) -> / 6: JWT {sub, name} <-
:AuthService
4: findByEmail() -> 5: bcrypt.compare() <-
:User

Figura — Diagrama de Comunicación CU13

CU14 — Enviar Mensaje al Chat Grupal

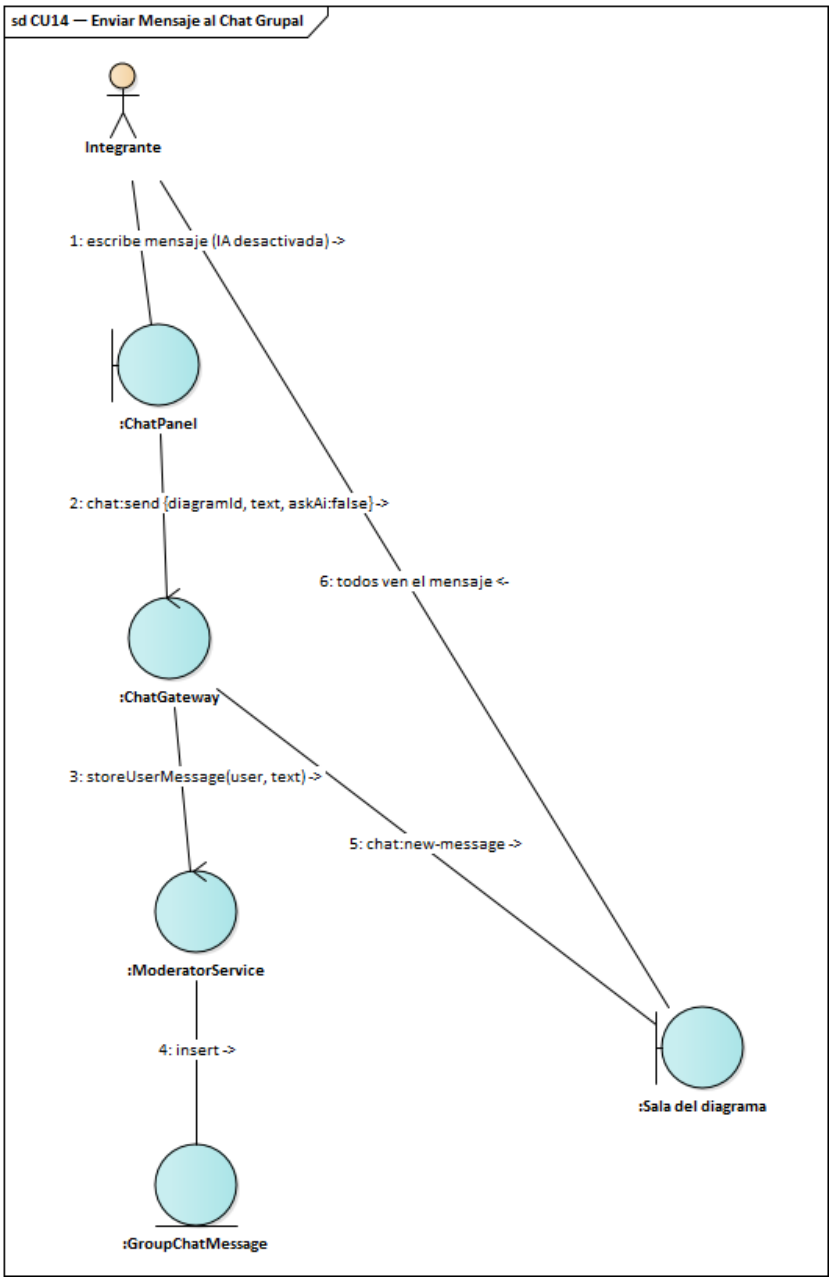


Figura — Diagrama de Comunicación CU14

CU15 — Consultar al Moderador IA

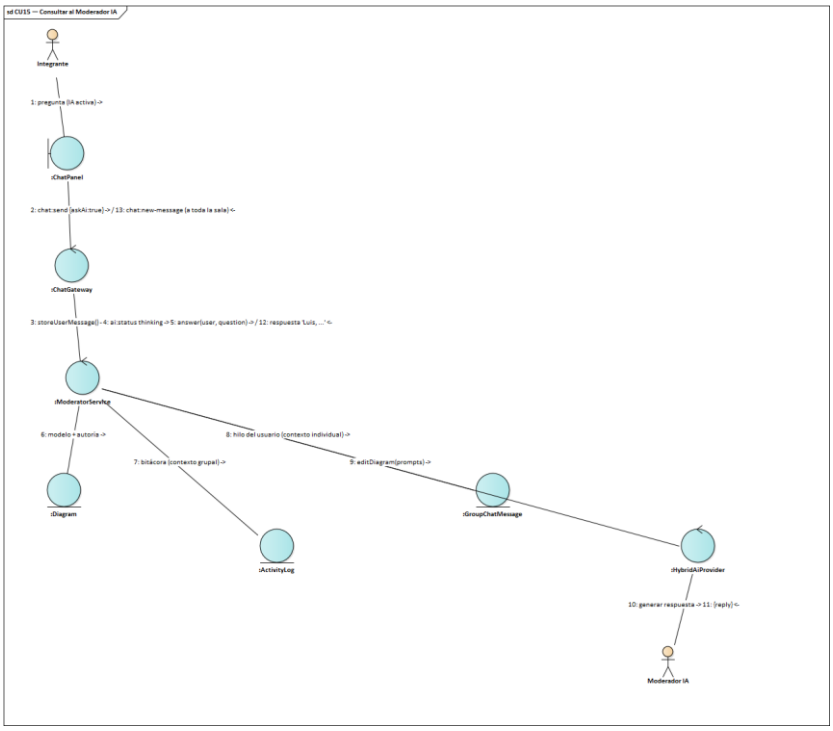


Figura — Diagrama de Comunicación CU15

CU16 — Solicitar Cambio del Diagrama a la IA

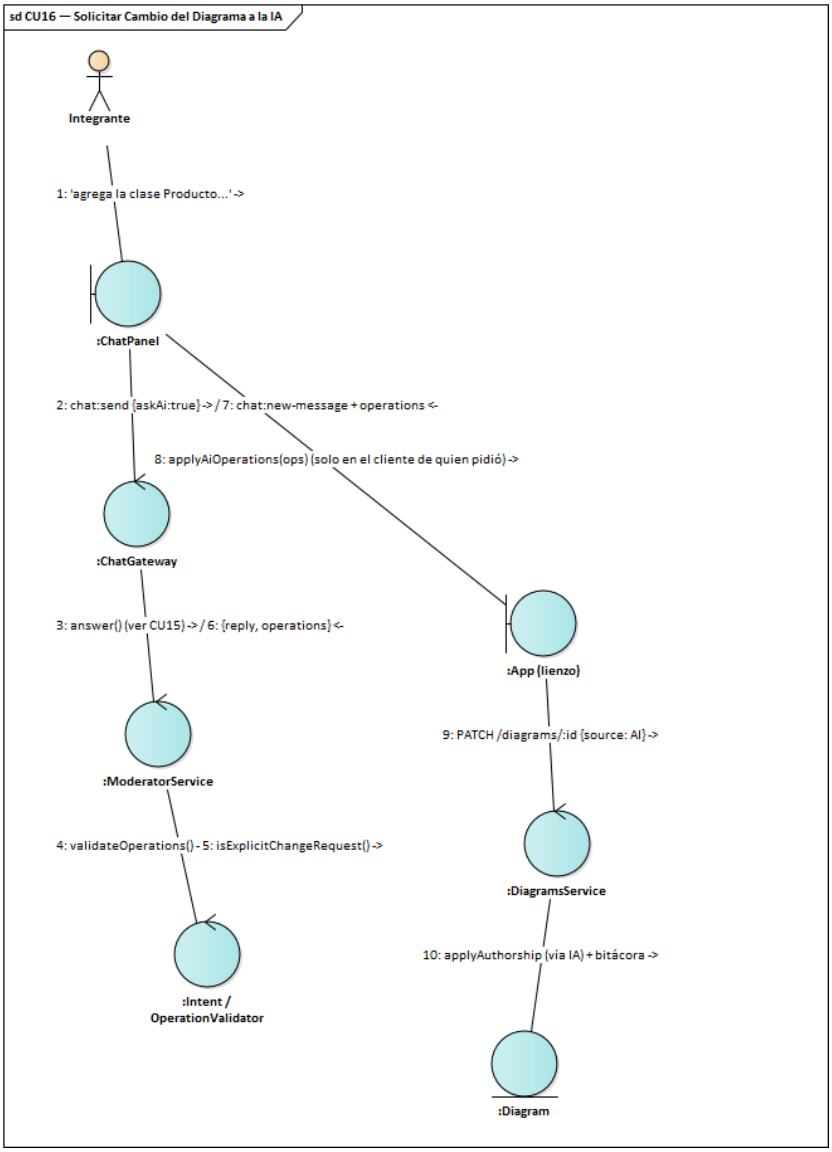


Figura — Diagrama de Comunicación CU16

CU17 — Consultar Autoría y Bitácora

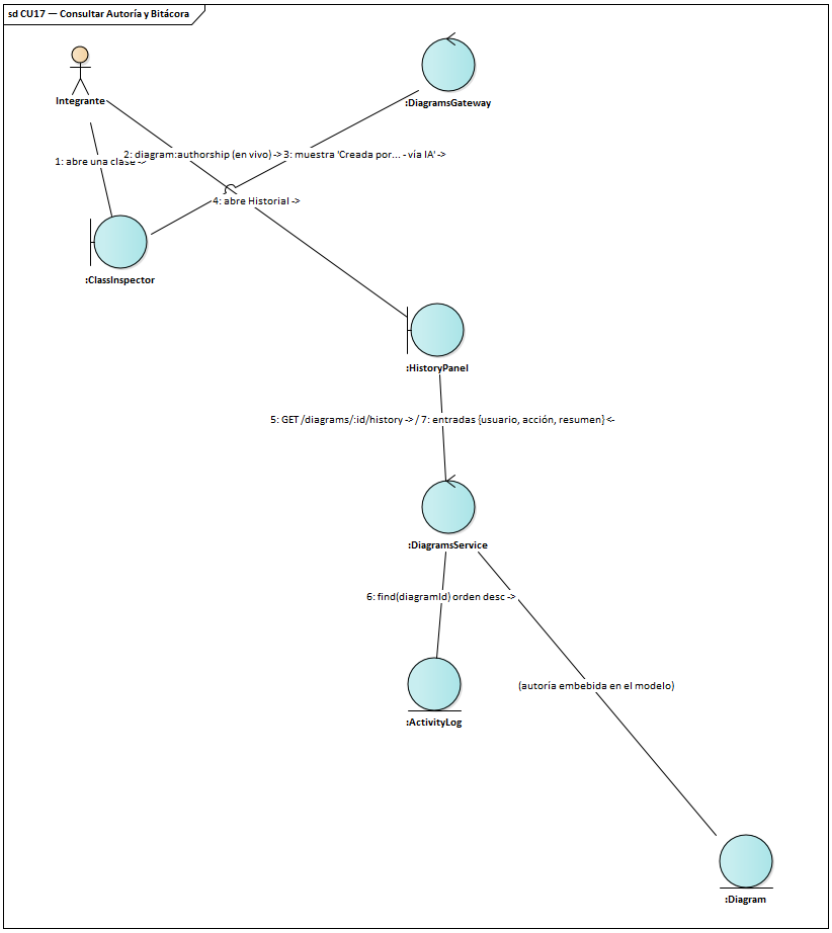


Figura — Diagrama de Comunicación CU17

CU18 — Reanudar Sesión de Trabajo

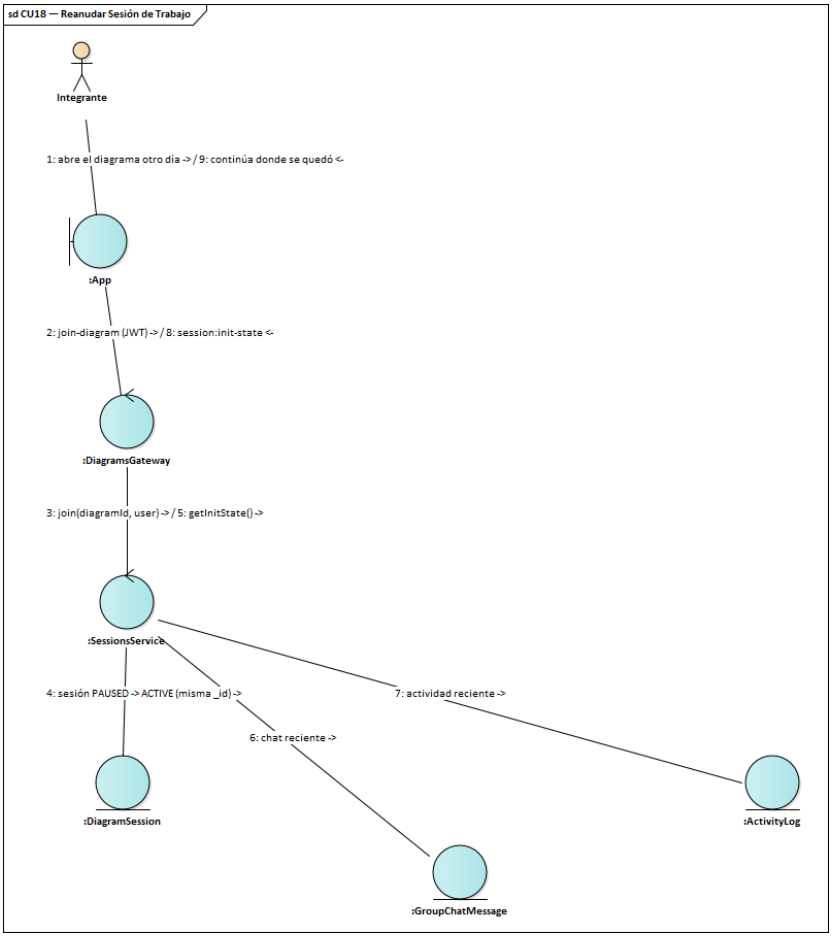


Figura — Diagrama de Comunicación CU18

CU19 — Visualizar Diagrama en Vivo (Móvil)

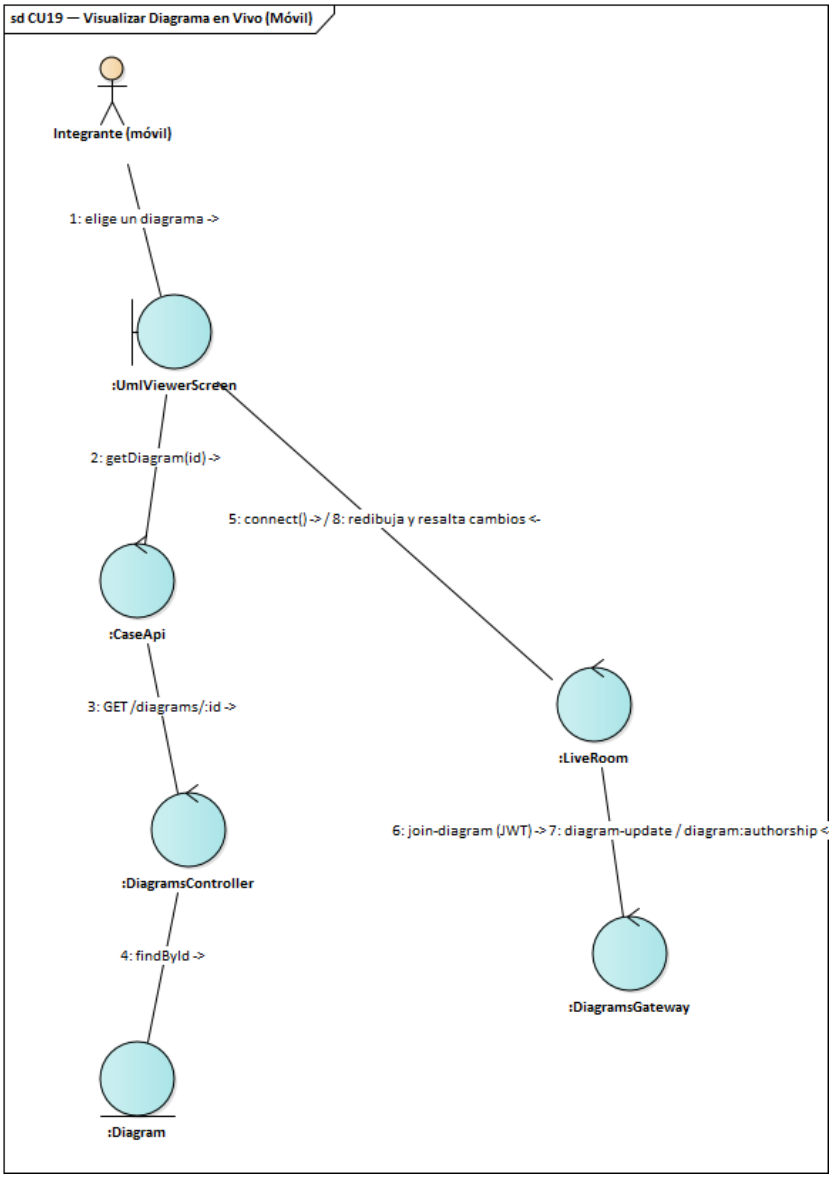


Figura — Diagrama de Comunicación CU19

CU20 — Recibir Notificaciones en el Móvil

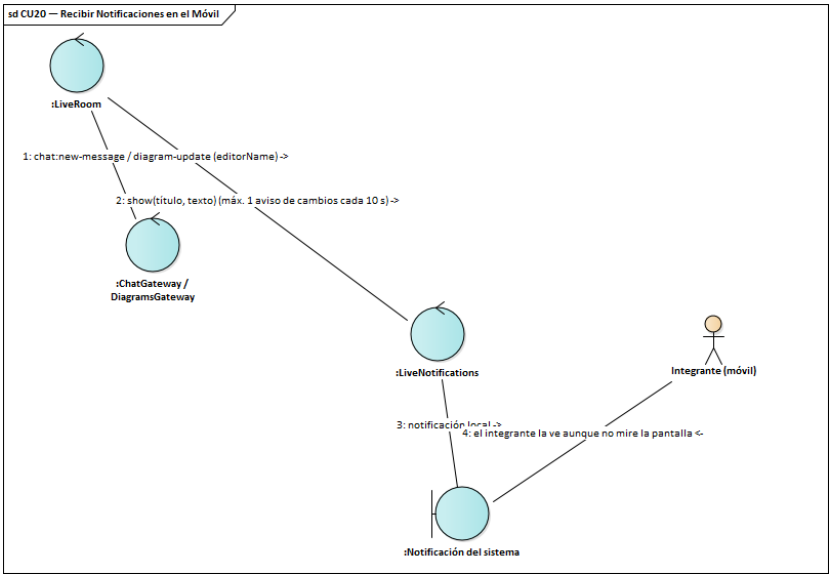


Figura — Diagrama de Comunicación CU20

3.3. Análisis de clase

Las clases de análisis siguen el patrón ECB: clases boundary (interfaz con el usuario), control (lógica de negocio) y entity (datos persistentes).

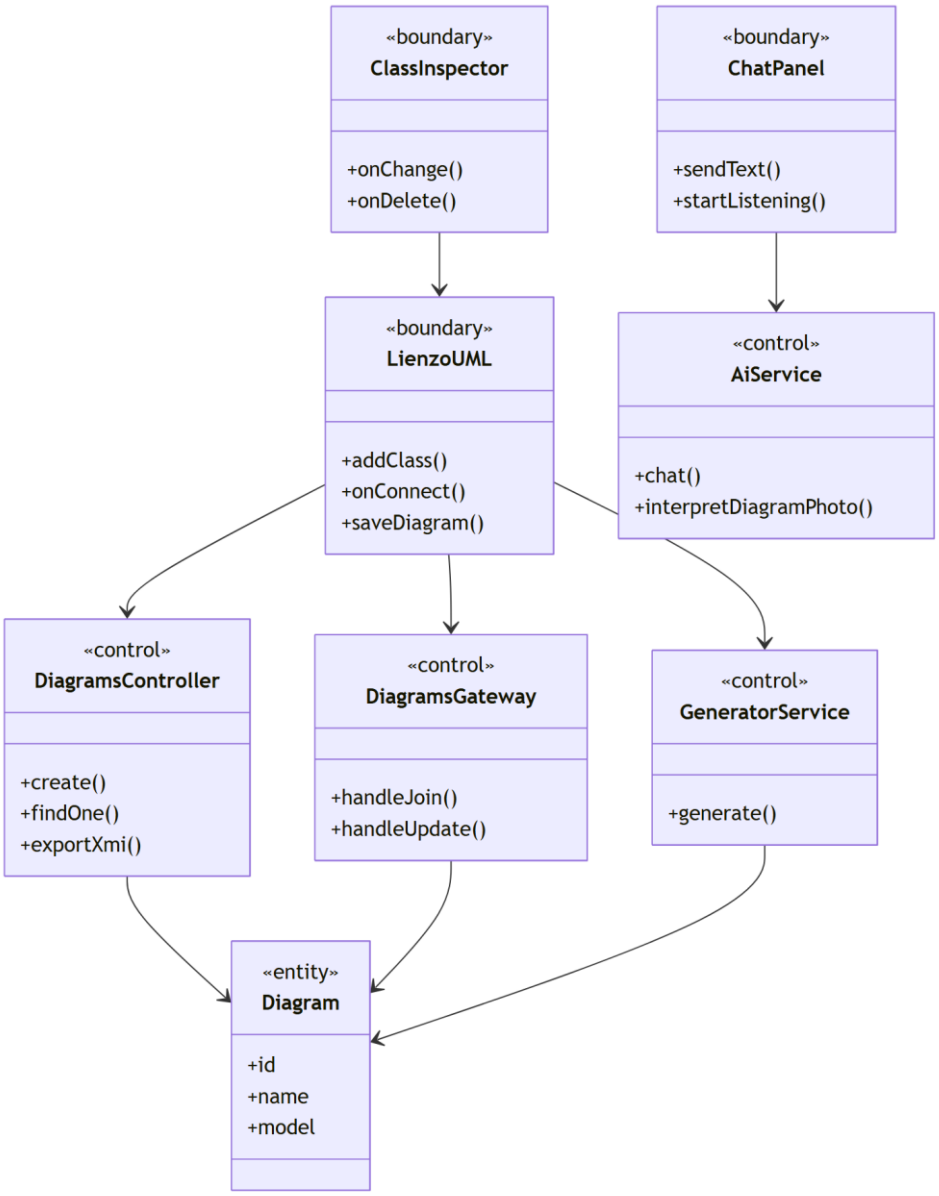


Figura 15. Clases de Análisis — Patrón Boundary-Control-Entity del sistema

Clases de análisis del Segundo Ciclo

Boundary:

- `LoginPage`
- `ChatPanel`
- `ClassInspector` (ahora con la sección de autoría)
- `HistoryPanel`
- `UmlViewerScreen` (móvil)

Control:

- Los gateways WebSocket del chat y de la sala.
- El Moderador IA, el motor híbrido y los servicios de diagramas y de sesiones.
- `LiveRoom`, que conecta el móvil a la sala.

Entity:

- Las colecciones de MongoDB: `Diagram` con su `AuthorStamp` embebido, `ActivityLog`, `DiagramSession` y `GroupChatMessage`.
- `User` en PostgreSQL.

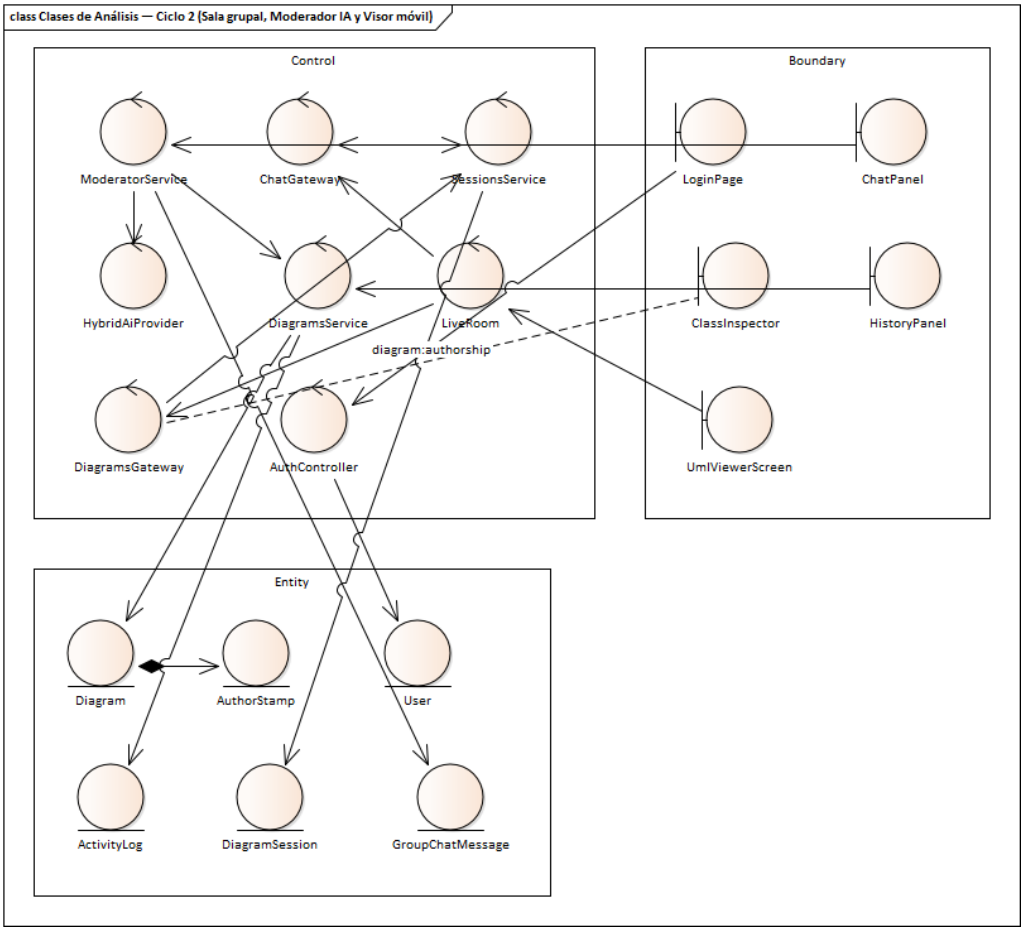


Figura — Clases de Análisis, Ciclo 2 (Boundary-Control-Entity)

3.4. Análisis de paquete

El diagrama de paquetes (Figura 2) organiza la arquitectura lógica en tres paquetes de backend con dependencias unidireccionales que minimizan el acoplamiento: el paquete generator depende del paquete diagrams para obtener el modelo a transformar, mientras que el paquete ai es independiente y solo depende del proveedor externo de IA.

En el segundo ciclo, el paquete **ai** deja de ser independiente: depende de **diagrams** para construir el contexto grupal del moderador (modelo con autoría y bitácora) y de **auth** para identificar al usuario en el socket. La dependencia es unidireccional: **diagrams** no conoce a **ai**. Por eso el gateway del chat vive en el módulo **ai** y comparte el mismo servidor Socket.io que la colaboración, sin crear un ciclo de dependencias.

4. Diseño

4.1 Diseño de arquitectura

4.1.1 Diseño físico de arquitectura (Diagrama de Despliegue)

El sistema se despliega como dos servicios independientes: el frontend (aplicación React servida al navegador) y el backend (Node.js/NestJS), con PostgreSQL como servidor de base de datos y Google Gemini como servicio de IA en la nube.

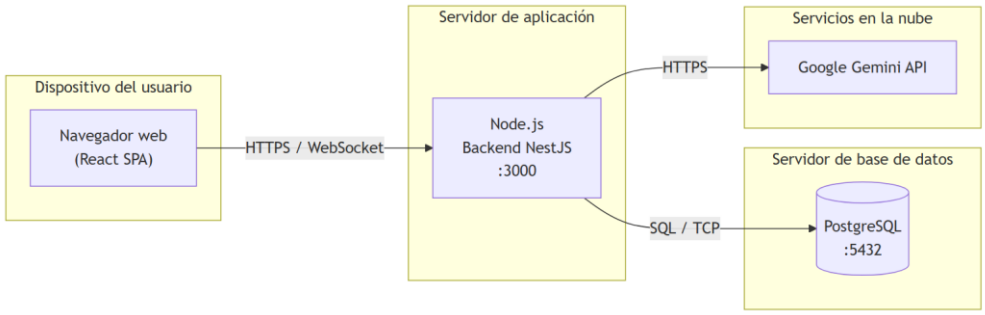


Figura 16. Diagrama de Despliegue del Sistema

Despliegue — Segundo Ciclo. Se agrega un cuarto contenedor, **MongoDB 7** con el volumen `mongo_data`, junto a PostgreSQL 16, el backend NestJS y el frontend (nginx con TLS de Let's Encrypt). nginx enruta además `/sessions`. El backend consulta Gemini en la nube y, como respaldo, un Ollama local (`docker-compose.local.yml` para la defensa sin internet). El celular se conecta por HTTPS/WSS al mismo dominio.

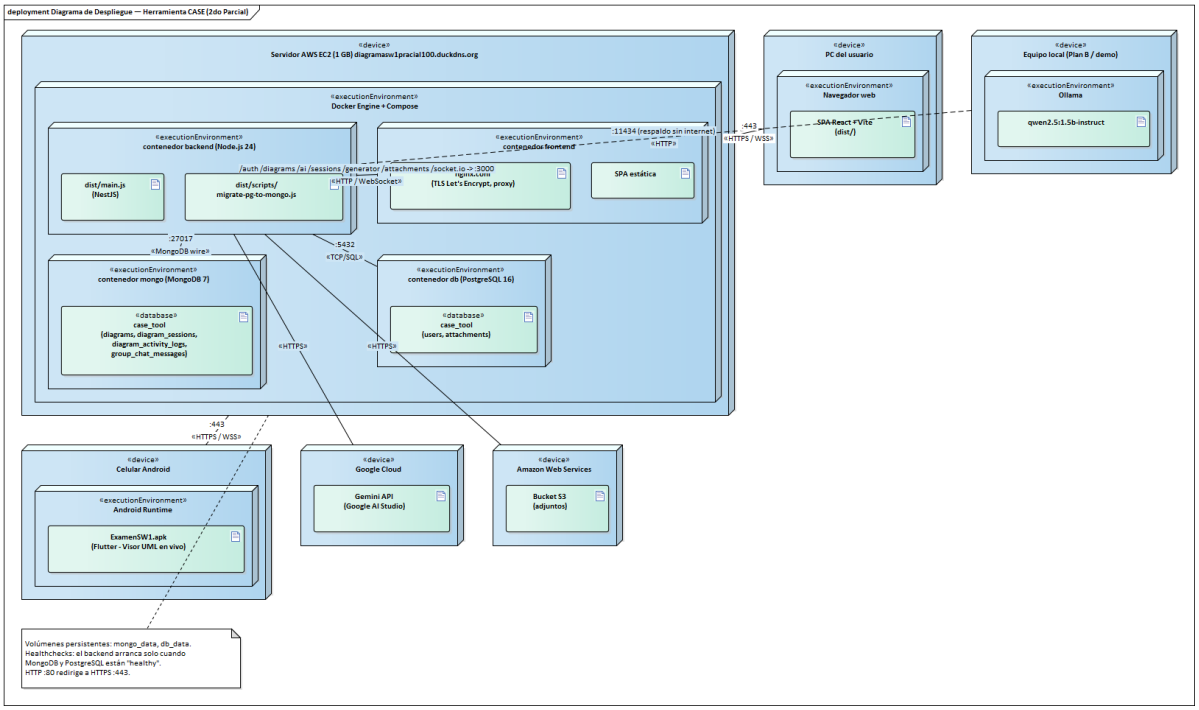


Figura — Diagrama de Despliegue, Ciclo 2 (UML 2.5.1)

4.1.2 Diseño lógico de arquitectura

La arquitectura lógica sigue el patrón de tres capas: Presentación (React SPA), Lógica de Negocio (NestJS: diagrams, generator, ai) y Persistencia (PostgreSQL). El frontend se comunica con el backend mediante HTTP REST y WebSockets (ver Figura 2).

Diseño lógico — Segundo Ciclo. El diagrama de componentes muestra las interfaces provistas por el backend (REST y eventos Socket.io de la sala) y quién las requiere: el frontend web (lienzo, chat e inspector) y la app móvil (visor).

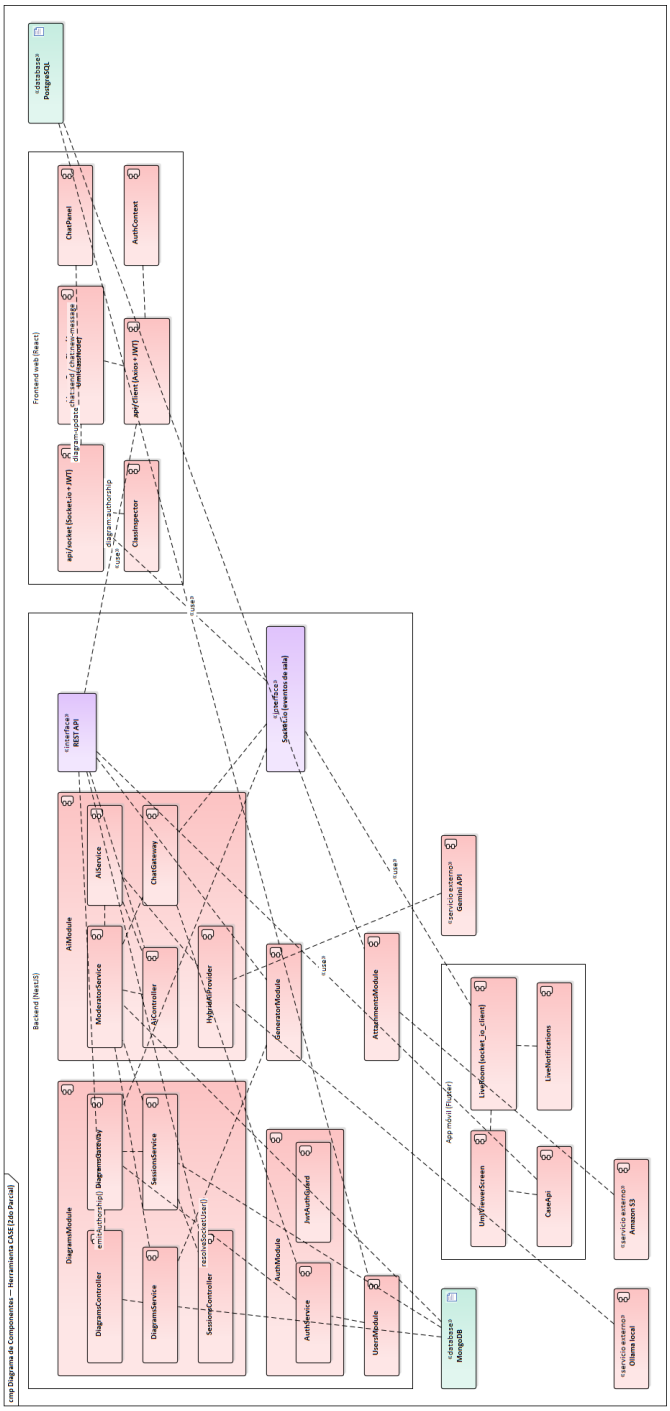


Figura — Diagrama de Componentes, Ciclo 2

4.2 Diseño de datos lógicos

El modelo de datos de la herramienta utiliza PostgreSQL. La entidad principal es:

- **Diagram:** id (UUID), name (String), model (JSONB — contiene las clases con sus atributos y posiciones, y las relaciones entre ellas), createdAt, updatedAt.

Se optó deliberadamente por una única entidad con un campo JSONB para el modelo UML, en lugar de tablas normalizadas por clase/atributo/relación: el modelo UML del usuario es un documento auto-contenido que se lee y escribe siempre completo (no se consulta una clase individual fuera de su diagrama), por lo que JSONB ofrece flexibilidad de esquema sin sacrificar las garantías transaccionales de PostgreSQL.

Diseño de datos lógicos — Segundo Ciclo. Se adopta **persistencia políglota**:

- **MongoDB:** guarda lo que tiene forma de árbol o crece con el uso.
    - `diagrams`: el modelo UML con la autoría **embebida** en cada clase, atributo y relación; `_id` UUID para conservar los identificadores migrados.
    - `diagram_sessions`: estado ACTIVE/PAUSED, participantes y bloqueos activos.
    - `diagram_activity_logs`: bitácora, una entrada por cambio, con actor, acción y destino.
    - `group_chat_messages`: chat público; las respuestas de la IA registran a quién van dirigidas (`addressedTo`) y el proveedor usado.
- **PostgreSQL:** conserva lo relacional: `users` (correo único) y `attachments`.

Las referencias entre colecciones son por identificador, sin claves foráneas, para que la bitácora y el chat sobrevivan aunque se elimine el diagrama.

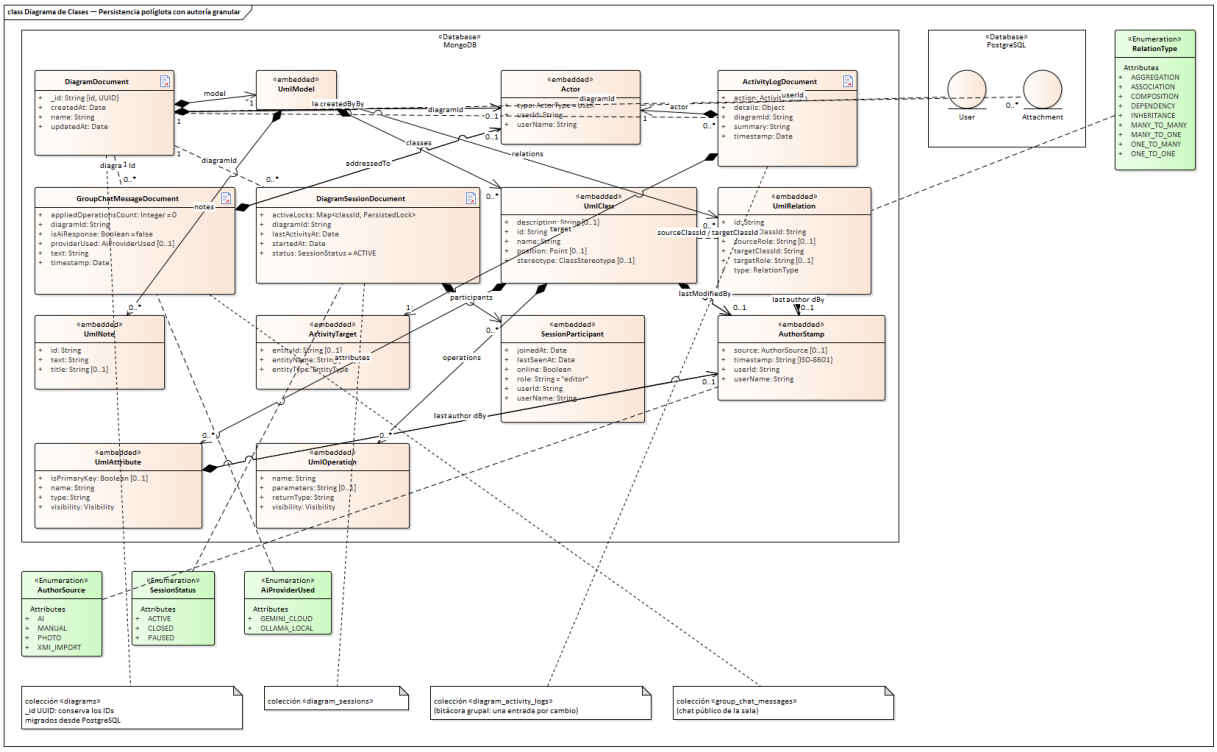


Figura — Diagrama de Clases de Persistencia, Ciclo 2

Diseño de clases del motor de IA. Se aplica el patrón **Strategy**: `AiProvider` con dos implementaciones (`GeminiProvider` y `OllamaProvider`), compuestas por `HybridAiProvider`, que agrega el orden configurable, los timeouts y el *circuit breaker*.

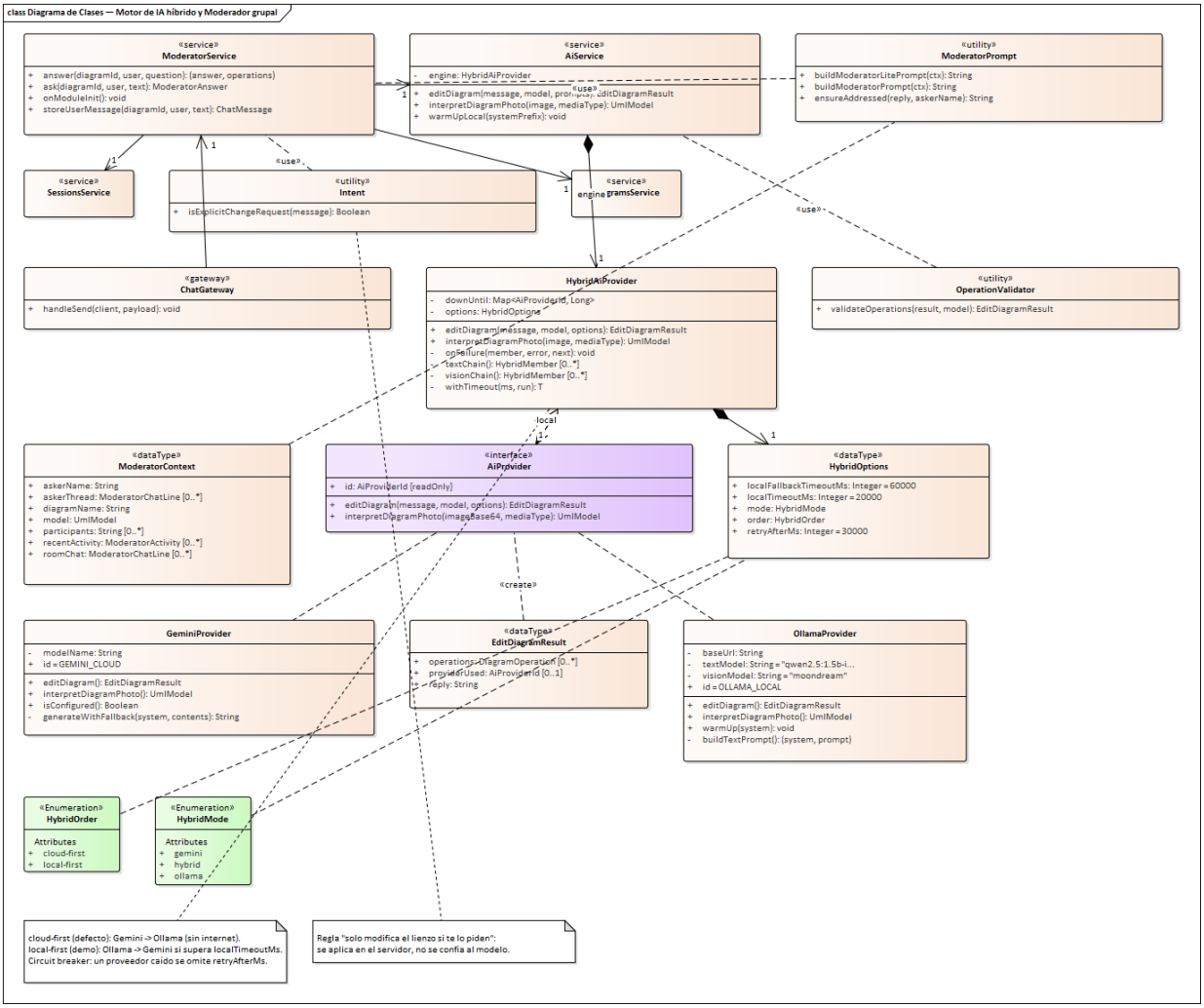


Figura — Diagrama de Clases del Motor de IA híbrido y el Moderador

Diseño de clases de la sala colaborativa. Reúne las clases de la web, del servidor y del visor móvil que aparecen como objetos en los diagramas de secuencia. Cada operación es un mensaje que esa clase recibe en alguna secuencia, y cada dependencia «use» une a quien envía con quien recibe. El diagrama se genera a partir de las secuencias (`docs/diagrams/ea/generar_clases_sala.py`), así que los dos modelos no pueden contradecirse.

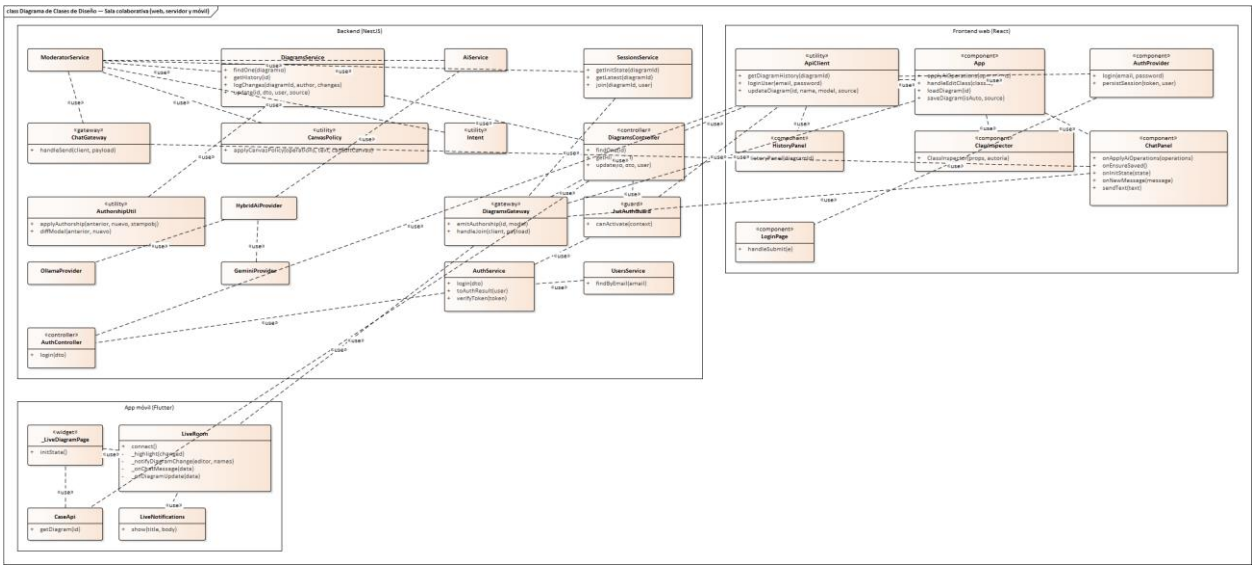


Figura — Diagrama de Clases de Diseño de la sala colaborativa (web, servidor y móvil)

Verificación modelo-código. El script `docs/diagrams/ea/verificar_secuencias.py` recorre los 10 diagramas de secuencia del ciclo y busca en el código cada mensaje que llega a un objeto. Resultado: **86 de 86 mensajes son operaciones de la clase que los recibe**. Otros 28 son operaciones de librerías: `Model` de Mongoose, `Server` de Socket.io y el plugin de notificaciones de Flutter. Ejemplo: en CU15, `ChatGateway` envía `answer()` a `ModeratorService`, y `answer()` está implementada en `ModeratorService`, no en `ChatGateway`.

4.3 Diseño de casos de uso

4.3.1 Diagrama de secuencia

Los diagramas de secuencia especifican el comportamiento dinámico de cada caso de uso mostrando el intercambio de mensajes entre los componentes reales del sistema (frontend, controladores NestJS, servicios, base de datos y la API de Gemini) en orden cronológico.

CU1 — Crear Clase UML

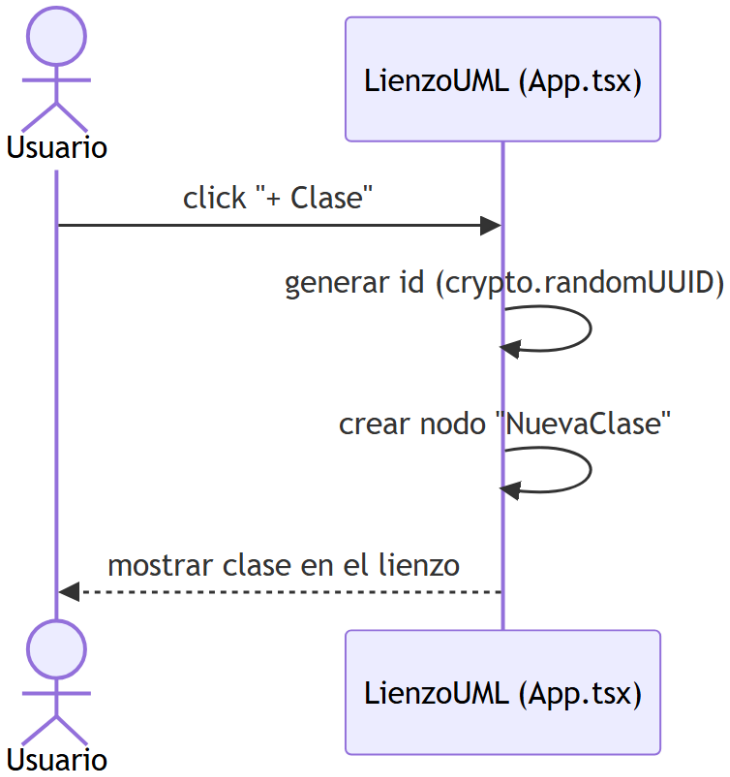


Figura — Diagrama de Secuencia CU1

CU2 — Editar Clase UML

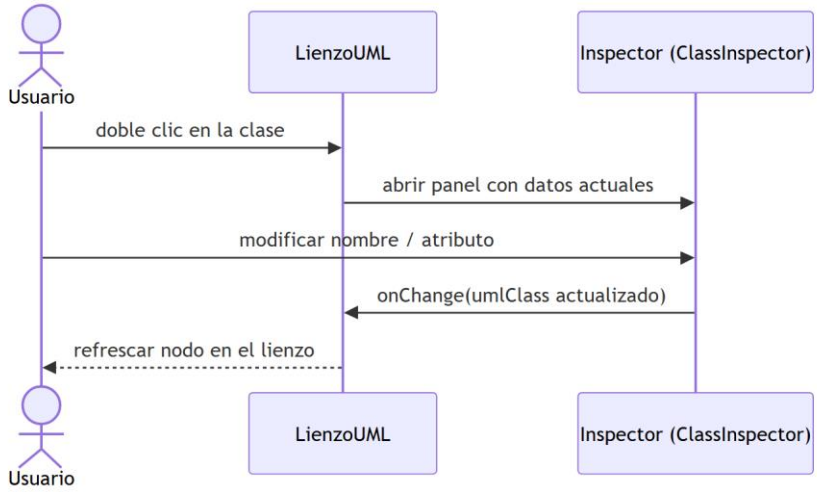


Figura — Diagrama de Secuencia CU2

CU3 — Eliminar Clase UML

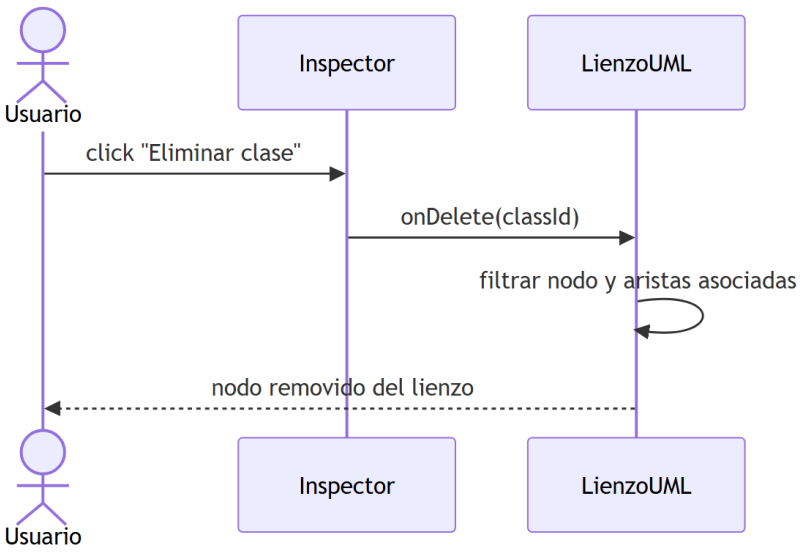


Figura — Diagrama de Secuencia CU3

CU4 — Crear Relación entre Clases

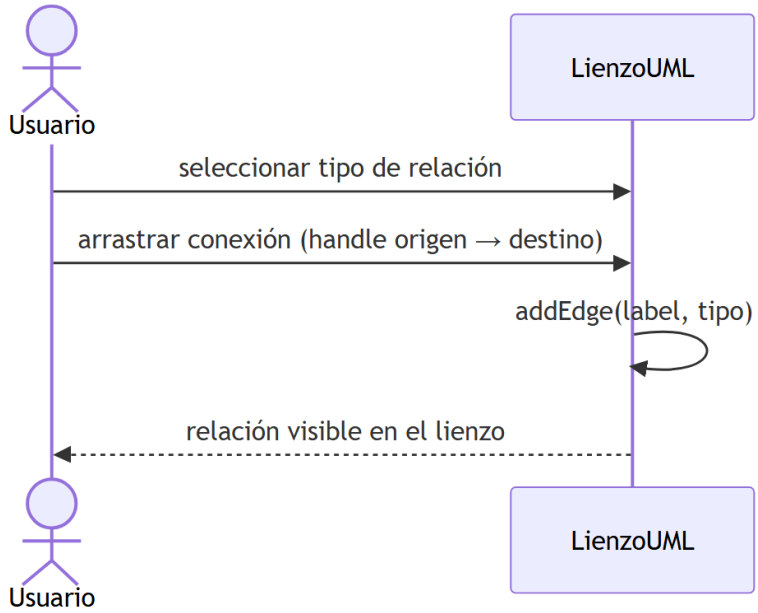


Figura — Diagrama de Secuencia CU4

CU5 — Guardar Diagrama

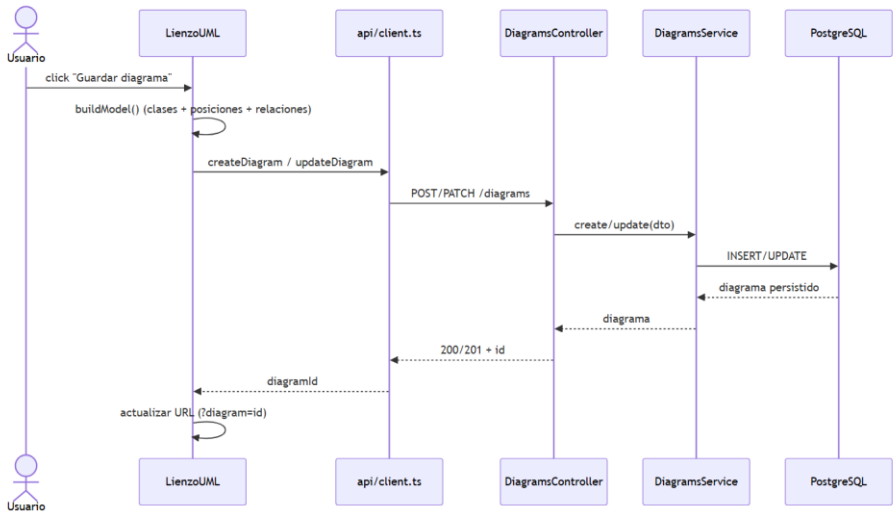


Figura — Diagrama de Secuencia CU5

CU6 — Cargar Diagrama Compartido

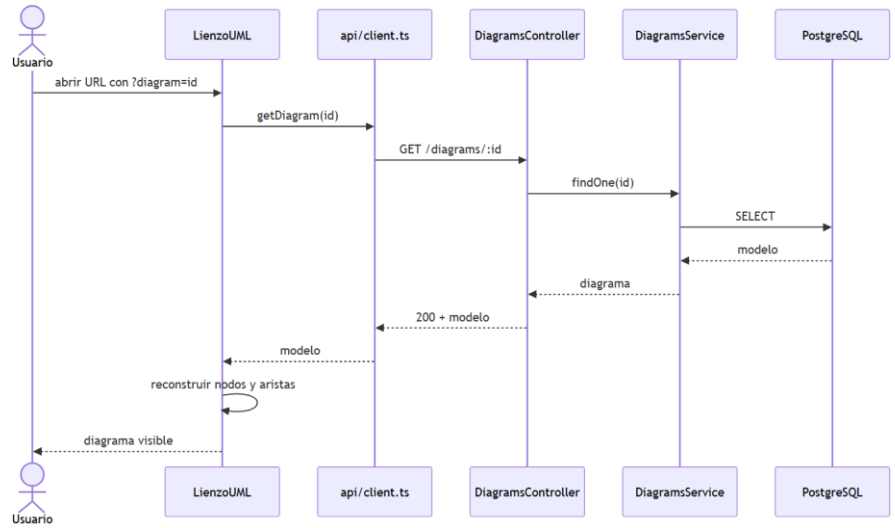


Figura — Diagrama de Secuencia CU6

CU7 — Colaborar en Tiempo Real

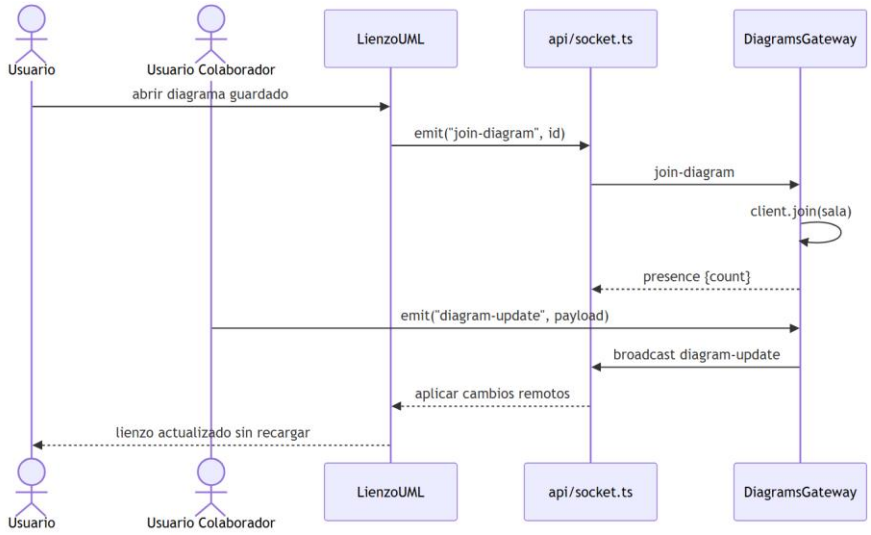


Figura — Diagrama de Secuencia CU7

CU8 — Generar Backend Spring Boot

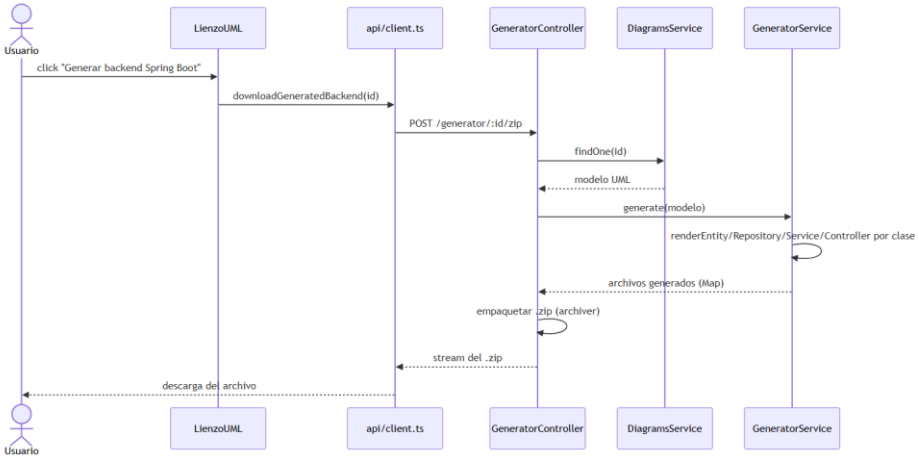


Figura — Diagrama de Secuencia CU8

CU9 — Exportar Diagrama a XMI

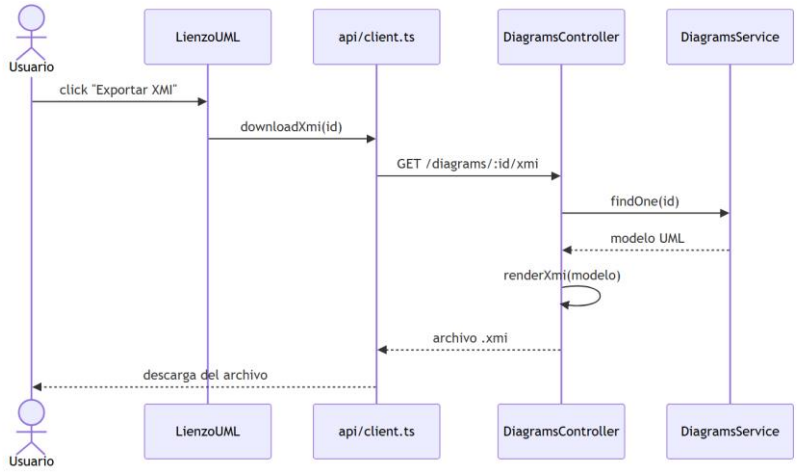


Figura — Diagrama de Secuencia CU9

CU10 — Consultar Asistente de IA

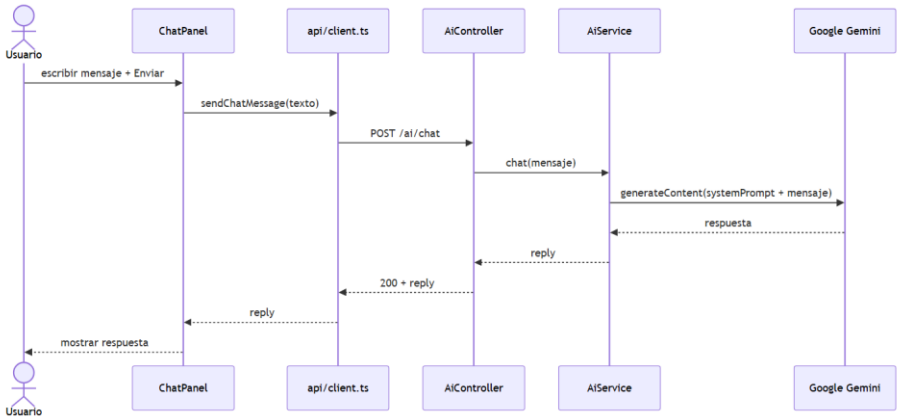


Figura — Diagrama de Secuencia CU10

CU11 — Dictar Comando por Voz

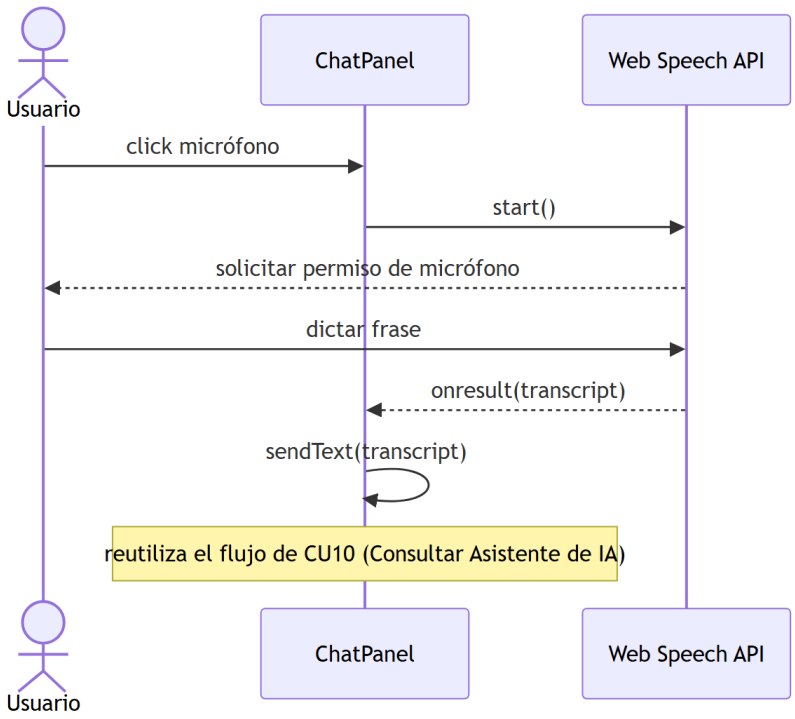


Figura — Diagrama de Secuencia CU11

CU12 — Interpretar Foto de Pizarra

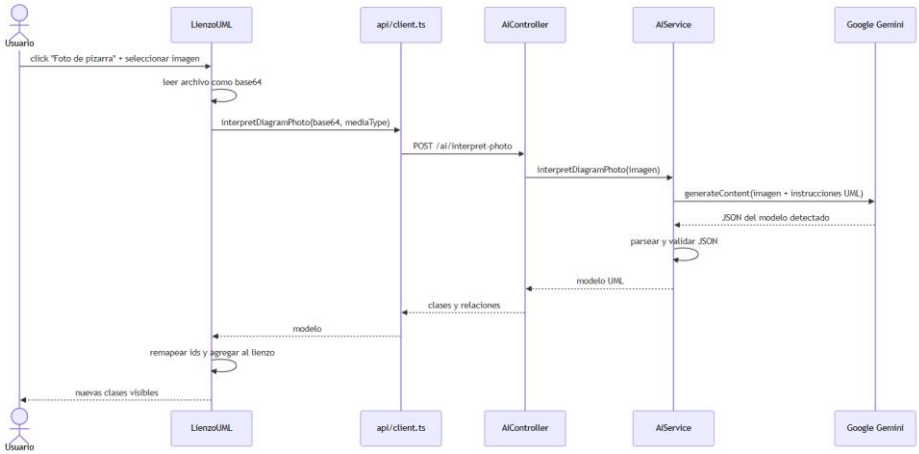


Figura — Diagrama de Secuencia CU12

CU13 — Iniciar Sesión

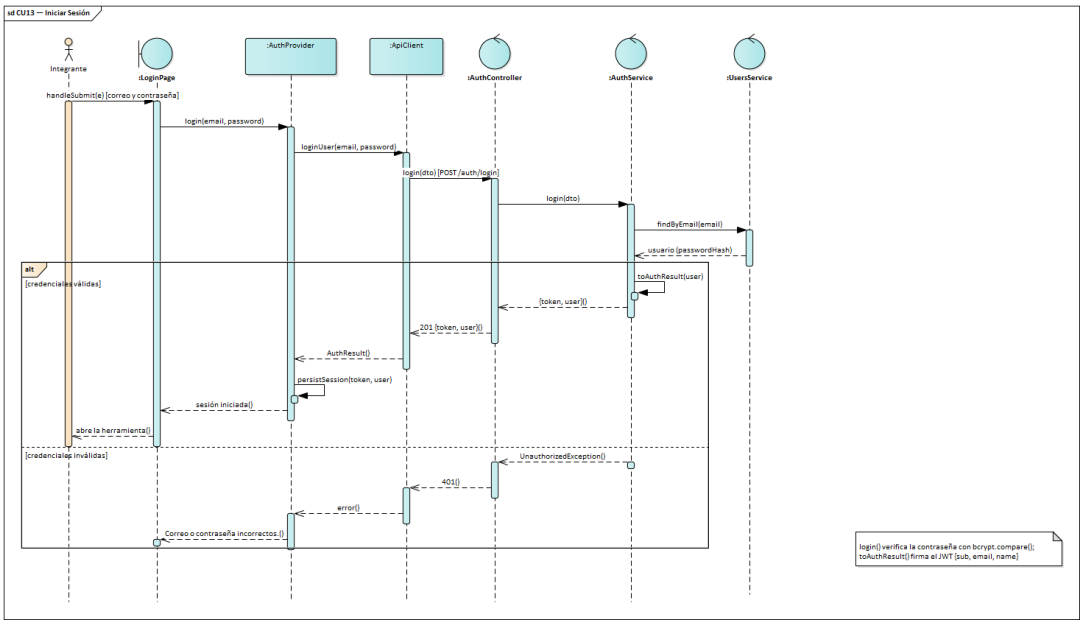


Figura — Diagrama de Secuencia CU13

CU14 — Enviar Mensaje al Chat Grupal

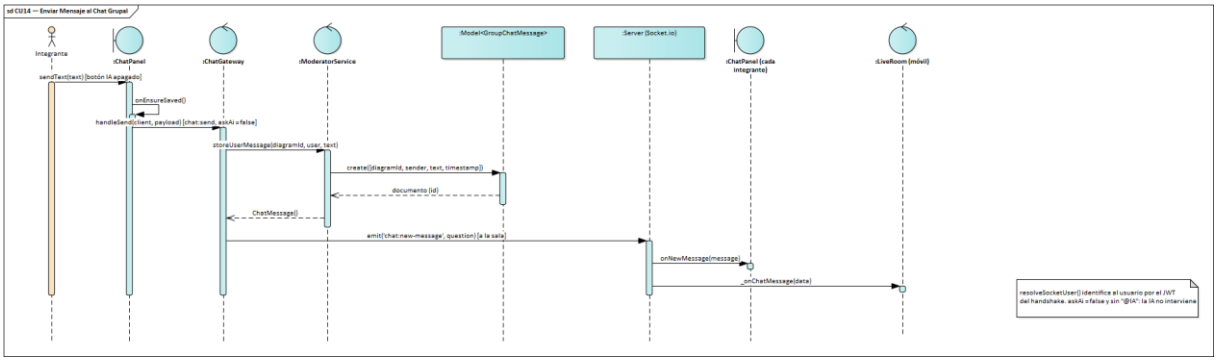


Figura — Diagrama de Secuencia CU14

CU15 — Consultar al Moderador IA

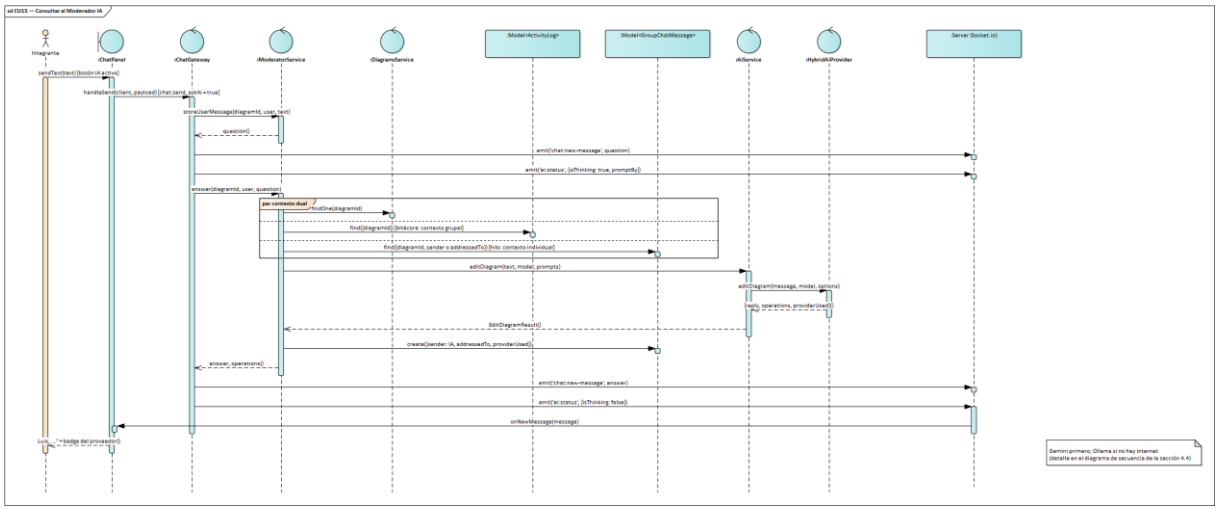


Figura — Diagrama de Secuencia CU15 (la versión detallada, con el respaldo Gemini ↔ Ollama, está en la sección 4.4)

CU16 — Solicitar Cambio del Diagrama a la IA

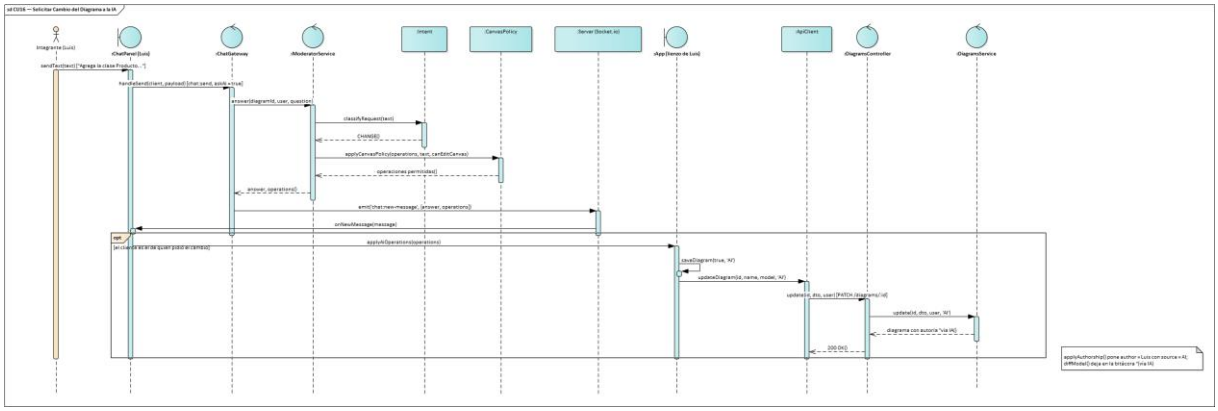


Figura — Diagrama de Secuencia CU16

CU17 — Consultar Autoría y Bitácora

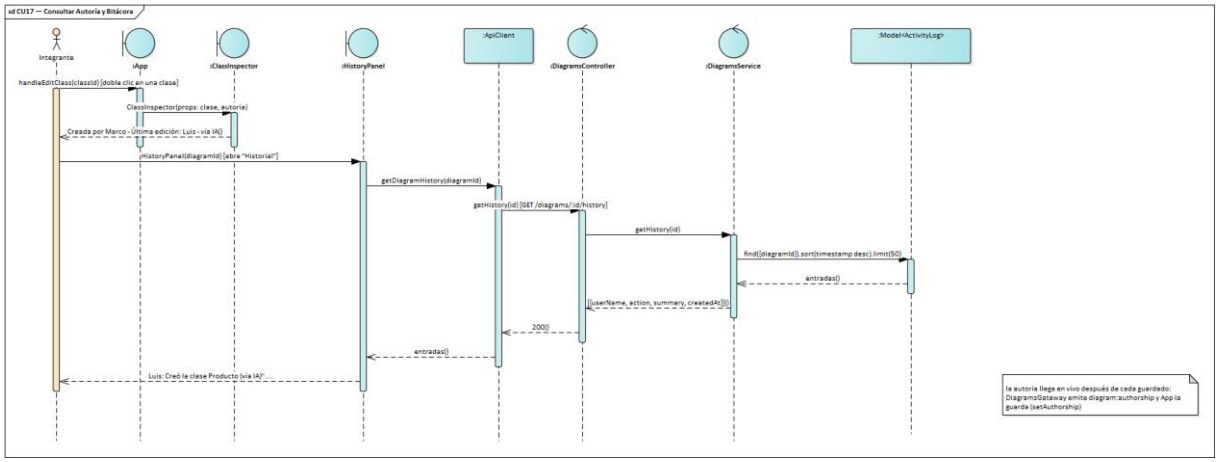


Figura — Diagrama de Secuencia CU17

CU18 — Reanudar Sesión de Trabajo

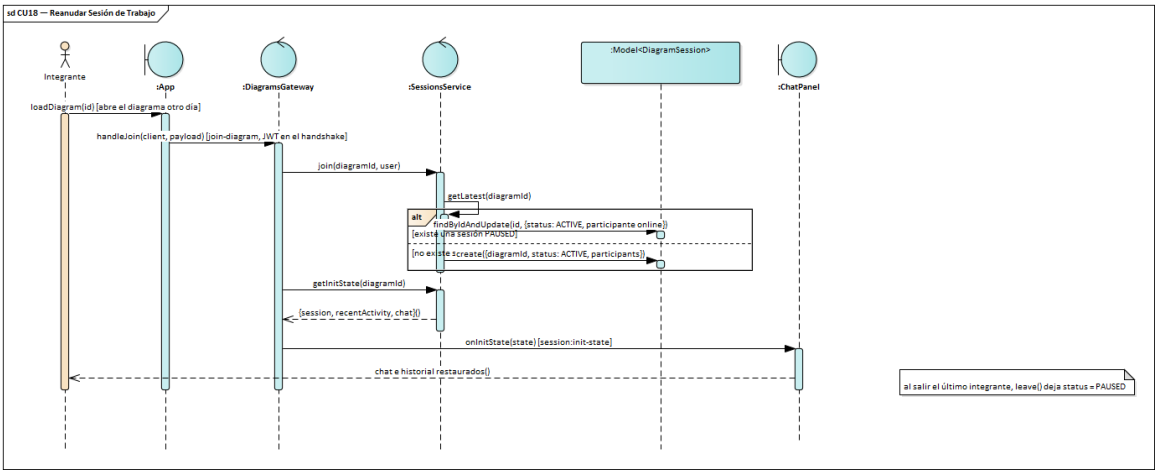


Figura — Diagrama de Secuencia CU18

CU19 — Visualizar Diagrama en Vivo (Móvil)

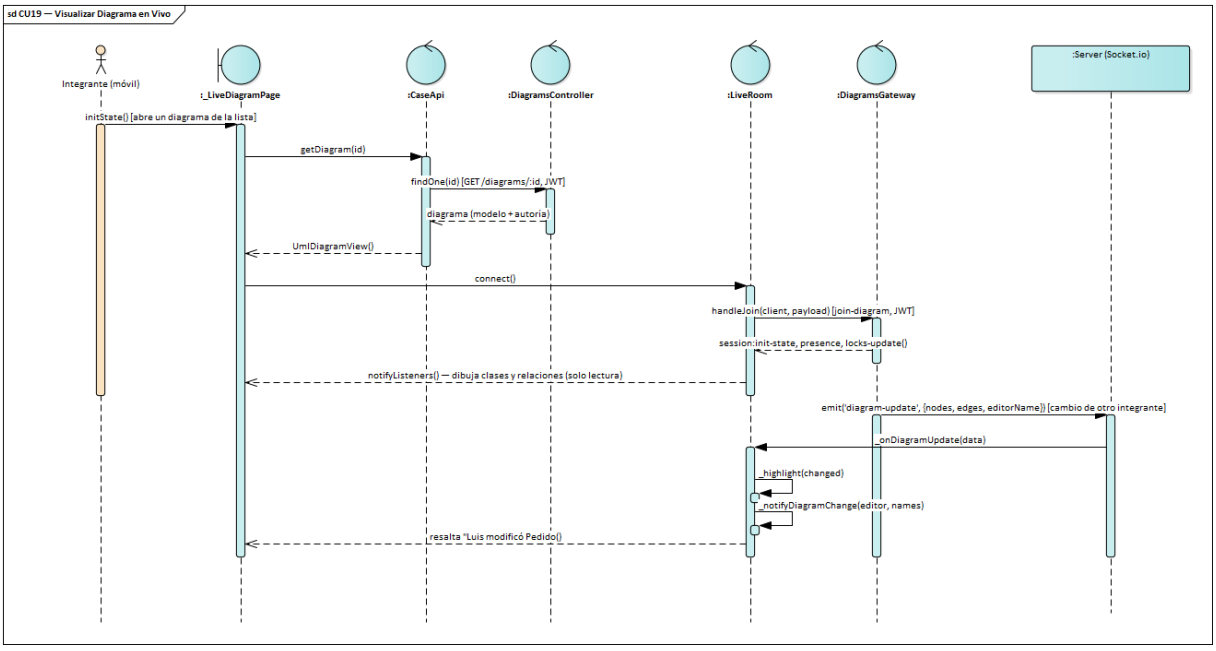


Figura — Diagrama de Secuencia CU19

CU20 — Recibir Notificaciones en el Móvil

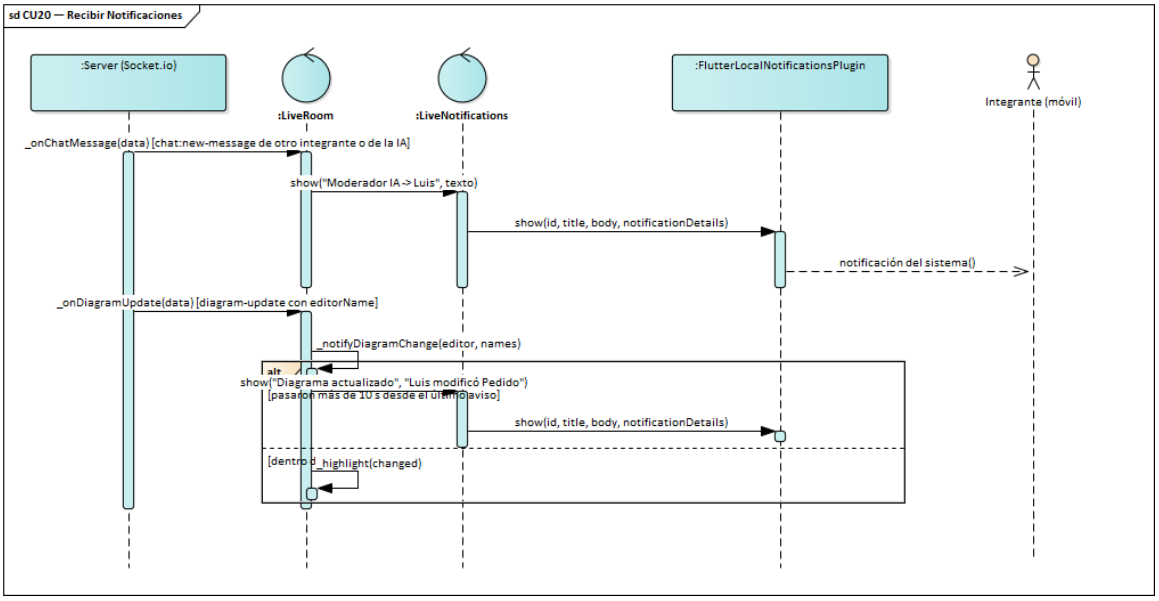


Figura — Diagrama de Secuencia CU20

4.4 Diseño detallado del Segundo Ciclo

Estos diagramas detallan los tres mecanismos centrales del ciclo 2:

- **La pregunta pública al Moderador IA**, con el contexto dual y el respaldo entre proveedores.
- **El guardado con autoría granular.**
- **El estado de la sesión continua y la decisión del motor híbrido.**

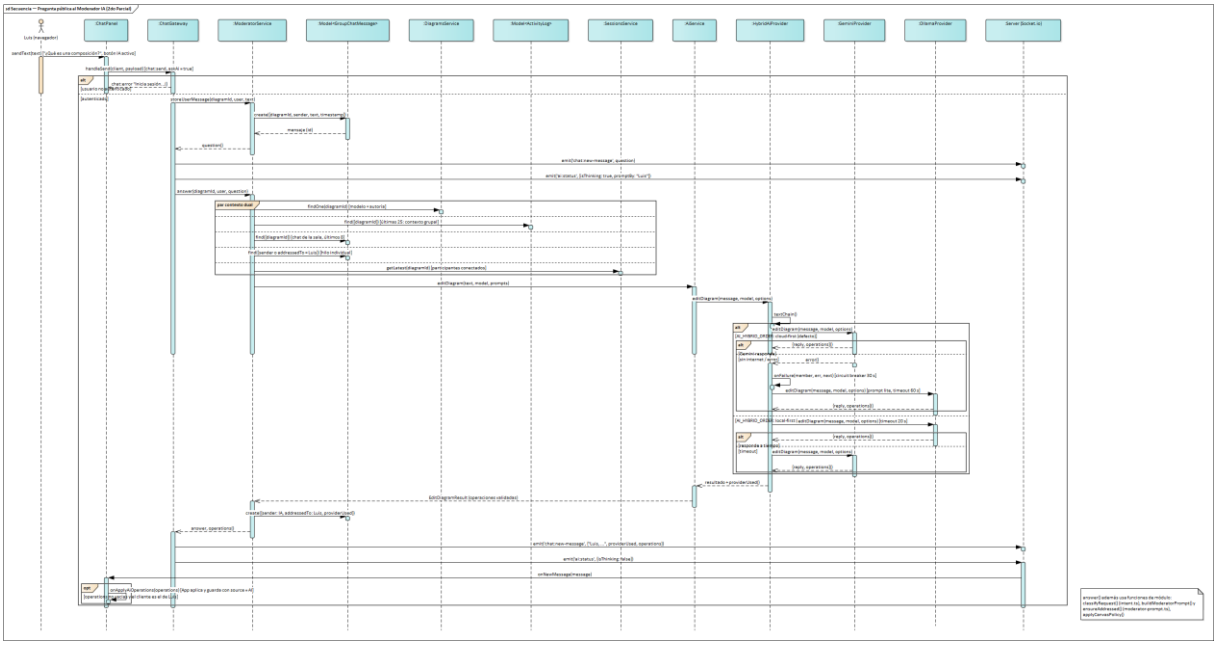


Figura — Secuencia detallada: pregunta pública al Moderador IA (alt / opt / par)

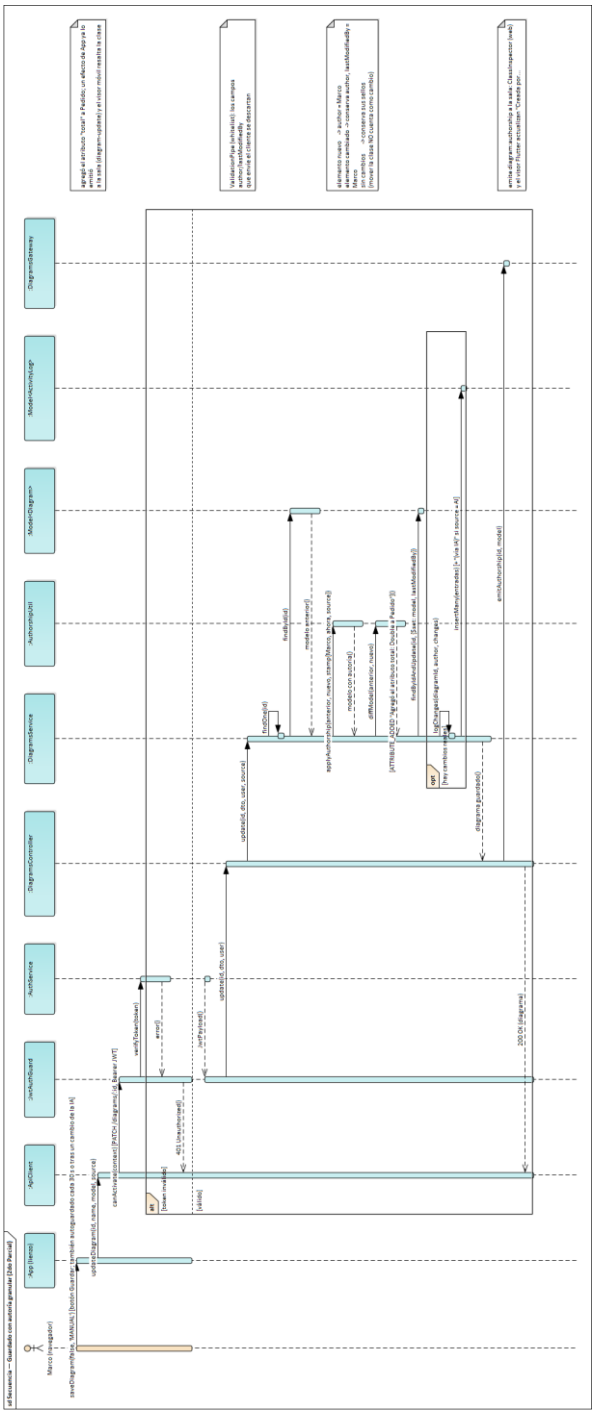


Figura — Secuencia detallada: guardado con autoría granular y bitácora

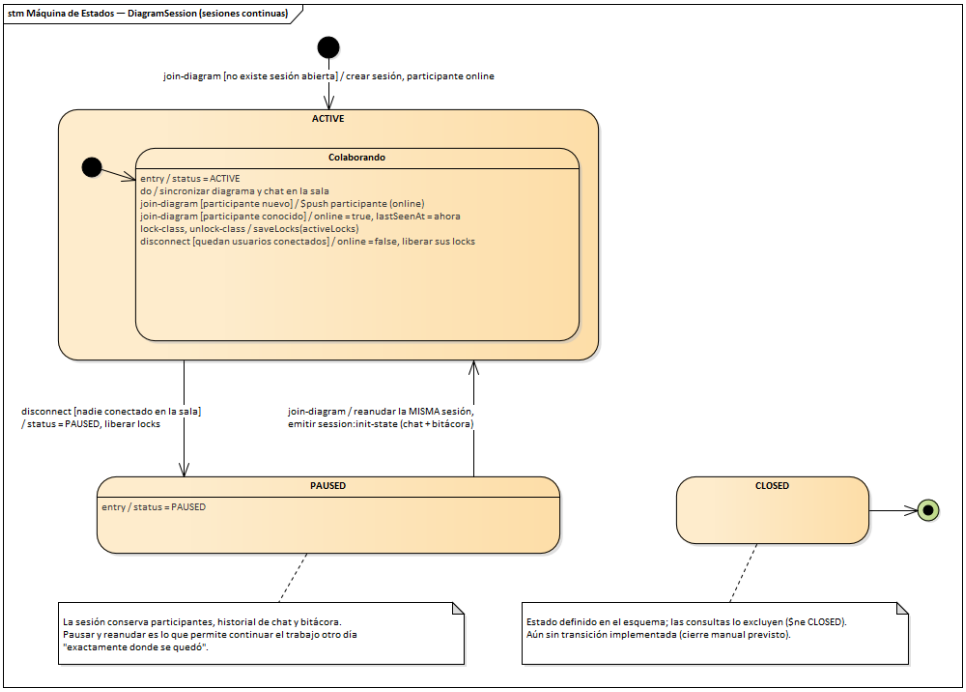


Figura — Máquina de estados de la sesión continua (ACTIVE ⇄ PAUSED)

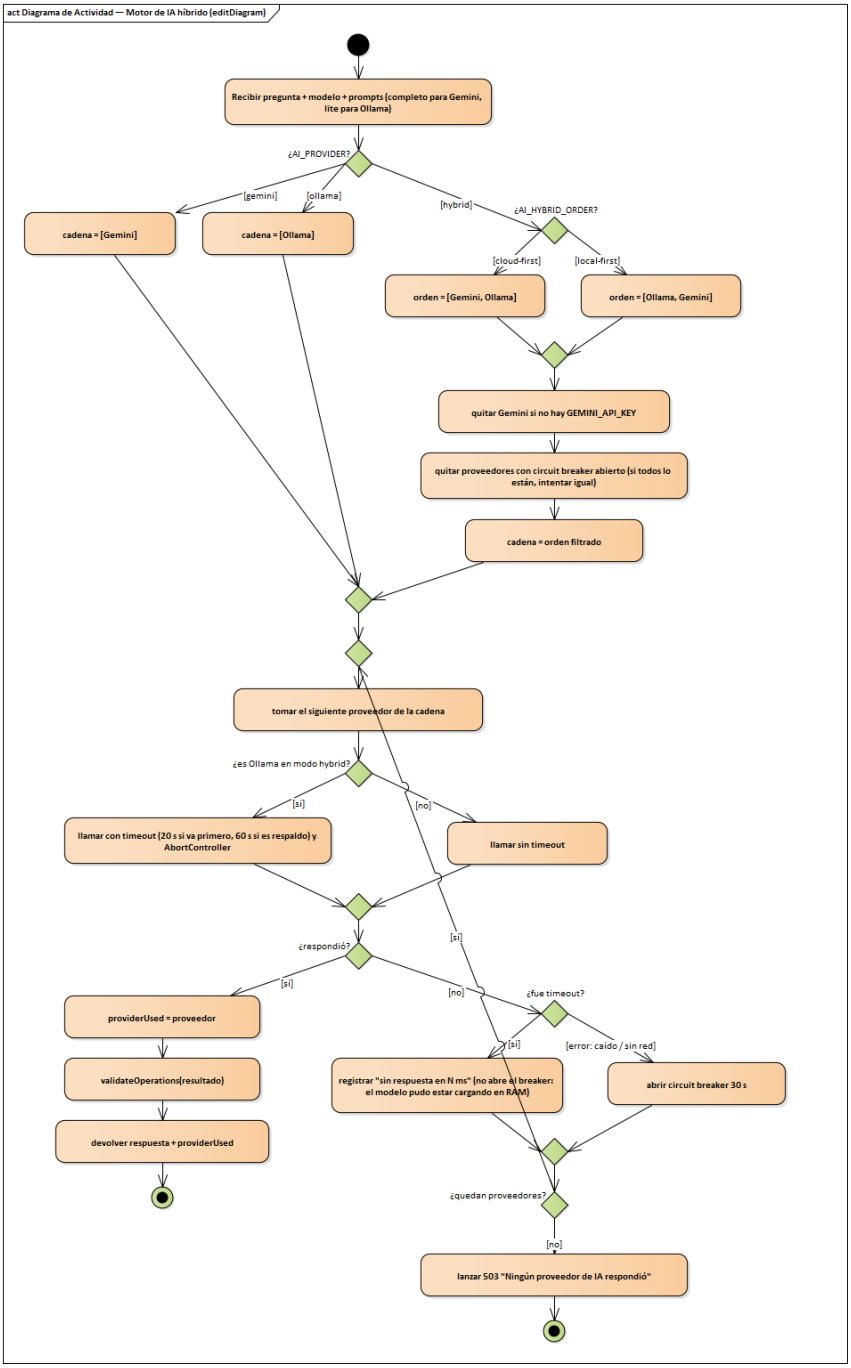


Figura — Diagrama de actividad del motor de IA híbrido

5. Implementación

5.1 Implementación de la arquitectura del sistema

El sistema está implementado con una arquitectura de dos servicios independientes:

- **Frontend — React + Vite:** SPA con el lienzo de diagramación (React Flow), colaboración vía Socket.io-client, y el panel de chat de IA con reconocimiento de voz nativo del navegador.
- **Backend — NestJS:** API REST con TypeORM sobre PostgreSQL, gateway de WebSockets (Socket.io) para colaboración, y el motor generador de código Spring Boot.

5.2 Implementación de la arquitectura de subsistema

1. Backend de la herramienta — NestJS

Arquitectura modular por dominio (diagrams, generator, ai), cada uno con su Controller → Service, y el módulo diagrams además con su Gateway de WebSockets y su entidad TypeORM.

2. Motor generador — Plantillas Java parametrizadas

Por cada clase del diagrama, el generador invoca las mismas cuatro funciones de plantilla (renderEntity, renderRepository, renderService, renderController), resolviendo las relaciones JPA mediante un módulo dedicado (relations.util.ts) que determina el lado propietario y el lado inverso de cada asociación.

3. Colaboración en Tiempo Real — Socket.io

El backend usa un Gateway de WebSockets. Los usuarios se unen a una sala identificada por el id del diagrama (evento join-diagram) y reciben los cambios de los demás participantes (evento diagram-update), además de un contador de presencia en tiempo real.

4. Inteligencia Artificial — Google Gemini

El backend invoca directamente el SDK de Google Gemini (sin microservicio intermedio) con instrucciones de sistema distintas según la tarea: una para el chat conversacional enjaulado a ingeniería de software, y otra para la interpretación de imágenes que enseña al modelo a leer la notación UML dibujada a mano y a devolver JSON estructurado.

5.3 Implementación — Segundo Ciclo

El segundo ciclo extiende los mismos servicios (backend NestJS, frontend React y app Flutter) sin romper lo del primer ciclo. Todo se desarrolló en la rama `feat/mongodb-autoria` y se integra mediante un Pull Request.

1. Backend — NestJS (nuevos módulos y cambios)

- **Persistencia (MongoDB + Mongoose):**
  - Esquemas `DiagramDocument`, `DiagramSessionDocument`, `ActivityLogDocument` y `GroupChatMessageDocument`.
  - `DiagramsService` reescrito sobre Mongoose con la **misma interfaz pública**, así que el generador y XMI no cambiaron.
  - `MongooseModule` convive con TypeORM: usuarios y adjuntos siguen en PostgreSQL.
- **Autoría y bitácora (`authorship.util.ts`):**
  - `applyAuthorship()` calcula los sellos `author`, `lastModifiedBy` y `source` comparando el modelo guardado con el nuevo; el cliente no puede falsificarlos.
  - `diffModel()` genera una entrada de bitácora por cambio.
  - `sanitizeModel()` normaliza los `null` que guarda Mongo.
- **Sesiones continuas (`SessionsService`):** `join` / `leave` (ACTIVE ⇄ PAUSED), `saveLocks` y `getInitState`; endpoint `GET /sessions/:diagramId/latest`.
- **Identidad en el socket (`socket-auth.ts`):** el JWT del *handshake* identifica al usuario; el guard JWT global ya no intercepta los mensajes WebSocket.
- **Chat grupal (`ChatGateway`):**
  - eventos `chat:send`, `chat:new-message`, `ai:status` y `chat:error`;
  - solo el cliente de quien pidió un cambio aplica las operaciones;
  - evento `ai:settings`: interruptor por sala «IA edita el lienzo» / «IA solo sugiere», guardado en el diagrama y registrado en la bitácora.
- **Moderador IA (`ModeratorService`, `moderator-prompt.ts`):**
  - contexto dual (25 entradas de bitácora, 8 mensajes de la sala y 8 del hilo individual);
  - prompt completo para Gemini y prompt *lite* con caché de prefijo para Ollama;
  - `ensureAddressed()` garantiza que la respuesta nombre a quien preguntó;
  - `applyCanvasPolicy()` aplica en el servidor el modo de la sala y `isExplicitChangeRequest()`: sin pedido explícito, o en modo «solo sugerencias», no se toca el lienzo; en ese modo las operaciones propuestas se devuelven como pasos para hacerlo a mano.
- **Modo IA (`AiModeBar`, `ChatPanel`):** vista propia de cada persona que cambia la forma de trabajar: en lugar de dibujar, se conversa. El chat pasa al centro con accesos directos, el diagrama queda como vista previa en vivo de solo lectura y la barra de edición se oculta. No afecta a los demás integrantes de la sala y se recuerda al recargar.

- **Modelo propio `moderador-uml` (Ollama):** *Modelfile* generado desde el código (`FROM qwen2.5:1.5b-instruct`, rol y glosario del moderador, temperatura 0, `num_ctx` 4096); se crea con `npm run ollama:modelo` y, si falta, el backend usa qwen2.5 base.
- **IA multifuncional (`classifyRequest`):** cada mensaje se clasifica como pregunta, cambio, revisión o proyecto. Una revisión recibe los resultados reales del validador, y el servidor los agrega a la respuesta. Un proyecto se modela completo de una vez (de 4 a 8 clases con sus relaciones) y se ordena al final.
- **Motor híbrido (`HybridAiProvider`):** cadena Gemini ⇄ Ollama según `AI_HYBRID_ORDER`, timeouts de 20 y 60 s, *circuit breaker* de 30 s y precalentamiento de Ollama al arrancar.
- **Migración:** script idempotente `migrate-pg-to-mongo` (copió 8 diagramas y 55 entradas de historial conservando los UUID).

2. Frontend — React (nuevos módulos)

- **`ChatPanel`** reescrito como chat público por WebSocket:
  - avatar, nombre, hora y destinatario de cada mensaje;
  - badge del proveedor e indicador animado *"pensando para…"*;
  - interruptor "preguntar a la IA" y dictado por voz.
- **`ClassInspector`:** sección **Autoría** (creada por, última edición y autor de cada atributo), actualizada con `diagram:authorship`.
- **`App.tsx`:**
  - aplica las operaciones de la IA y las guarda con `source: 'AI'`;
  - corrección de la sincronización en pestañas en segundo plano.
- **`api/socket.ts`:** envía el JWT en el *handshake*.

3. App móvil — Flutter (nuevo)

- **Carpeta `lib/live/`:**
  - `CaseApi` (login y diagramas);
  - `LiveRoom` (`socket_io_client`, solo escucha);
  - `UmlViewerScreen` (lienzo con zoom, ajuste a pantalla, resaltado de cambios, bloqueos, autoría y pestaña de chat);
  - `LiveNotifications` (`flutter_local_notifications`).
- **Acceso:** botón flotante "Visor UML en vivo".
- **Android:** permiso `POST_NOTIFICATIONS` y *desugaring*; APK de release compilado.

4. Despliegue — Segundo Ciclo

- **`docker-compose.prod.yml`:**
  - servicio `mongo:7` con *healthcheck* y volumen `mongo_data`;
  - variables `MONGO_URI`, `AI_HYBRID_ORDER`, `OLLAMA_TIMEOUT_MS` y `OLLAMA_FALLBACK_TIMEOUT_MS`.
- **nginx:** enruta `/sessions`.
- **Migración:** se ejecuta una vez con `docker compose exec backend node dist/scripts/migrate-pg-to-mongo.js`.

6. Pruebas

Esta fase valida que la implementación cumpla los requisitos definidos en el Perfil y el Modelo de Casos de Uso. A diferencia del resto de este documento, aquí no se describe lo que el sistema debería hacer, sino la evidencia concreta de que se verificó que lo hace.

6.1 Plan de pruebas

- **Alcance:** pruebas unitarias sobre la lógica pura del backend (el algoritmo de diff de la bitácora), pruebas funcionales del backend generado (Spring Boot) con Postman, y pruebas de integración y de sistema ejecutadas en vivo contra los servicios reales (PostgreSQL, Amazon S3, Google Gemini) y, para el sistema completo, contra el servidor de producción real en AWS (http://34.231.176.225), no contra localhost.
- **Estrategia:** caja blanca (Vitest) para la lógica de negocio aislable, y caja negra (peticiones HTTP reales vía curl/Postman y verificación desde el navegador) para los endpoints REST, los WebSockets y el flujo end-to-end del usuario.
- **Criterios de aceptación:** el código Spring Boot generado debe compilar y ejecutarse sin modificaciones; cada endpoint debe responder con el código HTTP y el cuerpo esperado; un archivo subido a S3 debe descargarse con el mismo contenido byte a byte que el original.

6.2 Diseño de pruebas (casos de prueba)

- **CP-01 — Generación de backend Spring Boot:** a partir de un diagrama con relaciones 1:1, 1:N y N:N, el backend generado debe incluir las cuatro capas (Entity, Repository, Service, Controller) por clase, con las anotaciones JPA correctas (@OneToMany/@ManyToOne, tabla intermedia para N:N).
- **CP-02 — Autenticación y bitácora:** registrar un usuario, iniciar sesión, crear y modificar un diagrama, y verificar que la bitácora registre el nombre real del usuario autenticado y un resumen legible del cambio (clases agregadas, atributos modificados, relaciones eliminadas).
- **CP-03 — Adjuntos en S3:** subir un archivo permitido y verificar que aparece en el listado del diagrama, descargarlo mediante la URL firmada y comparar el contenido con el original, eliminarlo y confirmar que desaparece del bucket, e intentar subir un archivo de tipo no permitido (.exe) y verificar el rechazo.
- **CP-04 — Colaboración en tiempo real:** con dos sesiones de navegador abiertas sobre el mismo diagrama, verificar que un cambio hecho en una se refleje en la otra vía WebSocket sin recargar la página.
- **Componente de prueba:** la suite model-diff.util.spec.ts (Vitest) ejercita la función pura summarizeModelDiff con 7 casos — clase agregada, eliminada, renombrada, atributos y operaciones modificados, y relaciones agregadas/eliminadas/con tipo cambiado — sin depender de base de datos ni red.

6.3 Pruebas de integración

- **Frontend ↔ Backend (vía Nginx):** en producción, Nginx actúa como proxy reverso hacia NestJS para /auth, /diagrams, /attachments, /generator, /ai y /socket.io. Se verificó cada ruta con peticiones reales contra la IP pública.
- **Backend ↔ PostgreSQL:** persistencia de diagramas, relaciones y entradas de la bitácora, con DB_SYNCHRONIZE=true generando el esquema automáticamente al iniciar el contenedor.
- **Backend ↔ Amazon S3:** subida, descarga con URL firmada (expira en 5 minutos) y eliminación, verificadas contra el bucket real diagramador-uml-adjuntos-fernando, no contra un mock.
- **Backend ↔ Google Gemini:** el chat conversacional, el dictado por voz y la interpretación de fotografías de diagramas dibujados a mano se probaron contra el servicio real de IA, no contra una respuesta simulada.

Defectos encontrados y corregidos durante la integración (antes del primer despliegue a producción, gracias a un ensayo local completo con Docker Compose sobre la misma imagen que se desplegaría):

- **Rutas de Nginx incompletas:** nginx.conf no reenviaba /auth ni /attachments al backend — se habían agregado esos módulos después de escribir la configuración original. Sin el fix, login y adjuntos habrían devuelto el HTML de la SPA en vez de JSON en producción, aunque funcionaran en desarrollo.
- **Incompatibilidad de versión de Node/npm en el build:** node:22-alpine (npm 10.x) resolvía package-lock.json de forma distinta al npm 11.x local, y npm ci fallaba dentro del contenedor con “Missing: typescript@5.9.3 from lock file”. Se corrigió alineando ambos Dockerfiles a node:24-alpine.
- **Variables de entorno incompletas en Docker Compose:** docker-compose.prod.yml no exponía JWT_SECRET ni las variables AWS_* al contenedor del backend — se agregaron esas variables después de escribir el archivo original.

6.4 Pruebas de sistema

Se ejecutó el flujo completo extremo a extremo contra el servidor de producción real (http://34.231.176.225), no contra un entorno local:

- **SYS-01 — Registro, autenticación y creación de diagrama:** un usuario nuevo se registra, inicia sesión, crea un diagrama y agrega clases — resultado: correcto.
- **SYS-02 — Bitácora de cambios:** cada modificación queda registrada con el nombre real del usuario y un resumen del cambio — resultado: correcto.
- **SYS-03 — Documentos adjuntos:** subida a S3 real desde dentro de la infraestructura de AWS — resultado: correcto.

- **SYS-04 — Generación de backend Spring Boot:** descarga del proyecto generado, revisión de las cuatro capas por clase — resultado: correcto.
- **SYS-05 — Recuperación ante reinicio del servidor:** los tres contenedores (restart: unless-stopped, Docker habilitado al arranque) vuelven a levantarse automáticamente tras un reinicio de la instancia EC2, sin intervención manual.

6.5 Evaluación de las pruebas

Todos los casos de prueba diseñados en 6.2 se ejecutaron y pasaron. Los tres defectos encontrados durante la integración (6.3) se detectaron antes de tocar la infraestructura de AWS —gracias a ensayar primero la pila completa de Docker Compose en local— y se corrigieron y verificaron antes del primer despliegue real, por lo que ninguno llegó a afectar al sistema en producción. Se considera que el sistema cumple los criterios de aceptación definidos en el plan de pruebas y está listo para la fase de despliegue documentada en la Parte I, sección 3.

6.6 Pruebas del Segundo Ciclo

- **Pruebas automatizadas:** 152 pruebas unitarias en 22 archivos (Vitest), todas aprobadas. Cubren autoría y bitácora, saneamiento del modelo, motor híbrido, prompt del moderador, detección de pedidos explícitos, validación de operaciones de la IA, generador y XMI.
- **CP-05 — Autoría granular:** Marco crea una clase y Luis le agrega un atributo. El sistema registra a Marco como autor, a Luis como última edición y a Luis como autor del atributo; un autor enviado por el cliente se ignora. *Resultado: correcto.*
- **CP-06 — Sesión continua:** salir de la sala la deja PAUSED; volver la reactiva con el mismo identificador y restaura el chat. *Resultado: correcto.*
- **CP-07 — Moderador IA:**
    - una pregunta teórica obtiene respuesta por nombre;
    - *"¿quién creó Pedido?"* se responde con la bitácora;
    - *"como te comenté antes…"* demuestra el contexto individual;
    - un pedido de cambio genera las operaciones correctas y una pregunta no modifica el lienzo.

  Resultado: correcto.
- **CP-08 — Motor híbrido (medido en CPU):**

Escenario	Proveedor que respondió	Tiempo
`cloud-first`	Gemini	4,3 s
Falla Gemini → respaldo	Ollama	13,5 s
`local-first`, modelo precalentado	Ollama	5,0 s

- **CP-09 — Dos usuarios en el navegador:**

    - indicador *“pensando para Luis”* visible para ambos;
    - el cambio pedido por Luis aparece en ambos lienzos;
    - el inspector muestra *“Creada por Luis · vía IA”*.

  Resultado: correcto.

- **CP-10 — Visor móvil:** un cambio pedido desde la web aparece en el celular sin recargar, junto con el chat y la autoría. *Resultado: correcto.*

Las pruebas con usuarios reales encontraron y corrigieron, entre otros, estos defectos (el registro completo está en la sección 7.5):

- una pestaña en segundo plano dejaba de sincronizar;
- las clases perdían su posición al recargar (defecto del ciclo 1);
- aparecían entradas falsas en la bitácora.

7. Registro del desarrollo (evidencia del proceso)

7.1 Ciclos del Proceso Unificado y forma de trabajo

7.1.1 Ciclos, fases y flujos de trabajo

El proyecto siguió el ciclo de vida del Proceso Unificado (capítulo 7 de la Parte 1) en **dos ciclos**. El Ciclo 1 corresponde al 1er Parcial y entregó la herramienta CASE base. El Ciclo 2 corresponde al 2do Parcial y entregó la IA multifuncional, NoSQL y el visor móvil. Cada ciclo recorrió las cuatro fases, y en cada fase se trabajaron los cinco flujos con distinta intensidad. Las fechas son las de los commits.

Ciclo 1 — 1er Parcial (30/08/2026 al 23/09/2026)

Fase	Fechas	Flujos con más trabajo	Resultado (hito)	Dónde está en el documento
Inicio	30/08	Requisitos	Perfil, alcance, actores y casos de uso CU1–CU12 priorizados	Perfil; Captura de requisitos
Elaboración	30/08 – 06/09	Requisitos, Análisis, Diseño	Casos de uso detallados, diagramas de comunicación, clases de análisis, arquitectura y datos lógicos	Análisis; Diseño
Construcción	30/08 – 06/09	Implementación, Prueba	Lienzo UML, generador Spring Boot, chat IA, colaboración en tiempo real y pruebas unitarias	Implementación; Pruebas
Transición	06/09 – 23/09	Prueba, despliegue	Despliegue en AWS con HTTPS, plan B local, IA offline verificada en un teléfono real	Pruebas; Anexo A

Ciclo 2 — 2do Parcial (17/10/2026 al 10/11/2026)

Fase	Fechas	Flujos con más trabajo	Resultado (hito)	Dónde está en el documento
Inicio	17/10	Requisitos	Plan del 2do Parcial, nuevos actores, casos de	Perfil (2do Parcial); Captura de

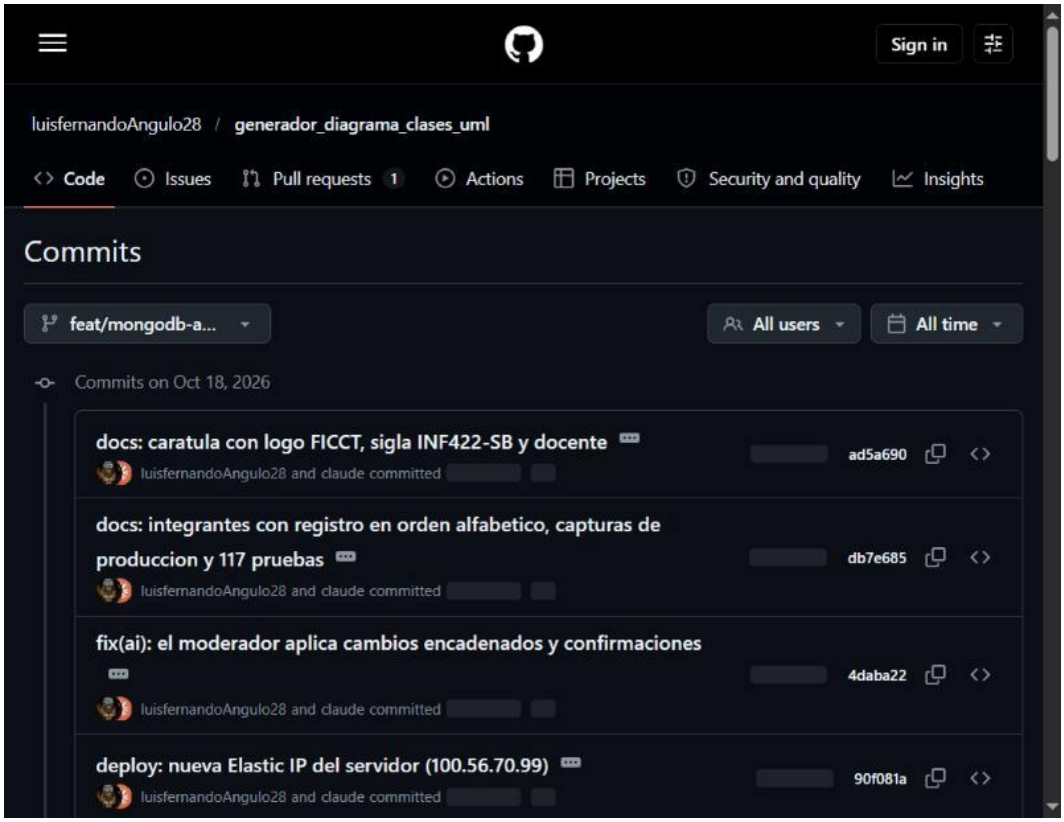
Fase	Fechas	Flujos con más trabajo	Resultado (hito)	Dónde está en el documento
			uso CU13–CU20 y modelo de casos de uso por ciclo	requisitos
Elaboración	17/10	Análisis, Diseño	Diagramas de comunicación CU13–CU20, clases de análisis, arquitectura de despliegue, componentes, persistencia políglota y motor de IA híbrido	Análisis; Diseño detallado del Segundo Ciclo
Construcción	17/10 – 02/11	Implementación, Prueba	MongoDB con autoría, motor híbrido, chat grupal, visor Flutter, ejecutor configurable, modelo propio de Ollama, Modo IA e IA multifuncional	Implementación — Segundo Ciclo; 7.3
Transición	02/11 – 10/11	Prueba, despliegue	Prueba de regresión del 1er Parcial, despliegue en AWS, documento final y defensa	7.4; Anexo A

En el Ciclo 2 las fases de Inicio y Elaboración fueron cortas porque la arquitectura del Ciclo 1 ya estaba probada: el nuevo ciclo la extendió (MongoDB, motor híbrido, visor) en lugar de rehacerla.

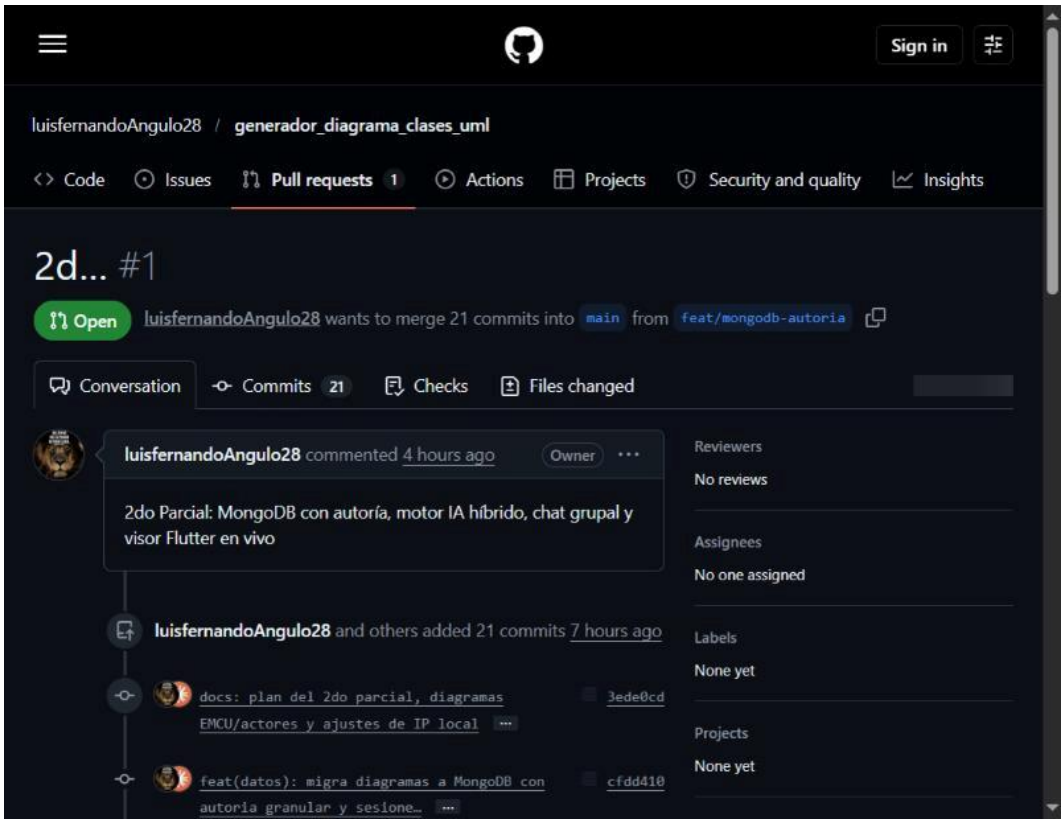
7.1.2 Control de versiones y forma de trabajo

- **Repositorio:** `github.com/luisfernandoAngulo28/generador_diagrama_clases_uml` (Git + GitHub), con historial completo desde el inicio del proyecto.
- **Volumen: 89 commits** entre el **30/08/2026** y el **17/10/2026** (hasta `b014a42`; 79 del 1er Parcial y 10 del 2do).
- **Ramas:**
    - El 1er Parcial se desarrolló sobre `main`.

    - El 2do Parcial se aisló en la rama **`feat/mongodb-autoria`** para no romper lo ya evaluado.
    - Esa rama se integra mediante el **Pull Request #1**, que GitHub reporta sin conflictos con `main`.
- **Convención de mensajes:** desde septiembre se usa *Conventional Commits* (`feat:`, `fix:`, `perf:`, `docs:`). Cada commit del 2do Parcial explica **qué** cambió y **por qué**.
- **Planificación:** `PLAN_EVOLUCION_2DO_PARCIAL.md` y `CONTEXTO_CLAUDE_2DO_PARCIAL.md` (cronograma de 4 semanas y checklist), y `PLAN_SEMANA1_MONGODB.md` (diseño de la migración antes de escribir código).



Captura 1: historial de commits de la rama feat/mongodb-autoria en GitHub.



Captura 2: el Pull Request #1, que integra los commits de feat/mongodb-autoria en main.

7.2 Línea de tiempo del 1er Parcial (base del proyecto)

Semana (ISO)	Fechas	Commits	Funcionalidades entregadas
W35	30 ago	8	Lienzo UML, generador Spring Boot, chat IA, colaboración en tiempo real (Socket.io), dictado por voz, foto de pizarra a diagrama, exportación XMI, plantilla Flutter
W36	31 ago – 06 sep	56	Documentación PUDS con 28 diagramas, validación y normalización del modelo, despliegue en AWS (Docker Compose), tests unitarios, tour de onboarding, estereotipos «enum», «abstract» e «interface», bloqueo por clase, deshacer/rehacer, plantillas tipo EA,

Semana (ISO)	Fechas	Commits	Funcionalidades entregadas
			autenticación JWT, notación UML 2.5, bitácora de cambios, adjuntos en S3, HTTPS con Let's Encrypt en DuckDNS, Swagger y Docker en el ZIP generado
W37–W38	12–20 sep	4	Plan B local: pila completa sin AWS e IA local con Ollama; reintento ante errores 503 de Gemini
W39	21–23 sep	11	Verificación de IA offline en dispositivo físico, estándar de codificación, métricas OO, diagramas de secuencia, BDD, mock API, gestión de usuarios, importación del XMI de Enterprise Architect

7.3 Registro del 2do Parcial: plan vs. ejecución real

El cronograma planificado distribuía el trabajo en cuatro semanas (17/10 al 10/11). En la práctica, las cuatro etapas se implementaron y verificaron el **17/10/2026**, como muestran las fechas de los commits. El tiempo restante hasta la entrega queda para la documentación, el despliegue y el simulacro.

Etapa del plan	Commit	Fecha	Funcionalidad implementada	Cómo se verificó
Preparación	`3ede0cd`	17/10	Plan del 2do Parcial, diagramas de actores y EMCU por ciclo	Revisión del plan
Semana 1 — MongoDB, autoría y sesiones	`cfdd410`	17/10	Persistencia políglota (Mongo + PostgreSQL), autoría granular por clase, atributo y relación, bitácora estructurada, sesiones continuas, JWT en Socket.io, script de migración	7 tests nuevos de autoría; migración real de 8 diagramas y 55 entradas de historial (idempotente); prueba E2E de las funciones del 1er Parcial: XMI de ida y vuelta, ZIP del

Etapa del plan	Commit	Fecha	Funcionalidad implementada	Cómo se verificó
				generador, locks
Semana 1 (ajuste)	`803ab14`	17/10	nginx enruta `/sessions`	Pila Docker completa probada detrás de nginx
Semana 2 — motor IA híbrido	`914167e`	17/10	`HybridAiProvider` (Ollama ↔ Gemini), System Prompt del moderador con contexto dual, `POST /ai/moderator/:id`	12 tests (motor y prompt); preguntas reales teórica, de contexto, de seguimiento y de acción
Semana 2 (configuración)	`a53a264`	17/10	`AI_PROVIDER=hybrid` documentado	—
Semana 2 (rendimiento)	`679ed16`	17/10	Prompt lite para el modelo local; filtros deterministas	Medición de latencia en CPU (ver 7.4.2); 14 tests nuevos
Semana 2 (orden configurable)	`602a785`	17/10	`AI_HYBRID_ORDER` cloud-first/local-first; precalentamiento de Ollama	4 tests nuevos; tres escenarios reales (ver 7.4.2)
Semana 3 — chat grupal	`814d912`	17/10	`chat:send` / `chat:new-message` / `ai:status`, ChatPanel público con badge del proveedor, autoría en vivo en el inspector	Prueba en navegador con dos usuarios simultáneos (ver 7.4.3)
Semana 4 — visor Flutter	`e5eeae3`	17/10	`UmlViewerScreen` de solo lectura con chat y notificaciones locales	`flutter analyze` sin problemas; versión web probada contra el backend; `flutter build apk --release` exitoso
Documentación	`b014a42`	17/10	Sección 1 del SDD (fundamentación teórica)	Cada afirmación referida a un archivo del código
Despliegue en AWS	`90f081a`	17/10	MongoDB en el VPS, migración y nueva Elastic IP	Dominio, TLS y WebSocket verificados en

Etapa del plan	Commit	Fecha	Funcionalidad implementada	Cómo se verificó
				producción
Corrección en producción	`4daba22`	17/10	Renombres encadenados y confirmaciones (“sí, hazlo”)	8 tests nuevos; prueba real en producción (Capturas 6 y 12)
Ejecutor configurable	`d003cf8`	18/10	Interruptor por sala «IA edita el lienzo» / «IA solo sugiere» (`ai:settings`, `applyCanvasPolicy`)	8 tests nuevos; prueba local con Ollama en ambos modos
Modelo propio en Ollama	`83e4e9e`	18/10	`moderador-uml`: Modelfile generado desde el código, `npm run ollama:modelo`, respaldo a qwen2.5 base	4 tests nuevos; respuesta real en el chat con 🏠 Ollama Local
Modo IA	`fee2261`	02/11	Vista propia por persona: barra compacta, chat al centro con accesos directos (revisar, crear proyecto, explicar, qué hizo el equipo) y el diagrama como vista previa en vivo de solo lectura	Prueba en navegador a 1100 px y 375 px: sin scroll horizontal, se recuerda al recargar y se sale al lienzo normal
Generador según OMT	`e4b9fbd`	05/11	La base del backend generado la crea solo el ORM; herencia JOINED; agregación y composición con su semántica UML; `data.sql` sobre las mismas tablas	Proyecto generado levantado con PostgreSQL (ver 7.4.7)
Historial de versiones	`557acef`	05/11	Una foto del modelo por sesión de edición (colección `diagram_versions`); *Archivo* →	6 tests nuevos; prueba de punta a punta por API y en el navegador (ver 7.4.8)

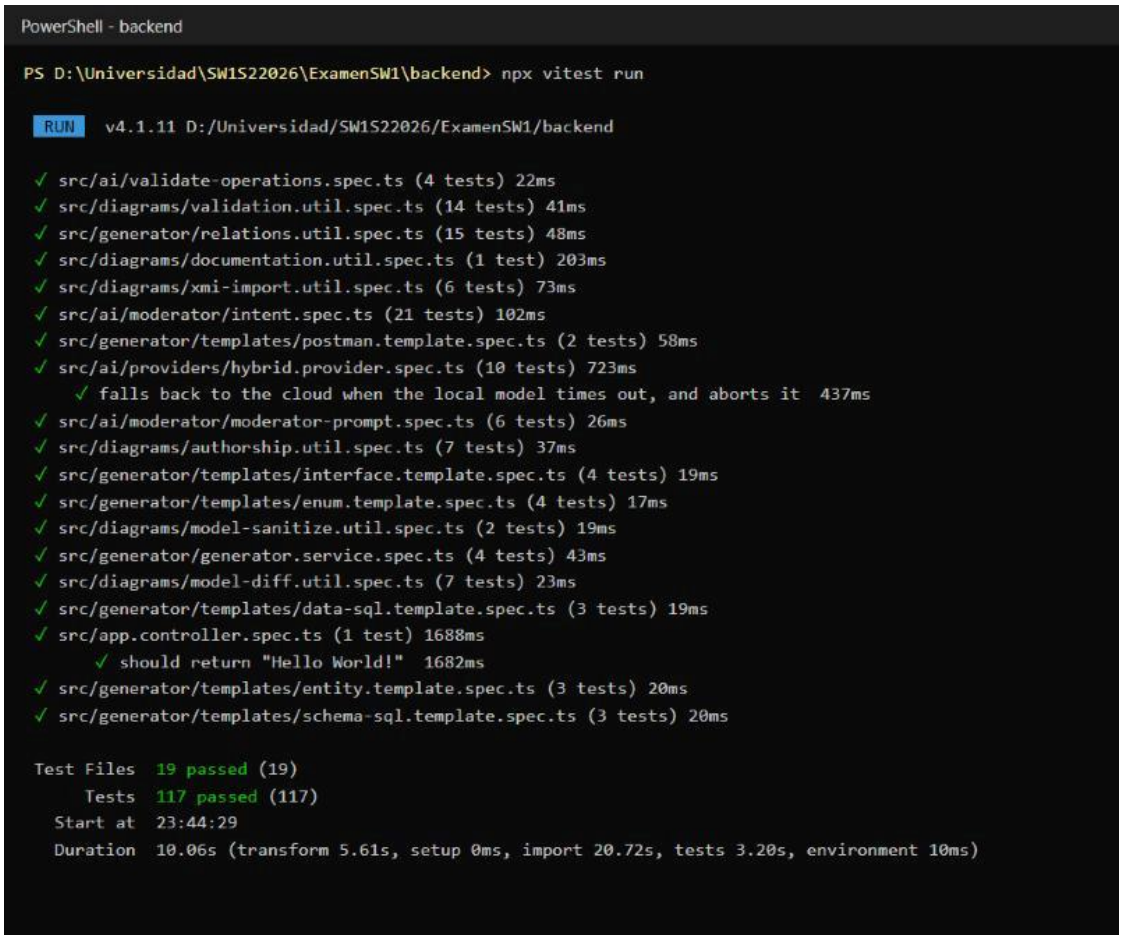
Etapa del plan	Commit	Fecha	Funcionalidad implementada	Cómo se verificó
			Historial de versiones con **Restaurar**, que crea una versión nueva y queda en la bitácora	
Redespliegue en AWS	`2e2a05a`	07/11	Cuenta nueva: la organización solo permite EC2 en **us-east-2 (Ohio)**. Servidor, IP fija, bucket y usuarios IAM nuevos (uno para desplegar y otro, solo del bucket, para la app); 2 GB de swap	Datos restaurados (13 diagramas, 210 entradas de bitácora, 88 mensajes); HTTPS válido; subida, descarga y borrado en S3 desde el backend
IA multifuncional	`4150960`	02/11	Revisiones con los resultados reales del validador, proyecto completo de una sola vez, y respuestas sin notas técnicas cuando nadie pidió un cambio	17 tests nuevos; prueba real con Gemini y con Ollama, y con un segundo usuario conectado

Tamaño del cambio del 2do Parcial: 94 archivos, **+8.673 / −252 líneas** (`348f1aa..b014a42`).

7.4 Evidencias de pruebas funcionales

7.4.1 Pruebas automatizadas

- **Backend: 152 pruebas unitarias en 22 archivos, todas aprobadas** (`npx vitest run`, 02/11/2026; la Captura 3 muestra la corrida del 17/10, con 117, antes del interruptor configurable). Cubren el generador y sus plantillas, validación, XMI, autoría y bitácora (`authorship.util.spec.ts`), saneamiento del modelo, motor híbrido (`hybrid.provider.spec.ts`), prompt del moderador, detección de pedidos explícitos y validación de operaciones de la IA.
- **Frontend:** compilación TypeScript y build de Vite sin errores; linter sin advertencias nuevas.
- **Flutter:** `flutter analyze` sin problemas en el código nuevo; APK de release compilado.



Captura 3: salida de `npx vitest run` en el backend: 19 archivos y 117 pruebas aprobadas.

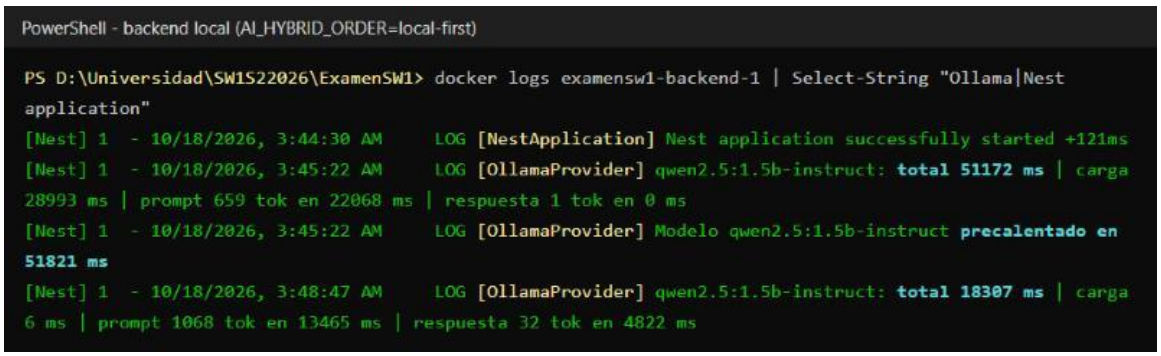
7.4.2 Motor de IA híbrido — mediciones reales (laptop del equipo, CPU)

Escenario	Proveedor que respondió	Tiempo
`cloud-first` (por defecto)	Gemini Cloud	4,3 s
Gemini falla → respaldo automático	Ollama Local	13,5 s
`local-first`, modelo precalentado	Ollama Local	5,0 s
Modelo local con el prompt completo (antes de optimizar)	Ollama Local	36–79 s
Modelo local con prompt lite + caché (después)	Ollama Local	5–18 s

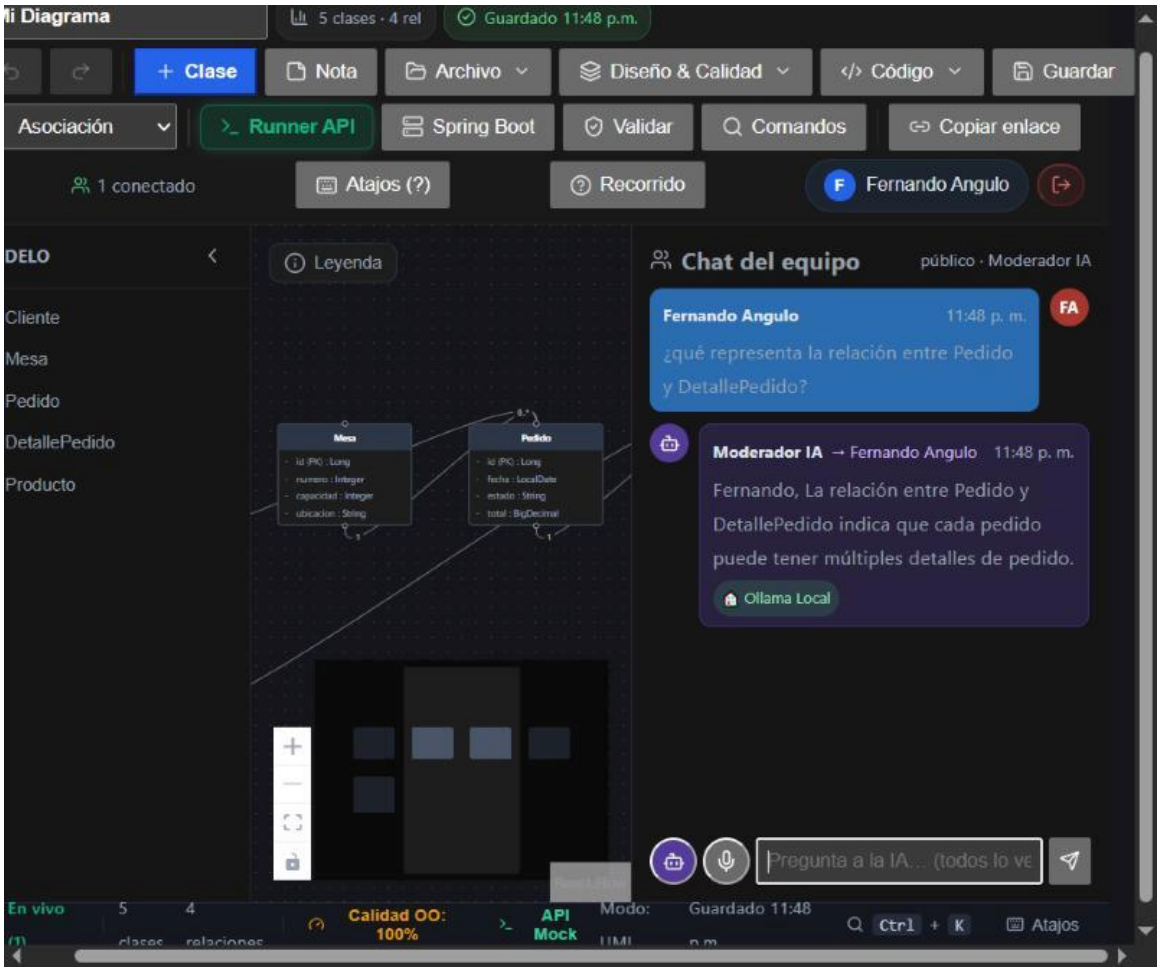
Comprobaciones de comportamiento del moderador:

- **Nombra a quien pregunta:** *“Trevor, un atributo derivado es…”*.

- **Contexto grupal:** respondió *“la clase Pedido fue creada por Marco…”* leyendo la bitácora.
- **Contexto individual:** *“como te comenté antes…”*.
- **Acción solo a pedido:** ante *“Agrega una clase Producto… muchos a muchos con Pedido”* propuso `CREATE_CLASS` + `MANY_TO_MANY`. Ante preguntas, cero operaciones.



Captura 4: log del backend local (AI_HYBRID_ORDER=local-first) con el precalentamiento del modelo y los tiempos de cada respuesta de Ollama.

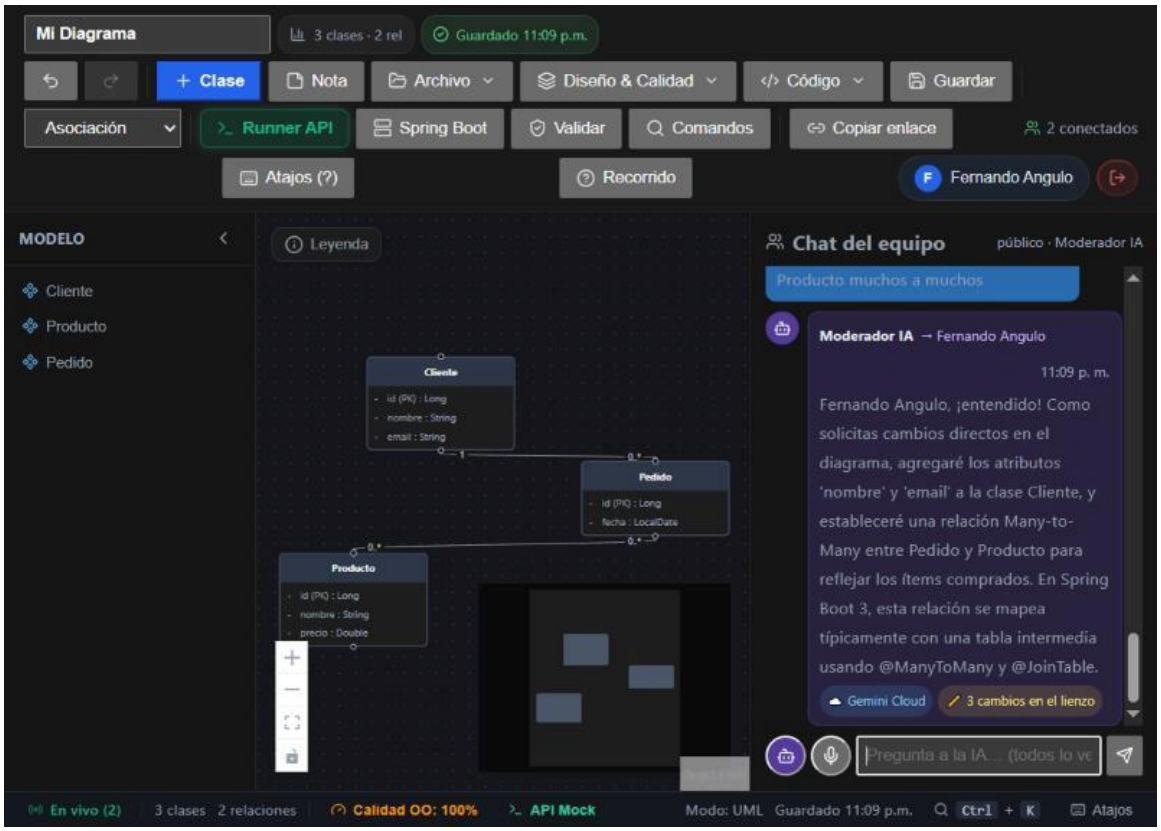


Captura 5: el Moderador IA responde con el modelo local (badge «🏠 Ollama Local»); en producción la misma vista muestra «☁ Gemini Cloud» (Captura 6).

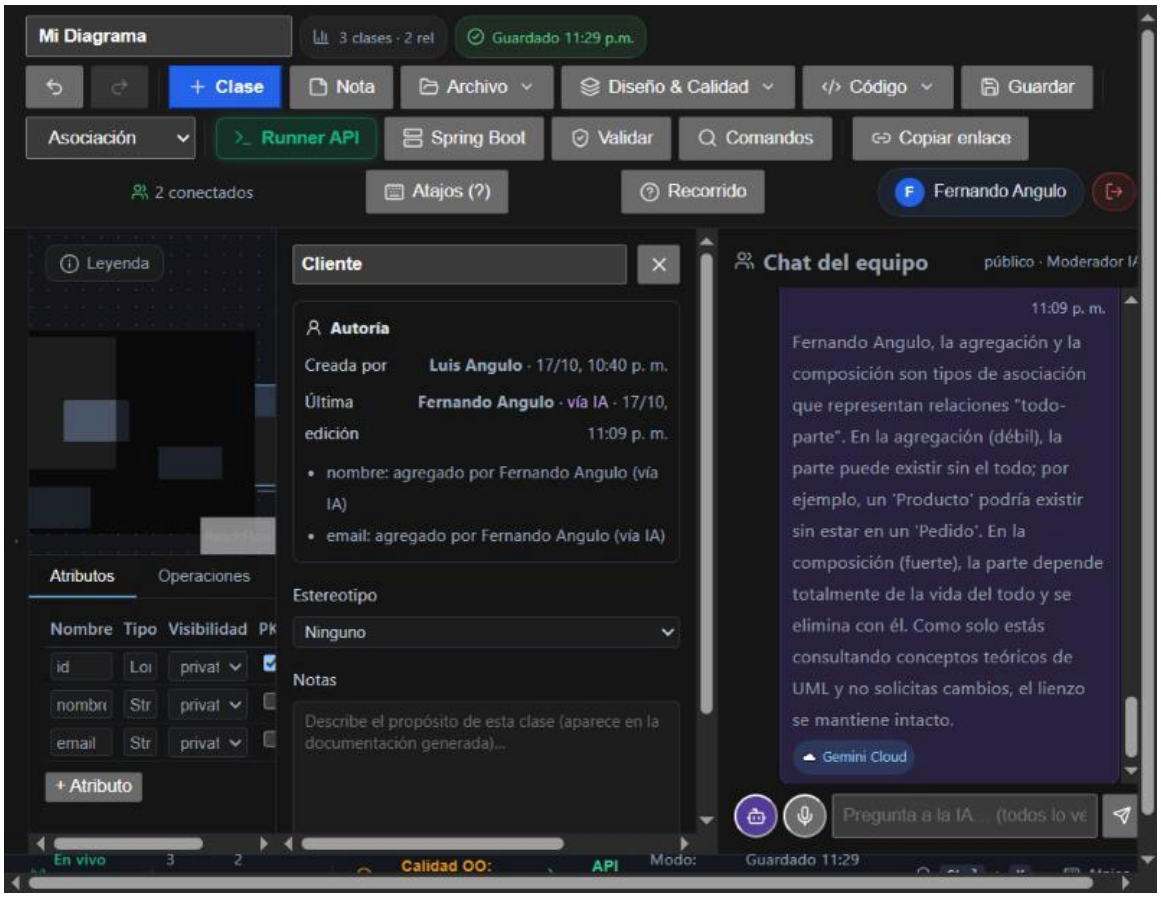
7.4.3 Prueba de colaboración con dos usuarios (navegador)

Dos sesiones simultáneas (Marco y Luis) sobre el mismo diagrama:

1. Luis pide a la IA crear `Producto` con relación N:M a `Pedido`.
2. La pestaña de Marco muestra en vivo *"La IA está pensando la respuesta para Luis"*, la respuesta *"Moderador IA → Luis"* y la clase nueva en su lienzo.
3. El inspector de Marco muestra *"Creada por Luis · vía IA"*.
4. La bitácora registra *"Luis: Creó la clase Producto (vía IA)"*, sin entradas falsas.
5. La barra indica **"2 conectados"**.



Captura 6: chat público en producción. Fernando pide un cambio y el Moderador IA lo aplica en el lienzo ("3 cambios en el lienzo", Gemini Cloud).

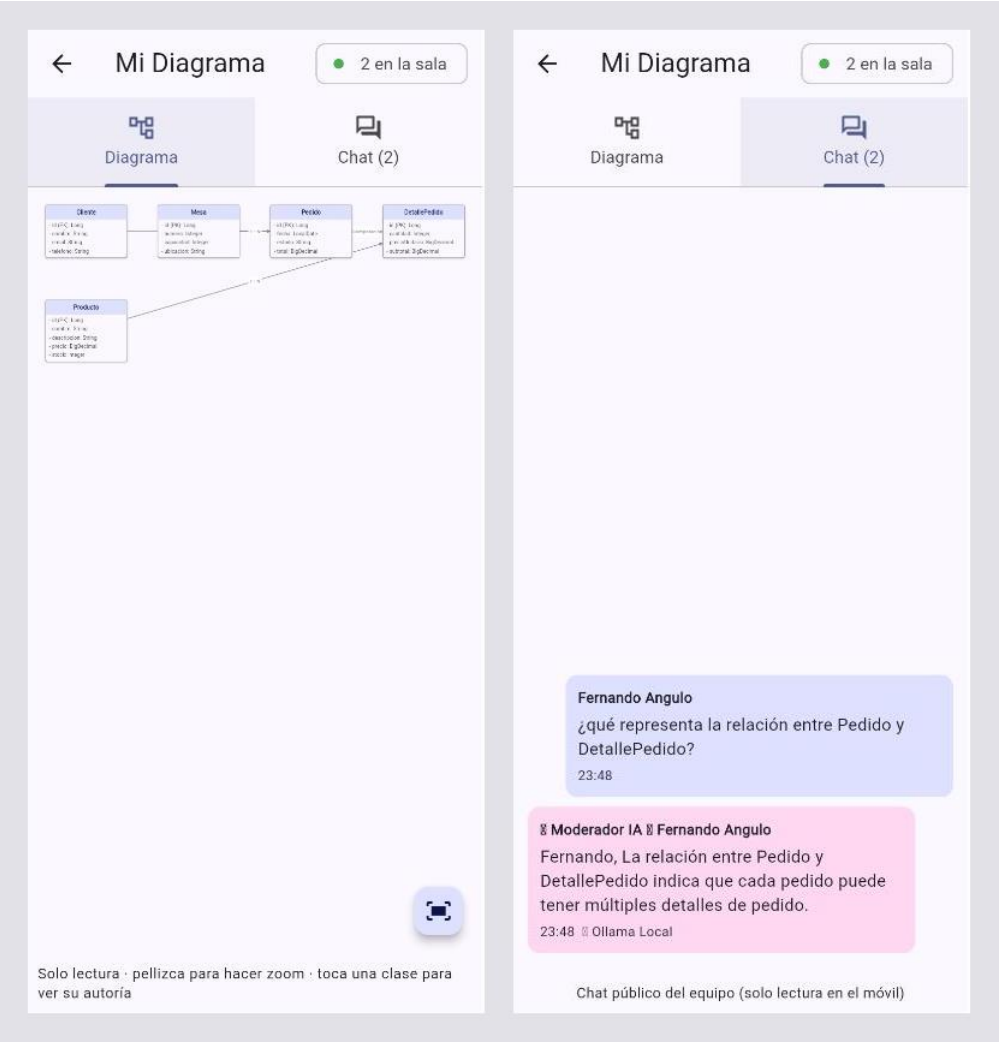


Captura 7: inspector de la clase Cliente con su autoría: creada por Luis y editada por Fernando vía IA, con los atributos agregados por el moderador.

7.4.4 Visor móvil en tiempo real

Con el visor Flutter abierto en el diagrama, Marco pidió desde la web *"Agrega el atributo fecha de tipo LocalDate a Pedido"*. **Sin recargar**, el móvil mostró:

- el atributo nuevo;
- el chat actualizado (de 2 a 4 mensajes);
- *"2 en la sala"*;
- en la hoja de detalle de `Producto`, *"Creada por: Luis · vía IA"*.



Captura 8: visor Flutter en vista móvil (375×812): el diagrama en solo lectura con «2 en la sala» y la pestaña de chat con la respuesta del Moderador IA.

7.4.5 Prueba de regresión del 1er Parcial (18/10/2026)

Regla de oro del enunciado: *todo lo del 1er Parcial debe seguir funcionando*. Se volvió a probar sobre el diagrama del restaurante (6 clases, 5 relaciones), con la versión que incluye el Moderador IA:

Función del 1er Parcial	Cómo se probó	Resultado
Exportar e importar XMI (Enterprise Architect)	`GET /diagrams/:id/xmi` → `POST /diagrams/import-xmi`	✅ Ida y vuelta idéntica: 6 clases con sus atributos y 5 relaciones del mismo tipo
Generar el backend Spring Boot	ZIP del generador → `docker compose up` (PostgreSQL + Spring Boot)	✅ Compila y arranca; Swagger en `/swagger-ui.html`
Probar con Postman	Las 30 requests de la colección	✅ **30/30 con 2xx** (CRUD de las 6

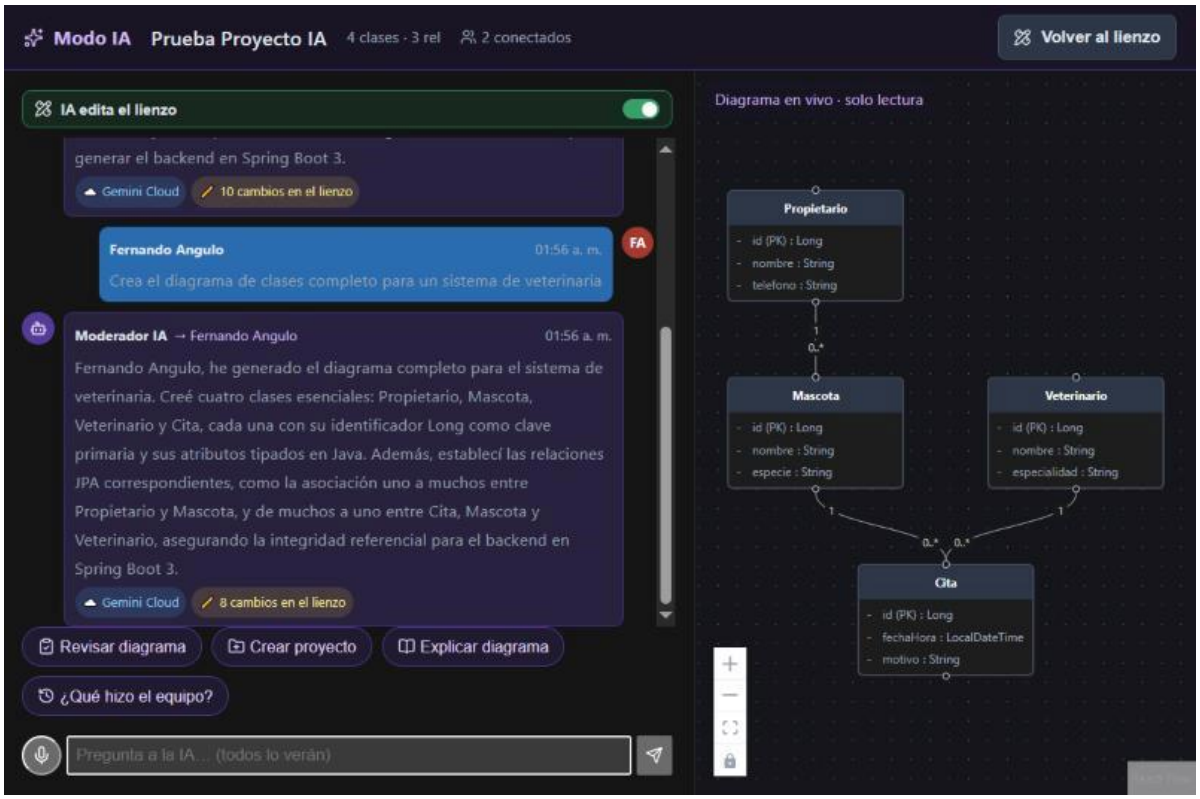
Función del 1er Parcial	Cómo se probó	Resultado
	generada, en orden (como el *Runner*)	entidades, incluida `Reserva`, creada por la IA)
Edición del diagrama por chat	Chat del equipo con pedidos explícitos	✅ Probado en producción (Capturas 5 y 6)
Dictado por voz	Luis dictó “Agrega la clase factura con número y total” en el chat del equipo, en producción	✅ La IA respondió por su nombre y creó `Factura` (Figura A.7b)
Foto de pizarra	*Archivo → Foto de pizarra (IA)* con un diagrama dibujado a mano, en producción	✅ Reconoció 5 clases con sus atributos y relaciones (Figura A.7a)

Limitación heredada del 1er Parcial, corregida el 05/11/2026: los datos de `data.sql` se cargaban en las tablas de un `schema.sql` propio (`cliente`) y no en las que mapea JPA (`clientes`), así que la API arrancaba vacía. Ahora la base la crea solo el ORM y `data.sql` usa las mismas tablas y claves foráneas que las entidades (defecto 13; prueba completa en 2.4.7).

7.4.6 Modo IA e IA multifuncional (02/11/2026)

Pedido	Motor	Tiempo	Resultado
“Crea el diagrama de clases completo para un sistema de biblioteca”	Gemini	3 s	✅ 5 clases con PK y atributos, 4 relaciones, diagrama ordenado
“…para un sistema de veterinaria”, desde el Modo IA	Gemini	< 10 s	✅ 4 clases y 3 relaciones en la vista previa; guardado con autoría “vía IA” y recibido en vivo por un segundo usuario conectado
“Revisa nuestro diagrama…”	Ollama	38 s	✅ Observación del modelo más el resultado real del validador, escrito por el servidor
“Crea el diagrama completo…”, **sin internet**	Ollama	25 s	⚠ Solo 2 clases

Limitación conocida: el modelo local (qwen2.5 de 1,5 mil millones de parámetros, en CPU) no alcanza para modelar un sistema completo. Por eso, en estos pedidos el motor híbrido intenta primero con la nube aunque el orden sea `local-first`. Sin internet responde el modelo local, con un resultado parcial.



Captura 13: Modo IA. El chat al centro con sus accesos directos y el diagrama como vista previa en vivo, de solo lectura, después de pedir un proyecto completo.

7.4.7 Prueba del backend generado contra las reglas de OMT (05/11/2026)

Se generó un backend con todos los casos de mapeo y se levantó con PostgreSQL (`backend/scripts-prueba/generar-prueba-omt.mjs`, `docker compose up`). El modelo de prueba tiene herencia con una clase abstracta (Persona → Cliente, Empleado), una relación N:M (Producto–Categoría), una composición (Pedido ◆— LíneaPedido), una agregación (Tienda ◇— Empleado), una 1:1 (PerfilCliente–Cliente) y un enum (EstadoPedido).

Regla (OMT y UML)	Resultado en la base que crea Hibernate
Una tabla por clase	10 tablas, sin duplicados
Herencia JOINED	`empleados.id` es clave foránea de `personas.id`; los ids siguen una secuencia compartida
N:M con tabla intermedia	`producto_categorias (producto_id, categoria_id)` con 3 filas
FK del lado “muchos” o de la parte	`linea_pedidos.pedido_id` y `empleados.tienda_id`
Composición: la parte muere con el todo	`DELETE /api/pedidos/1` → 204, y sus líneas se borran
Agregación: la parte sobrevive	`DELETE /api/tiendas/1` → 204, y el empleado queda sin tienda

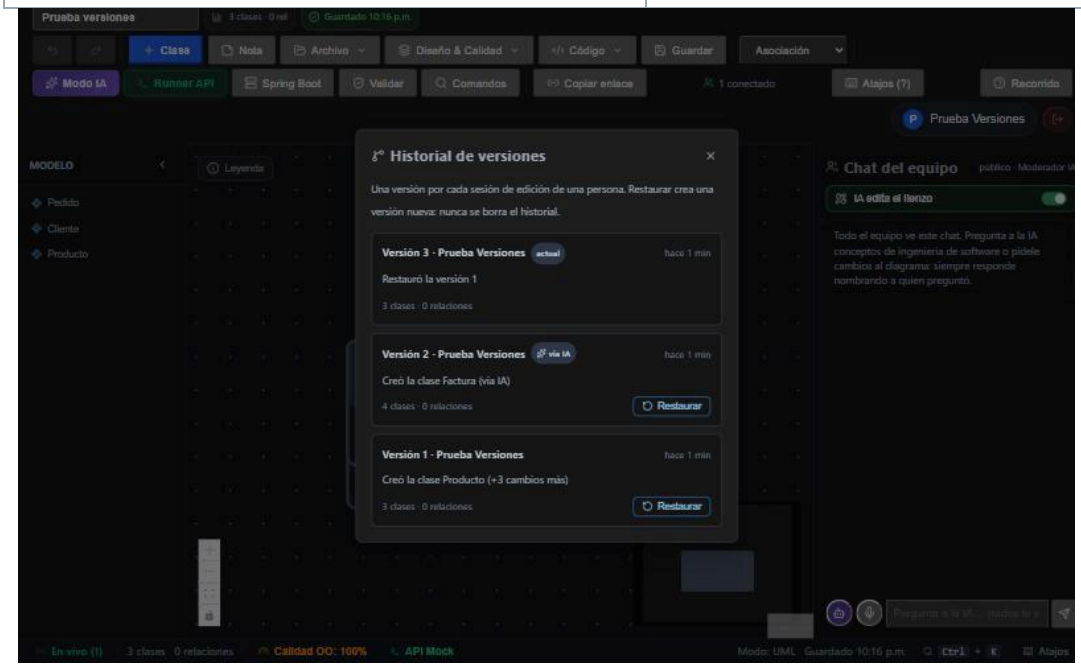
Regla (OMT y UML)	Resultado en la base que crea Hibernate
Datos de prueba visibles en la API	`GET /api/clientes` devuelve los clientes con sus pedidos y el enum `PENDIENTE`

La primera corrida encontró los defectos 13 a 16 de la tabla siguiente; esta tabla muestra el resultado después de corregirlos.

7.4.8 Historial de versiones (05/11/2026)

`backend/scripts-prueba/probar-versiones.py`, contra la pila local:

Paso	Resultado
Crear el diagrama y guardar dos veces seguidas (misma persona)	✅ Una sola versión (v1, 4 cambios): el autoguardado no llena el historial
Un cambio aplicado por el Moderador IA	✅ Versión propia (v2, «vía IA»)
Restaurar v1	✅ El diagrama vuelve a sus 3 clases y se crea **v3** (“Restauró la versión 1”); v1 y v2 siguen en el historial
Bitácora	✅ “Restauró la versión 1 del diagrama” y el efecto (“Eliminó la clase Factura”)
Restaurar desde el panel, en el navegador	✅ El lienzo se recarga con el modelo restaurado; sin errores en la consola



Captura 14: Archivo → Historial de versiones. La versión actual, un cambio «vía IA» y el botón Restaurar en las anteriores.

7.5 Registro de defectos encontrados y corregidos

Las pruebas con dos usuarios reales encontraron defectos que las pruebas unitarias no veían. Este registro es evidencia de un proceso de verificación efectivo.

#	Defecto	Causa raíz	Corrección	Commit
1	Los mensajes WebSocket fallaban al unirse a la sala	El guard JWT global también interceptaba Socket.io (sin cabeceras HTTP)	El guard solo valida HTTP; la identidad del socket se toma del *handshake*	`cfdd410`
2	La ruta `/sessions` daba 404 en producción	nginx no la enrutaba al backend	Se agregó a la regla del proxy	`803ab14`
3	Respuesta de Ollama en 36–79 s	Prompt de 1.380 tokens en CPU (~50 tok/s)	Prompt lite + caché de prefijo + precalentamiento	`679ed16`, `602a785`
4	La IA local creaba clases que nadie pidió	El ejemplo few-shot de edición se colaba en el moderador	Ejemplos propios + filtro determinista de pedidos explícitos	`679ed16`
5	Una pestaña en segundo plano dejaba de sincronizar	El flag remoto se reseteaba con `requestAnimationFrame`, que no se ejecuta en pestañas ocultas	El flag se consume en el propio efecto de difusión	`814d912`
6	Las clases perdían su posición al recargar (defecto del 1er Parcial, desde `4198c8b`)	`PositionDto` sin decoradores: el `ValidationPipe` borraba `x`/`y`	Decoradores de validación + saneamiento del modelo	`814d912`
7	Entradas falsas en la bitácora (“modificó estereotipo…”)	Mongo guardaba `undefined` como `null`	`sanitizeModel` + `ignoreUndefined`	`814d912`
8	El botón del visor quedaba cortado en celulares de 375 px	La AppBar no tenía espacio	Botón flotante “Visor UML en vivo”	`e5eeae3`
9	En producción la IA decía “renombré y agregué” pero el lienzo no cambiaba	El validador no seguía un renombre dentro del mismo lote, y “sí, hazlo” no contaba como pedido explícito	El validador sigue renombres y eliminaciones; se aceptan confirmaciones; si se descarta algo, la	`4daba22`

#	Defecto	Causa raíz	Corrección	Commit
			respuesta lo aclara	
10	En modo «solo sugerencias», el modelo local respondía “agregué la clase…” sin haberla agregado	El modelo pequeño ignora la instrucción del prompt	El servidor reescribe la respuesta con los pasos a seguir, generados desde las operaciones	`d003cf8`
11	Con el *Runner* de Postman, 6 de 30 requests daban 404 sobre una base vacía	La colección generada hacía `GET /{id}` antes del `POST`	Cada carpeta ahora es POST → GET → GET/{id} → PUT → DELETE (30/30)	`9faa2f3`
12	Al cambiar de diagrama, el navegador seguía recibiendo los cambios del diagrama anterior	El servidor nunca sacaba al socket de la sala vieja	Al unirse a otra sala (o con `leave-diagram`) se sale de la anterior, liberando locks y presencia; nuevo *Archivo → Nuevo diagrama*	`1a2fce3`
13	El backend generado devolvía listas vacías aunque tenía datos de prueba, y la base tenía cada tabla dos veces (`persona` y `personas`)	El proyecto ejecutaba un `schema.sql` propio, con otros nombres de tabla, además de las tablas que crea Hibernate; el `data.sql` llenaba las del script	La base la crea solo el ORM (Hibernate, desde las entidades); `data.sql` se genera con las mismas tablas y claves foráneas que las entidades	`e4b9fbd`
14	Una agregación Tienda ◇— Empleado generaba “una tienda tiene un empleado”	La agregación se trataba como una asociación simple, con la FK en el todo	El todo lista sus partes y la parte guarda la FK con `ON DELETE SET NULL` (la parte sobrevive a su todo)	`e4b9fbd`
15	Borrar un cliente borraba también sus pedidos	Toda relación 1..* llevaba `cascade = ALL` y `orphanRemoval`	Solo la **composición** borra sus partes; las asociaciones conservan la integridad referencial	`e4b9fbd`
16	La herencia no tenía estrategia	Sin `@Inheritance` (Hibernate usaba	`@Inheritance(JOINED)`: una tabla por	`e4b9fbd`

#	Defecto	Causa raíz	Corrección	Commit
	explícita y los datos de prueba de enums y precios eran inválidos	una sola tabla); los enums recibían `EstadoPedido_OPCION_1`; “pre**ci**o” se tomaba por un número de carnet (CI)	clase, como en OMT; literales reales del enum; la detección de CI solo con la palabra completa	

7.6 Trazabilidad requisito → código → prueba (insumo para V&V)

Requisito del 2do Parcial	Implementación	Verificación
La IA nombra a quien responde	`moderator-prompt.ts` (regla 1) + `ensureAddressed()`	`moderator-prompt.spec.ts`; respuestas reales
Chat completamente público	`chat.gateway.ts` (difunde a toda la sala)	Prueba con dos usuarios
Mentor teórico y práctico	Prompt del moderador + operaciones + `intent.ts`	`intent.spec.ts`, `validate-operations.spec.ts`
El Modo IA activa una nueva vista y un nuevo enfoque	`AiModeBar.tsx`, `ChatPanel.tsx` (`aiMode`), estilos `.app--ai-mode`	Prueba en navegador a 1100 px y 375 px
IA multifuncional: revisiones y hacer un proyecto	`intent.ts` (`classifyRequest`), `moderator-prompt.ts`, `moderator.service.ts` (`validatorSummary`)	`intent.spec.ts`, `moderator-prompt.spec.ts`; pruebas reales (2.4.6)
Ejecutor **configurable**	`aiCanEditCanvas` en el diagrama + `ai:settings` + `canvas-policy.ts`	`canvas-policy.spec.ts`; prueba en ambos modos
Contexto grupal e individual	`moderator.service.ts` (bitácora, sala, hilo)	Respuestas “creada por Marco” / “como te comenté antes”
IA híbrida local + nube	`hybrid.provider.ts`	`hybrid.provider.spec.ts`; tres escenarios medidos
Modelo local configurado por el equipo	`ollama-modelfile.ts` + `backend/ollama/Modelfile`	`ollama-modelfile.spec.ts`, `ollama.provider.spec.ts`; `ollama run moderador-uml`
Autoría granular	`authorship.util.ts`	`authorship.util.spec.ts`; inspector y visor
Sesiones continuas	`sessions.service.ts`	E2E: pausa y reanudación con el mismo ID
Visor móvil en tiempo real	`flutter-template/lib/live/`	Prueba en vivo web → móvil; APK compilado
No romper el 1er Parcial	Rama aislada + interfaz de	E2E de XMI, generador y locks

Requisito del 2do Parcial	Implementación	Verificación
	`DiagramsService` intacta	

7.7 Herramientas utilizadas y uso de asistencia de IA

- **Desarrollo:** Git y GitHub, Node.js 24, NestJS 12, React + Vite, Flutter 3.38, Docker Desktop, MongoDB 7, PostgreSQL 16, Ollama 0.40.
- **Modelado (herramienta CASE):** los diagramas UML del Ciclo 2 (casos de uso, comunicación, clases de análisis, paquetes, clases de diseño, secuencia, estados, actividad, componentes y despliegue) están hechos en **Enterprise Architect 13.5**, en el proyecto `docs/diagrams/ea/ciclo2.eap`. Primero se escribió un borrador textual de cada diagrama a partir del código (`docs/diagrams/2do_parcial/`); el script `docs/diagrams/ea/ea_builder.py` lo construyó en Enterprise Architect por automatización (su API COM). Las imágenes del documento se exportaron desde Enterprise Architect, donde cada diagrama se puede abrir, revisar y editar.
- **Asistente de programación:** el equipo trabajó con **Claude Code** (Anthropic) como asistente de programación. Por transparencia, los commits en que participó llevan la línea `Co-Authored-By: Claude` (82 de 89 commits). El equipo definió los requisitos y el plan, tomó las decisiones de diseño (persistencia políglota, orden del motor híbrido, alcance de cada semana), ejecutó las pruebas y revisó cada cambio antes de integrarlo.
- Todos los commits se hicieron desde la cuenta `luisfernandoAngulo28`.

7.8 Pendientes registrados (al 18/10/2026)

- ☒ Desplegar en el VPS: levantar Mongo y correr una sola vez `migrate-pg-to-mongo.js` (17/10).
- ☐ Probar las notificaciones en un Android real (en web no se pudo conceder el permiso).
- ☐ Probar el corte de internet real (WiFi apagado) con `cloud-first`.
- ☐ Limitación conocida: el guardado reemplaza el modelo completo (*last-write-wins*), mitigado con locks por clase.
- ☐ Simulacro final con tres usuarios + visor móvil.

PARTE 3

Mecanismo para que el Usuario Utilice la Aplicación

3.1. Un agente asistente del software

El mecanismo principal mediante el cual el usuario interactúa con la aplicación no se limita al lienzo de arrastrar y soltar: la herramienta incorpora un agente asistente de IA, visible como un panel de chat permanente junto al lienzo, que acompaña al usuario durante todo el proceso de modelado.

Este agente cumple tres funciones como mecanismo de uso:

- **Guía conversacional:** el usuario puede preguntarle sobre modelado UML, normalización o arquitectura en capas, y el agente responde dentro de ese dominio, sin desviarse a temas ajenos al proyecto.
- **Canal de entrada alternativo a dibujar:** en lugar de arrastrar manualmente cada clase, el usuario puede dictar por voz un comando ("crea dos clases relacionadas") o mostrarle una fotografía de un diagrama ya dibujado en una pizarra o cuaderno, y el agente traduce esa entrada —hablada o visual— directamente en clases y relaciones sobre el lienzo.
- **Reducción de la barrera de entrada:** un usuario sin experiencia formal en UML puede describir su necesidad en lenguaje natural y obtener un primer borrador del modelo, que luego refina manualmente en el lienzo.

De esta forma, el agente asistente no es una función aislada, sino el mecanismo transversal que conecta los tres canales de entrada del sistema —dibujo manual, voz y fotografía— en una sola experiencia de modelado.

3.2. Manual de Usuario

Esta sección describe, paso a paso, el uso funcional de la herramienta desde la perspectiva del usuario final: cómo acceder, modelar un diagrama de clases, validarlo, generar el backend Spring Boot correspondiente, exportarlo a XMI y apoyarse en el asistente de IA descrito en la sección 3.1.

3.2.1 Acceso a la herramienta

La aplicación es accesible desde cualquier navegador web moderno en la dirección del servidor donde esté desplegada. A diferencia de un sistema empresarial con roles diferenciados, en el primer ciclo el lienzo no requería registro; desde el segundo ciclo se inicia sesión para que el equipo y el Moderador IA sepan quién es cada integrante (ver 3.2.8). Para trabajo colaborativo, el enlace de un diagrama específico (generado al guardarlo por primera vez) es lo que se comparte entre los miembros del equipo.

3.2.2 Modelado del diagrama de clases

El lienzo central permite construir el diagrama de clases de forma completamente visual, sin escribir código:

- **Crear una clase:** el botón “+ Clase” agrega un nuevo nodo UML al lienzo con un atributo “id” marcado como clave primaria por defecto.
- **Editar una clase:** un clic sobre el nodo abre el panel inspector, donde se renombra la clase y se agregan, editan o eliminan atributos (nombre, tipo de dato, visibilidad y si es clave primaria).
- **Crear una relación:** se elige el tipo de relación (asociación, agregación, composición, herencia, uno a uno, uno a muchos, muchos a uno o muchos a muchos) en el selector de la barra de herramientas y luego se arrastra desde el borde de una clase hasta otra; la etiqueta del tipo elegido queda visible sobre la línea de conexión.

3.2.3 Validación lógica del modelo

Antes de generar código, el botón ” Validar diagrama” ejecuta un análisis automático del modelo sobre tres frentes: integridad estructural (clases sin clave primaria, nombres o atributos duplicados, relaciones que apuntan a una clase inexistente), redundancia por desnormalización según la Tercera Forma Normal (atributos que copian datos de una clase ya relacionada) y claves foráneas manuales redundantes (atributos tipo “clienteId” cuando la relación ya genera esa clave automáticamente). Los resultados se muestran en un panel lateral, separando errores —que impiden un modelo relacional correcto— de advertencias de buenas prácticas.

3.2.4 Guardado, colaboración y recorrido guiado

- **Guardar el diagrama:** el botón “Guardar diagrama” persiste el modelo actual y, en el primer guardado, asigna un identificador único que queda reflejado en la URL del navegador.

- **Colaborar en tiempo real:** el botón ” Copiar enlace” genera una URL compartible; cada persona que la abre se une a la misma sesión sobre WebSockets, y los cambios de todos se propagan al instante entre los conectados, con un contador visible en la barra superior.
- **Recorrido guiado:** en la primera visita, un tour de siete pasos resalta cada control principal de la barra de herramientas; puede repetirse en cualquier momento con el botón “❓ Recorrido”, útil tanto para nuevos usuarios como para una demostración en vivo.

3.2.5 Generación del backend y exportación a XMI

- **Generar backend Spring Boot:** descarga un proyecto Spring Boot completo en formato .zip, con arquitectura de cuatro capas (Entity, Repository, Service, Controller) generada a partir del diagrama, listo para importarse en un IDE Java y ejecutarse contra PostgreSQL.
- **Exportar a XMI:** descarga el diagrama en formato XMI 2.1 / UML 2.1, importable directamente en Enterprise Architect u otras herramientas UML estándar, para su uso fuera de la plataforma.

3.2.6 Uso del asistente de IA

El panel de chat, siempre visible junto al lienzo, admite tres canales de entrada descritos en la sección 3.1: texto libre, dictado por voz (icono de micrófono, vía Web Speech API) y una fotografía de un diagrama dibujado a mano (botón “📷 Foto de pizarra”), que el asistente interpreta y traduce en clases y relaciones agregadas automáticamente al lienzo.

3.2.7 Conexión con la aplicación móvil

Una vez generado y desplegado el backend, el usuario final lo conecta a una aplicación móvil desarrollada en Flutter. El proyecto incluye una plantilla reutilizable (flutter-template) con un cliente REST genérico cuyo endpoint y esquema de datos se configuran en tiempo de ejecución —sin recompilar la app—, pensada para conectar en minutos cualquier backend generado por la herramienta durante una demostración en vivo.

3.2.8 Iniciar sesión (Segundo Ciclo)

Desde el segundo ciclo, la herramienta pide **registrarse e iniciar sesión** con correo y contraseña. El nombre de la cuenta es el que verá el equipo en el chat y en la autoría de cada clase.

3.2.9 Chat del equipo y Moderador IA

El panel derecho es el **chat público del equipo**: todos los conectados ven cada mensaje.

1. Con el **botón del robot activo**, el mensaje se envía también al **Moderador IA**. Todos ven *“La IA está pensando la respuesta para …”* y luego la respuesta, que empieza con tu nombre y muestra qué IA respondió (🏠 Ollama Local o ☁ Gemini Cloud).

2. Con el robot **apagado**, el mensaje es solo para el equipo.
3. Para que la IA **cambie el diagrama**, pídeselo con un verbo de acción (*"agrega la clase Producto con nombre y precio y relaciónala muchos a muchos con Pedido"*). El cambio aparece en el lienzo de todos y queda registrado como "vía IA".
4. El **micrófono** permite dictar la pregunta.

3.2.10 Ver quién hizo qué (autoría e historial)

- **Doble clic en una clase:** el inspector muestra *"Creada por …"*, *"Última edición …"* y el autor de cada atributo.
- **Botón Historial:** lista la bitácora del diagrama, con una entrada por cambio y el nombre de quien lo hizo.

3.2.11 Retomar el trabajo otro día

Al volver a abrir un diagrama, la herramienta **reanuda la misma sesión**: el chat anterior y la actividad reciente aparecen sin hacer nada.

3.2.12 Visor UML en vivo (celular)

1. En la app móvil, toca **"Visor UML en vivo"** e inicia sesión con la misma cuenta de la web.
2. Elige un diagrama: verás las clases y relaciones (pellizca para hacer zoom; el botón de ajuste encuadra todo el diagrama).
3. Cuando alguien cambia el diagrama, el visor lo actualiza y **resalta** la clase modificada.
4. **Toca una clase** para ver su autoría.
5. La pestaña **Chat** muestra la conversación del equipo con la IA.
6. Si concedes el permiso, recibirás **notificaciones** de mensajes y cambios.

3.3. Mecanismo de aprendizaje del usuario (Segundo Parcial)

3.3.1 Enfoque: la aplicación enseña a usarse a sí misma

El aprendizaje del usuario se diseñó en **cuatro niveles**, del más guiado al más autónomo. Un usuario nuevo los recorre en ese orden:

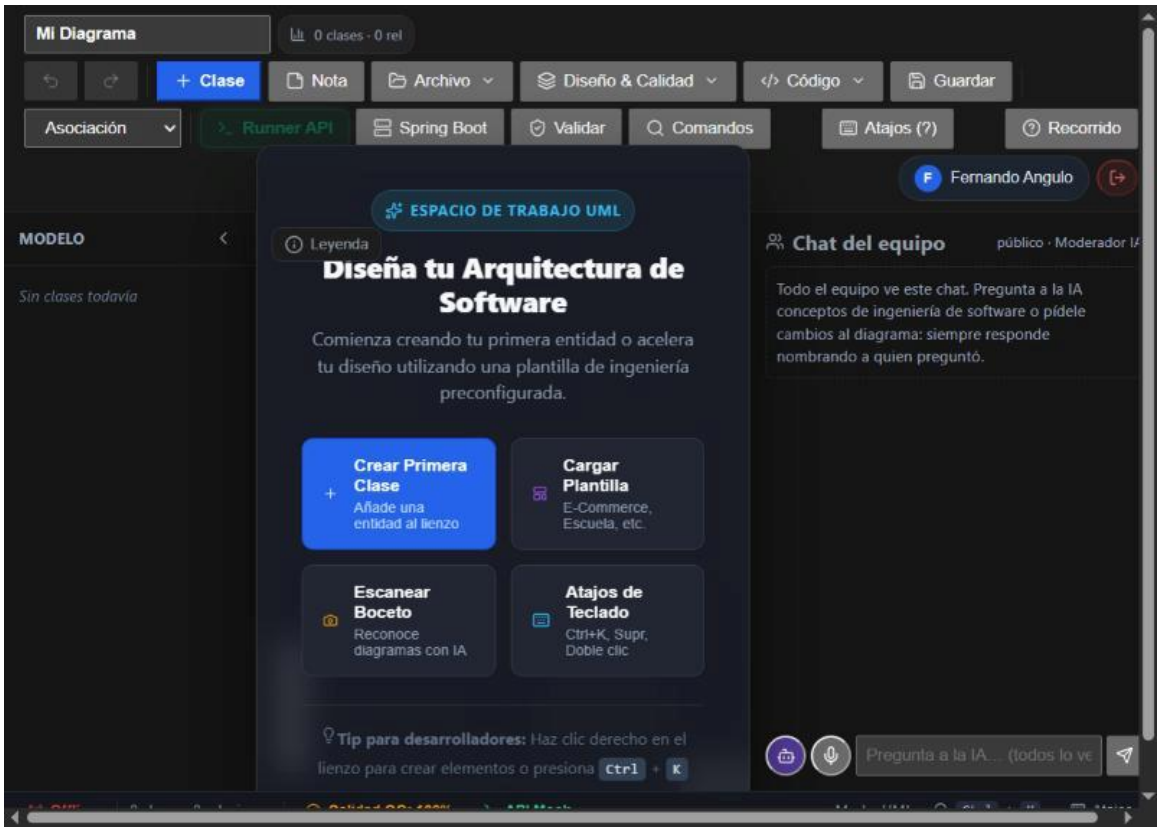
Nivel	Mecanismo	Cuándo aparece	Qué aprende el usuario
1. Primer contacto	Estado vacío del lienzo	Al abrir un diagrama sin clases	Por dónde empezar
2. Guiado	Recorrido (tour) de 7 pasos	Automáticamente la primera vez; luego con el botón **Recorrido**	El flujo completo: modelar → validar → generar → exportar → consultar a la IA
3. Consulta	Leyenda UML, atajos, paleta de comandos, plantillas, tooltips	Cuando el usuario los necesita	La notación UML y las funciones avanzadas
4. Retroalimentación	Validador del modelo y Moderador IA del chat	Mientras trabaja	Qué hizo mal y por qué; conceptos de ingeniería de software

3.3.2 Nivel 1 — Estado vacío del lienzo

Un lienzo en blanco no le dice nada a un principiante. Por eso, cuando el diagrama no tiene clases, se muestra una tarjeta *"Diseña tu Arquitectura de Software"* con cuatro puntos de partida:

1. **Crear primera clase:** agrega una entidad al lienzo.
2. **Cargar plantilla:** abre la galería de plantillas.
3. **Escanear boceto:** reconoce con IA un diagrama dibujado en papel o pizarra.
4. **Atajos de teclado:** muestra los atajos principales.

Implementación: `frontend/src/components/CanvasEmptyState.tsx`



Captura 9: estado vacío del lienzo con accesos rápidos y el chat del equipo.

3.3.3 Nivel 2 — Recorrido guiado (tour)

La primera vez que el usuario entra, la aplicación inicia sola un **tour de 7 pasos**. Cada paso **resalta el control real de la interfaz** y explica para qué sirve:

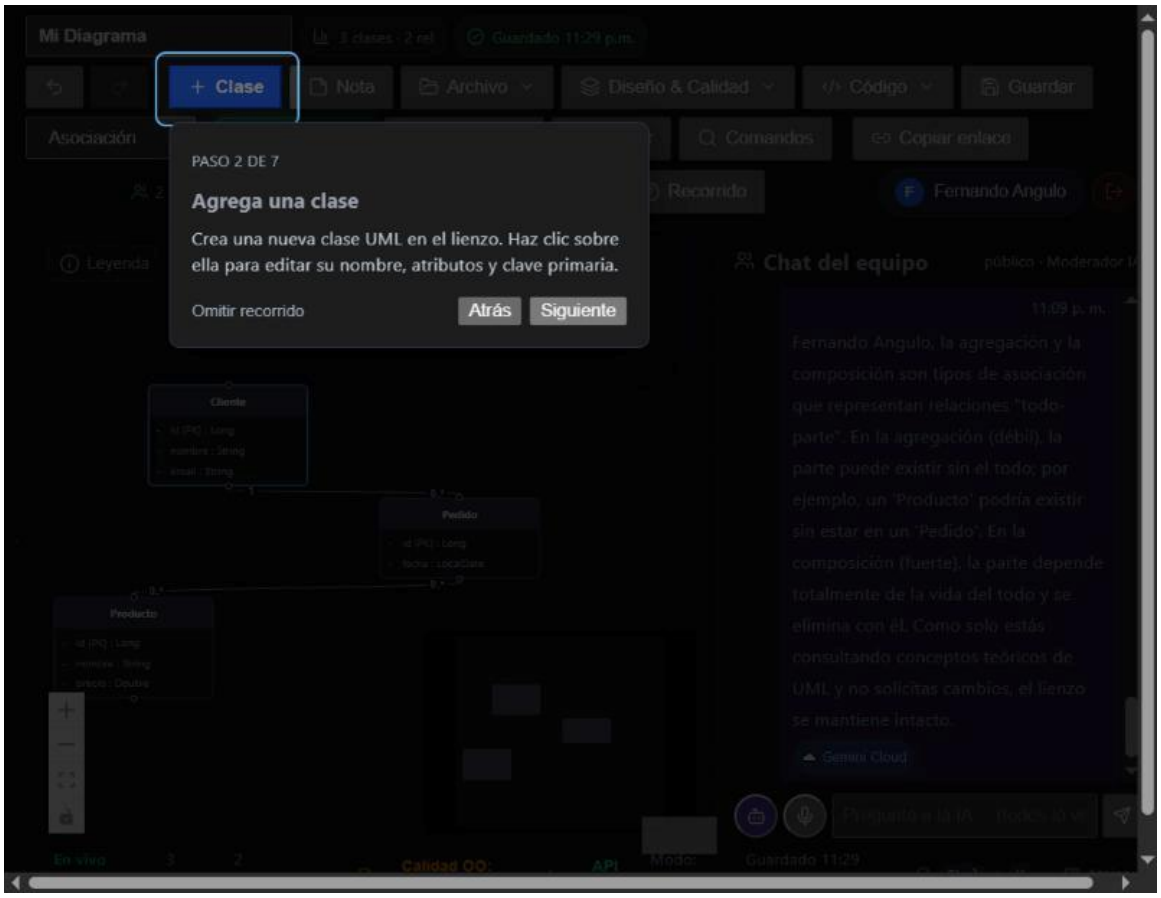
Paso	Control resaltado	Lo que explica
1	Nombre del diagrama	Con qué nombre se guarda el diagrama
2	Botón **+ Clase**	Crear una clase y editar sus atributos y su clave primaria
3	Selector de relación	Elegir el tipo de relación y arrastrar entre clases para conectarlas
4	**Validar**	Revisar integridad y normalización (3FN) antes de generar
5	**Spring Boot**	Descargar el backend completo de 4 capas
6	Exportar **XMI**	Llevar el modelo a Enterprise Architect (XMI 2.1)
7	**Chat del equipo**	El chat es público; el Moderador IA

Paso	Control resaltado	Lo que explica
		responde por nombre y solo cambia el diagrama si se lo piden

Detalles de diseño:

- **Se muestra una sola vez:** al terminar se guarda la marca `case-tool-tour-seen` en el navegador.
- **Se puede repetir:** el usuario lo vuelve a lanzar con el botón **Recorrido** de la barra de herramientas.
- **El globo de ayuda se reubica solo** (arriba, abajo o al costado del control) para no salirse de la pantalla.
- **2do Parcial:** el paso 7 se actualizó para presentar el **chat grupal público** en lugar del asistente privado del 1er Parcial.

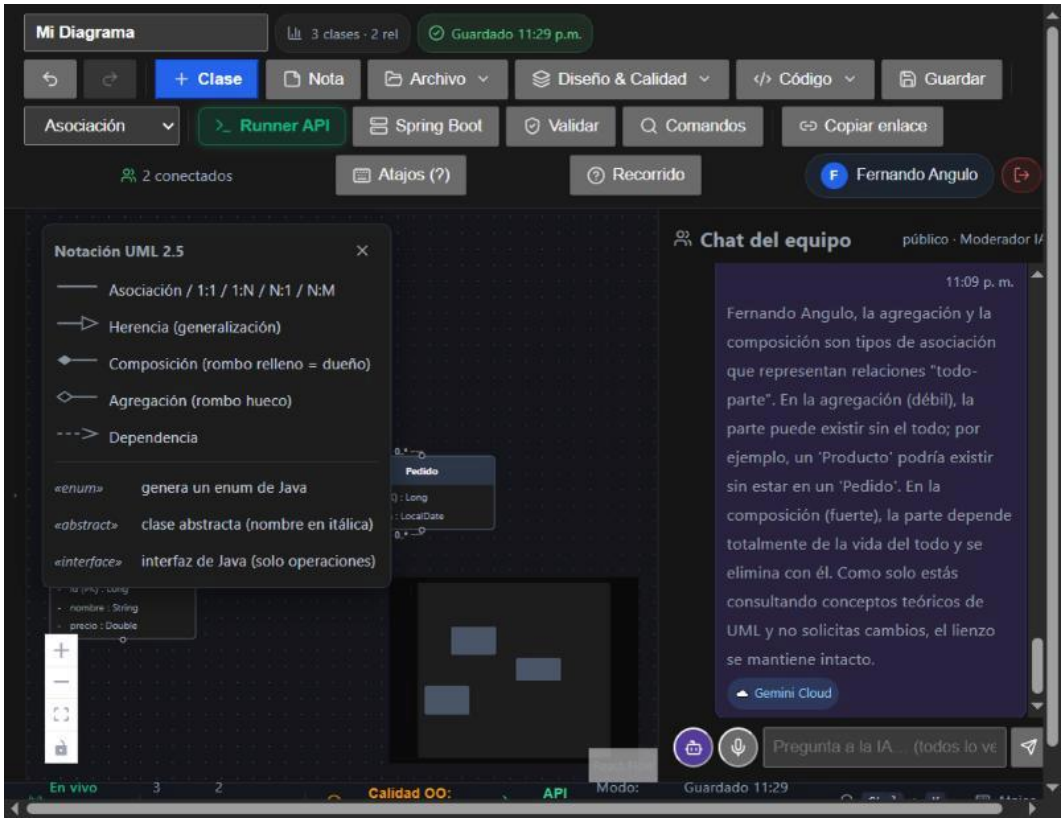
Implementación: `frontend/src/components/Tour.tsx` (componente); `TOUR_STEPS` y `TOUR_SEEN_KEY` en `frontend/src/App.tsx`.



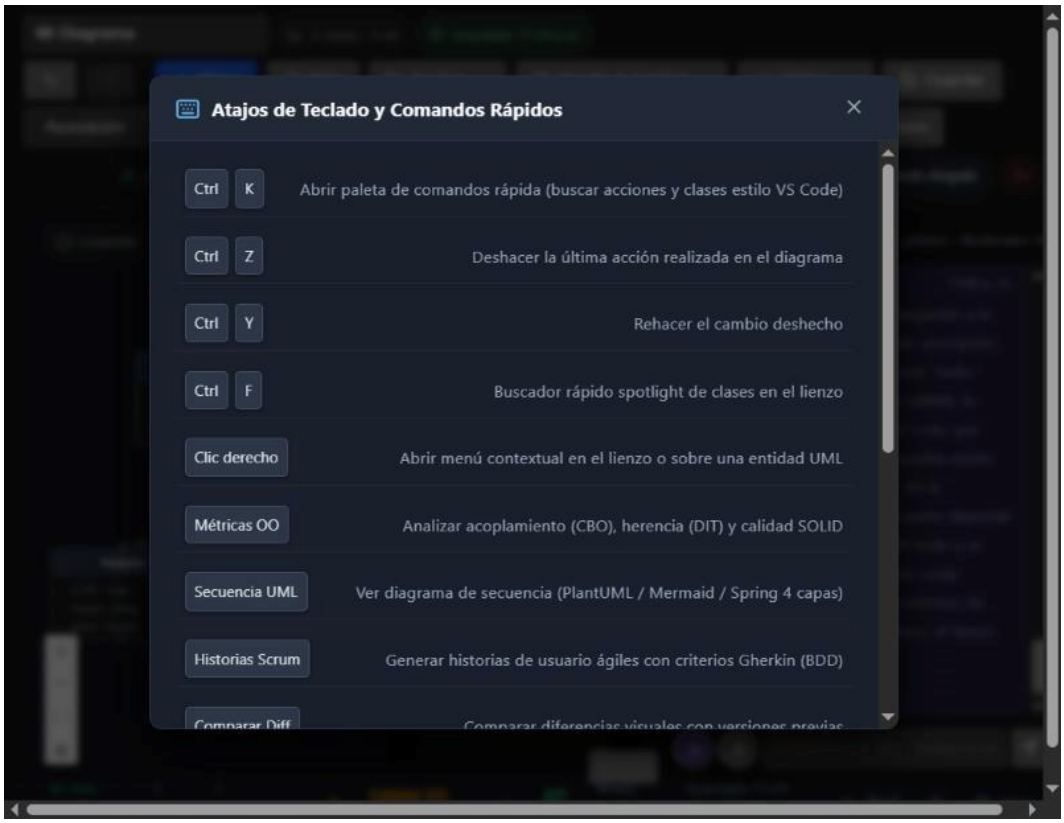
Captura 10: paso 2 de 7 del recorrido guiado, resaltando el botón "+ Clase".

3.3.4 Nivel 3 — Ayuda bajo demanda

- **Leyenda de notación UML 2.5** sobre el lienzo: muestra qué significa cada flecha y rombo (herencia, composición, agregación, dependencia…), para aprender la notación mientras se modela. `UmlLegend.tsx`
- **Guía de atajos de teclado** (15 atajos): `Ctrl+K`, `Ctrl+Z` / `Ctrl+Y`, `Ctrl+F`, `Supr`, doble clic para editar, arrastrar para relacionar, etc. `ShortcutsModal.tsx`
- **Paleta de comandos** (`Ctrl+K`, estilo VS Code): busca cualquier acción o clase escribiendo su nombre, sin conocer de antemano dónde está el menú. `CommandPaletteModal.tsx`
- **Galería de plantillas** (7): Herencia básica, Composición (todo-parte), Muchos a muchos, Asociación 1:N, Sistema de Restaurante, Sistema Clínico / Hospital y Sistema Académico Universitario. **Se aprende viendo ejemplos correctos** de cada tipo de relación. `TemplatesPanel.tsx`, `lib/templates.ts`
- **Tooltips descriptivos** en los botones de la barra, por ejemplo *"Deshacer (Ctrl+Z)"* o *"Abrir paleta rápida de comandos (Ctrl + K)"*.



Captura 11a: leyenda de notación UML 2.5 sobre el lienzo.



Captura 11b: guía de atajos de teclado y comandos rápidos.

3.3.5 Nivel 4 — Retroalimentación mientras se trabaja

3.3.5.1 El validador como maestro de modelado

Validar no solo dice “error”: explica **qué regla de modelado** se rompió. Estos son los mensajes reales:

- *“La clase”Pedido” no tiene ningún atributo marcado como clave primaria (PK).”*
- *“Hay 2 clases con el mismo nombre (”Cliente”). Los nombres de clase deben ser únicos.”*
- *“Hay una relación que apunta a una clase que ya no existe en el diagrama.”*
- Avisos de **normalización**: atributos desnormalizados y claves foráneas escritas a mano que deberían modelarse como relación.

Así el usuario aprende las reglas de OMT y de normalización **en el momento en que las necesita**. `backend/src/diagrams/validation.util.ts`, `ValidationPanel.tsx`

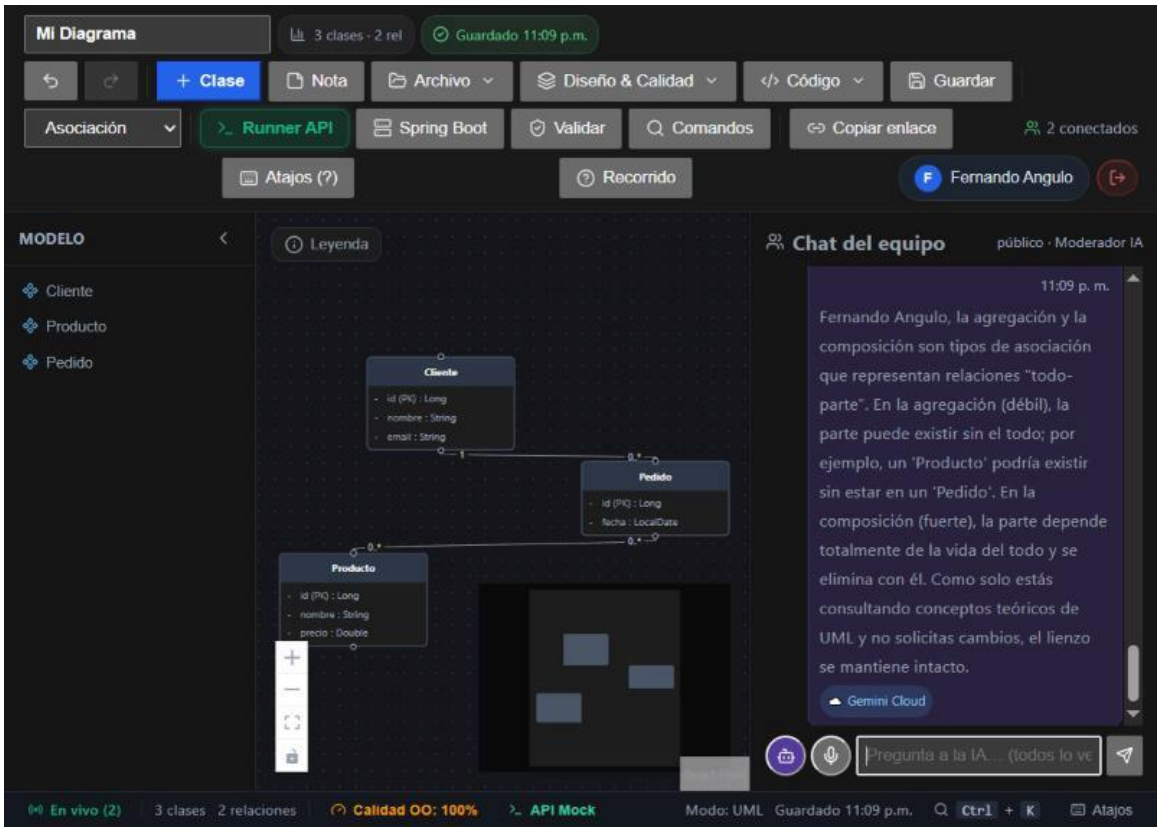
3.3.5.2 El Moderador IA: un agente de ayuda en línea (2do Parcial)

El chat público del equipo funciona como un **docente en el aula**:

- **Responde por nombre** a quien pregunta (*“Trevor, aquí tu respuesta…”*), y todos los integrantes ven la pregunta y la respuesta. **Lo que aprende uno lo aprende el equipo.**

- **Mentor teórico:** explica conceptos (atributo derivado, composición vs. agregación, normalización, JPA), con ejemplos tomados **del diagrama del propio equipo**.
- **Mentor práctico:** si se le pide explícitamente (*"agrega la clase Producto y relaciónala muchos a muchos con Pedido"*), aplica el cambio y lo deja marcado como **"vía IA"** en la autoría.
- **Memoria individual:** recuerda las preguntas previas de cada integrante y da continuidad (*"como te comenté antes…"*).
- **Funciona sin internet:** si Gemini no está disponible, responde el modelo local (Ollama).
- **Voz:** también se le puede preguntar por dictado.

`ChatPanel.tsx`, `backend/src/ai/moderator/`, `backend/src/ai/chat/chat.gateway.ts`



Captura 12: el Moderador IA responde por nombre una pregunta teórica y no modifica el lienzo.

3.3.6 Aprendizaje en el cliente móvil

El visor UML de Flutter está pensado para usarse sin instrucciones:

- **Acceso directo:** el botón flotante **"Visor UML en vivo"** en la pantalla principal.
- **Mismo login de la web:** el usuario entra con la misma cuenta y elige un diagrama de la lista.

- **Texto de ayuda en el lienzo:** *“Solo lectura · pellizca para hacer zoom · toca una clase para ver su autoría”*.
- **Notificaciones:** avisan de cambios y de respuestas de la IA, así el integrante que sigue la clase desde el celular se entera sin buscar.

`flutter-template/lib/live/uml_viewer_screen.dart`

3.3.7 Cómo aprende un usuario nuevo: recorrido de ejemplo

1. **Entra por primera vez:** el tour lo lleva en 7 pasos por todo el flujo.
2. **Ve el lienzo vacío:** elige **Cargar plantilla** → **Composición** y observa un modelo correcto.
3. **Crea su propia clase y valida:** el validador le avisa *“no tiene clave primaria”* y aprende la regla.
4. **Duda qué relación usar:** consulta la **leyenda UML** o le pregunta al **Moderador IA** en el chat, y su equipo ve la explicación.
5. **Genera el backend Spring Boot.** Nunca abrió un manual.

El **Manual de Usuario** (`MANUAL_DE_USUARIO.md`) existe como respaldo escrito, pero no es necesario para empezar a usar la herramienta.

Anexos

Anexo A. Evidencia del sistema en funcionamiento

Este anexo reúne la evidencia de que el sistema fue construido, desplegado y verificado de extremo a extremo. Las secciones A.1 a A.7 corresponden al 1er Parcial (se mantienen vigentes) y las A.8 a A.12 a lo agregado en el 2do Parcial.

A.1 Despliegue en producción

La herramienta está desplegada en un servidor accesible públicamente, con certificado SSL válido (HTTPS):

- **URL de producción:** `https://diagramasw1pracial100.duckdns.org`
- **Infraestructura:** servidor AWS EC2 (t3.micro, región us-east-2) con Docker Compose. Contenedores:
    - frontend (nginx con SSL);
    - backend NestJS;
    - PostgreSQL 16;
    - **MongoDB 7** (nuevo en el 2do Parcial).

  Los servicios se reinician solos y HTTP redirige a HTTPS.

  La versión del 2do Parcial (con MongoDB y el Moderador IA) está desplegada y verificada en producción desde el 17/10/2026. El 07/11/2026 se volvió a desplegar en una cuenta nueva de AWS, restaurando los respaldos de ambas bases de datos.
- **Dominio y certificado:** DuckDNS + Let’s Encrypt.
- **Almacenamiento en la nube:** los archivos adjuntos de los diagramas se guardan en un *bucket* de Amazon S3. El backend usa un usuario IAM que solo puede leer, escribir y borrar en ese bucket (mínimo privilegio).

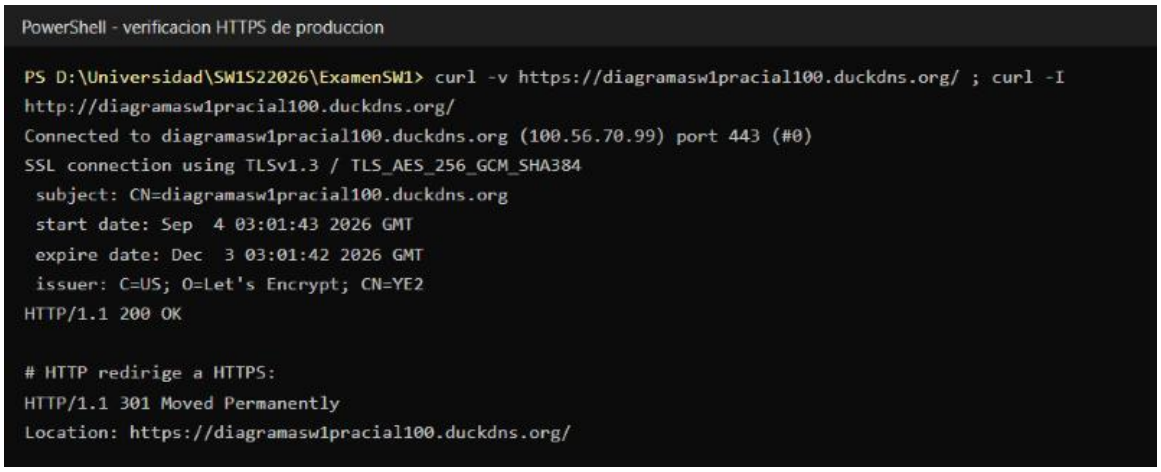


Figura A.1: verificación del sitio en producción: certificado de Let's Encrypt válido hasta el 03/12/2026, TLS 1.3 y redirección de HTTP a HTTPS.

A.2 Modelado y generación de código

Evidencia del lienzo, las relaciones con multiplicidades (1, 0..*) y la edición de atributos: Capturas 6 y 7 del capítulo 7.

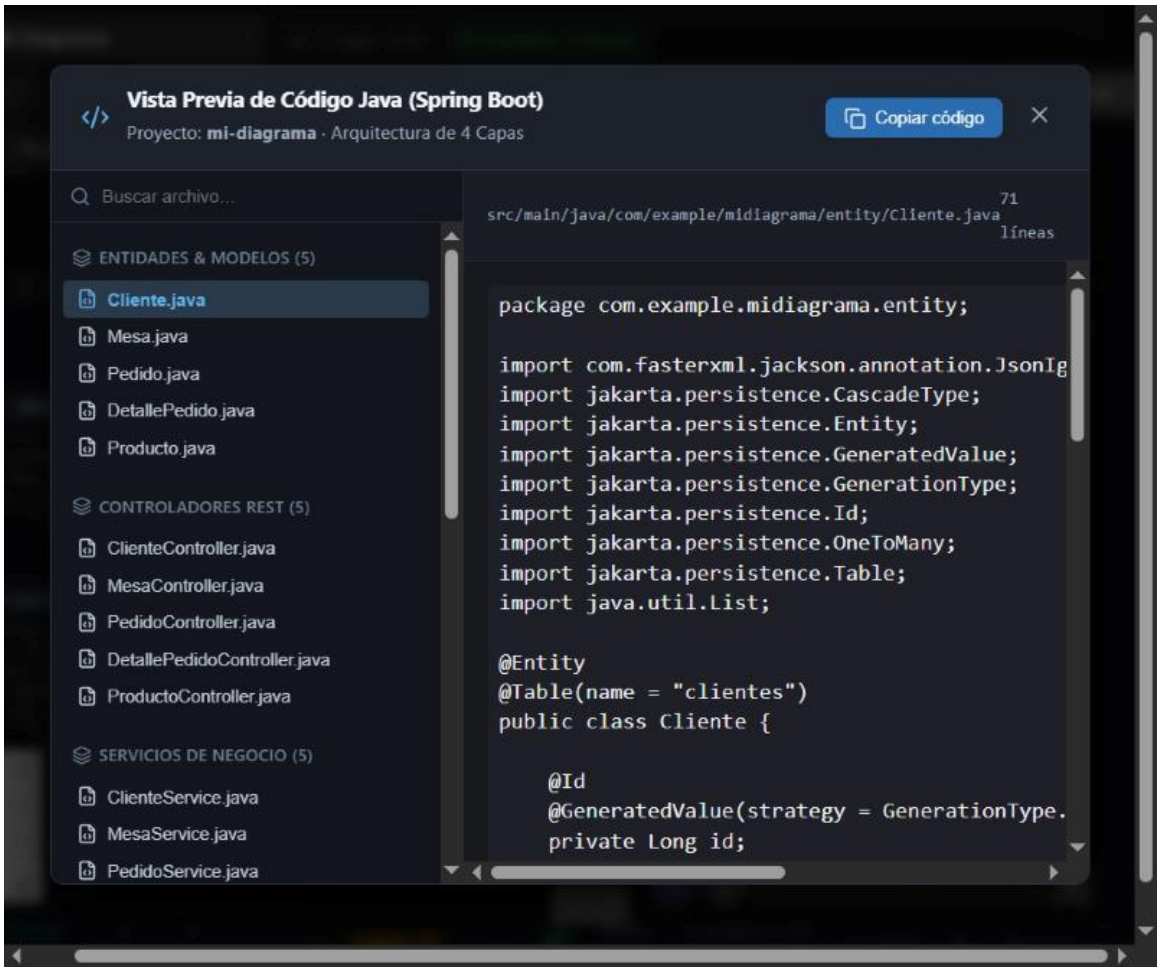


Figura A.5: vista previa del código Java Spring Boot generado: entidades, controladores REST y servicios en arquitectura de 4 capas.

A.3 Colaboración en tiempo real

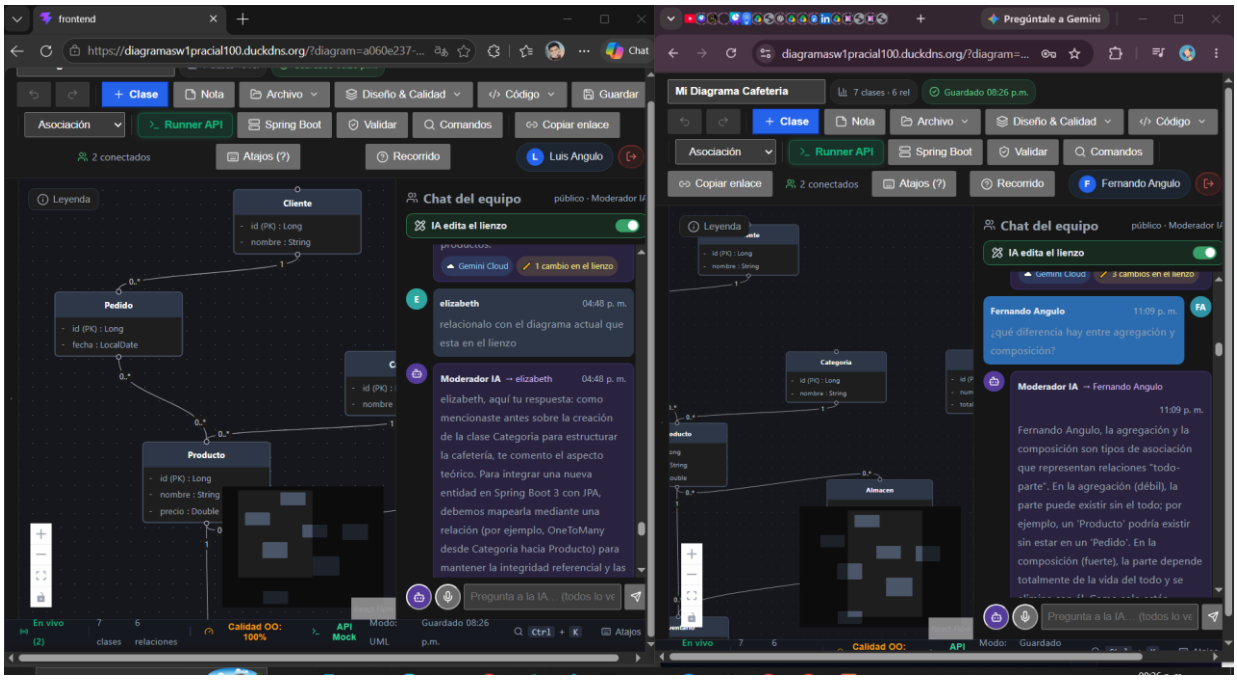


Figura A.6: el mismo diagrama abierto en dos navegadores, Edge con la sesión de Luis y Chrome con la de Fernando. Ambos muestran «2 conectados», el mismo lienzo y el mismo chat público, incluida la pregunta de Elizabeth y la respuesta del Moderador IA dirigida a ella.

A.4 Asistencia por inteligencia artificial (1er Parcial)

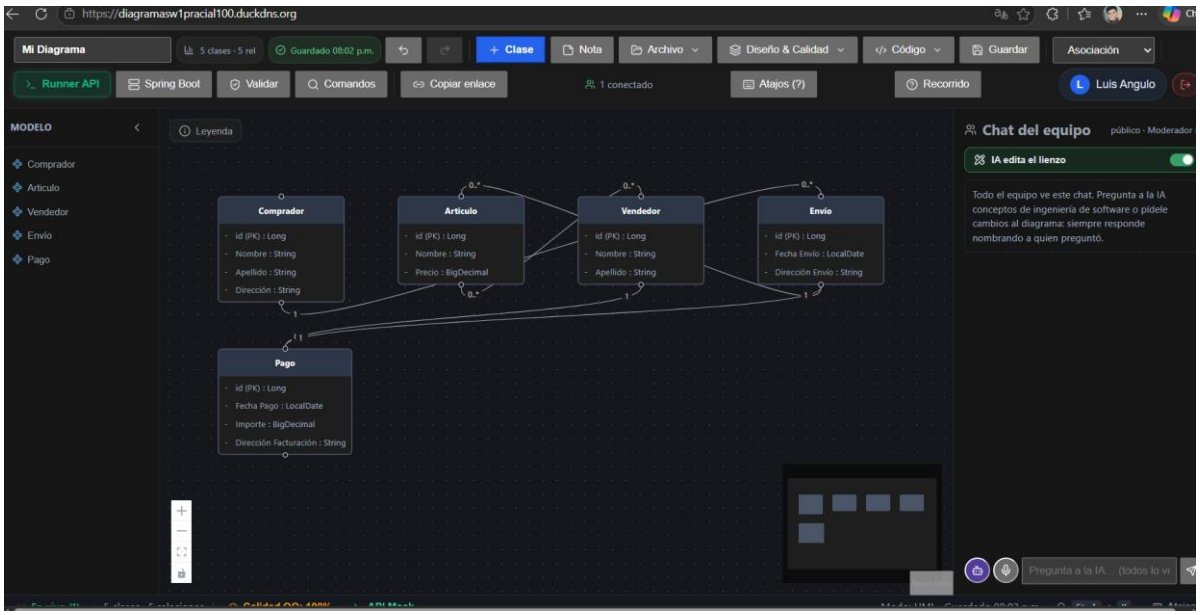


Figura A.7a: diagrama creado desde una fotografía de pizarra (Archivo → Foto de pizarra (IA)): cinco clases con sus atributos y relaciones, en producción.

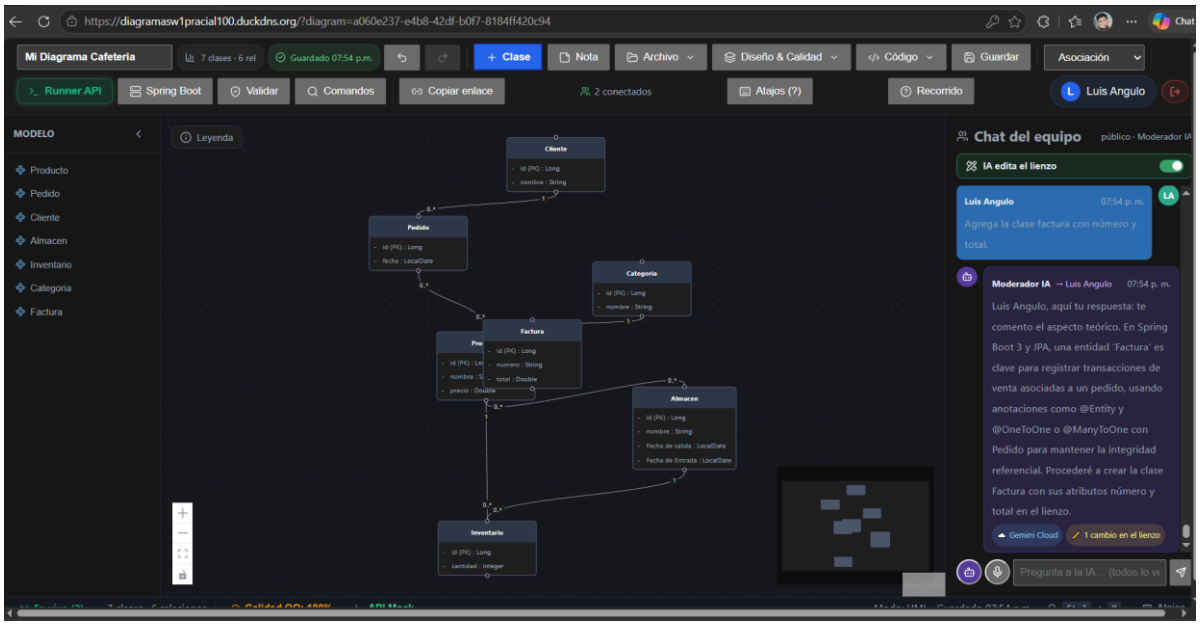


Figura A.7b: clase Factura creada por dictado de voz en el chat del equipo; el Moderador IA responde a Luis por su nombre y aplica el cambio.

A.5 Interoperabilidad con Enterprise Architect (XMI)

El diagrama de veterinaria que el Moderador IA creó en el Modo IA se exportó desde el sistema (Archivo → Exportar XMI, `docs/diagrams/ea/veterinaria_modo_ia.xmi`) y se importó en Enterprise Architect 13.5 (Import Package from XMI). EA reconoce las 4 clases con sus atributos tipados y las 3 relaciones con su multiplicidad. El proyecto de EA queda en `docs/diagrams/ea/veterinaria_modo_ia.eap`.

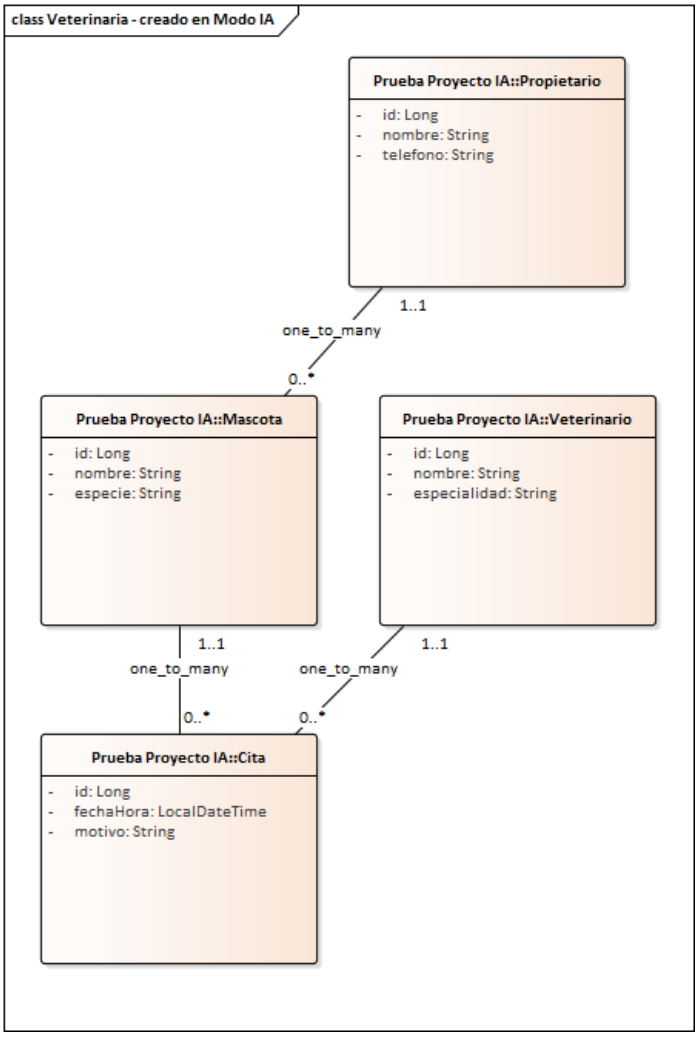


Figura A.8: modelo exportado en XMI desde el sistema e importado en Enterprise Architect 13.5: las clases Propietario, Mascota, Veterinario y Cita con sus atributos y las relaciones uno a muchos.

A.6 Cliente móvil y IA local verificados en dispositivo físico

- **APK compilado (modo release):** `ExamenSW1.apk`.
- Verificado en un teléfono Android real:
    - creación y listado de registros contra el backend generado (GET y POST persistidos en PostgreSQL);
    - dictado por voz;
    - **asistente de IA local funcionando en modo avión**.

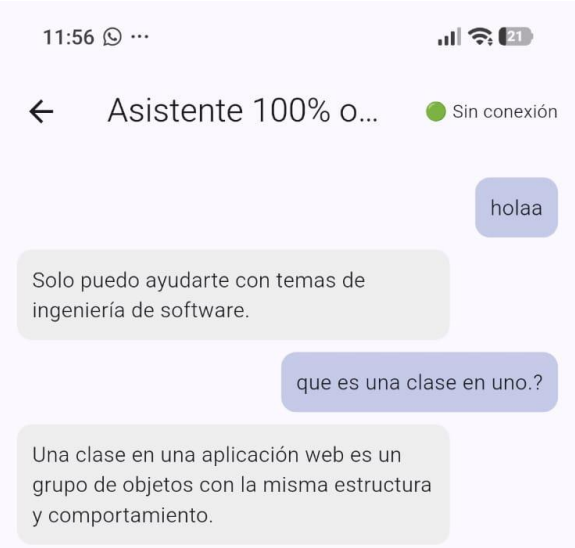


Figura A.9: asistente de IA local de la aplicación móvil en un teléfono Android real, sin conexión: rechaza un saludo fuera de tema y responde qué es una clase.

A.7 Pruebas automatizadas

El backend tiene **152 pruebas unitarias en 22 archivos** (Vitest), todas aprobadas al 05/11/2026. Cubren:

- el generador y cada plantilla;
- validación y XMI;
- autoría y bitácora;
- el motor de IA híbrido y el prompt del moderador.

Evidencia: Captura 3 del capítulo 7 (`npx vitest run` con “117 passed”).

A.8 Persistencia en MongoDB con autoría granular (2do Parcial)

- Colecciones `diagrams`, `diagram_versions`, `diagram_sessions`, `diagram_activity_logs` y `group_chat_messages`.
- Historial de versiones con **Restaurar** (*Archivo → Historial de versiones*; Captura 14 del capítulo 7).
- Cada clase, atributo y relación registra quién la creó, quién la modificó y desde qué fuente (manual, IA o XMI).
- Migración desde PostgreSQL con IDs conservados: 8 diagramas y 55 entradas de historial. Se ejecutó dos veces sin duplicar datos.

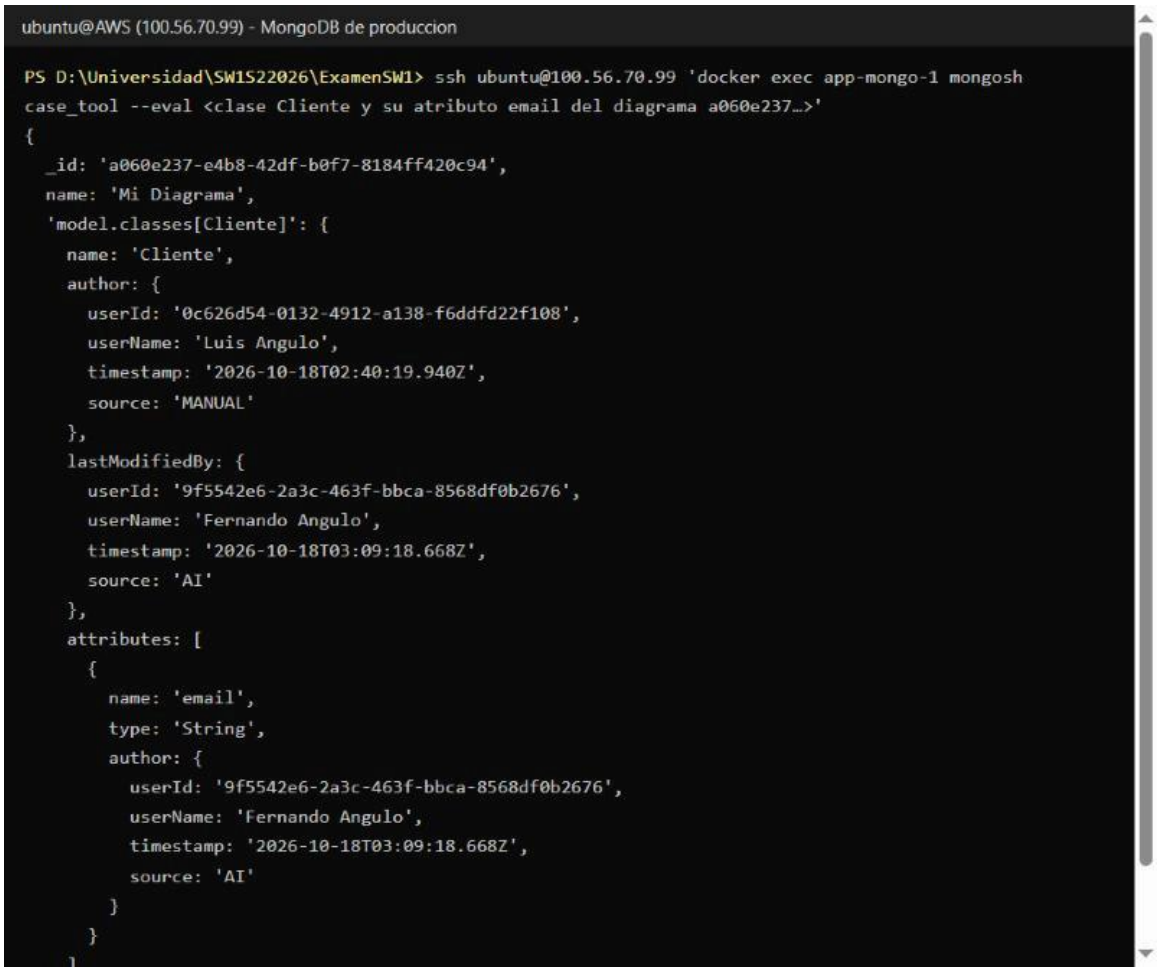


Figura A.11: documento real en el MongoDB de producción: la clase Cliente con `author` (Luis, manual) y `lastModifiedBy` (Fernando, vía IA), y el atributo email con su propia autoría.

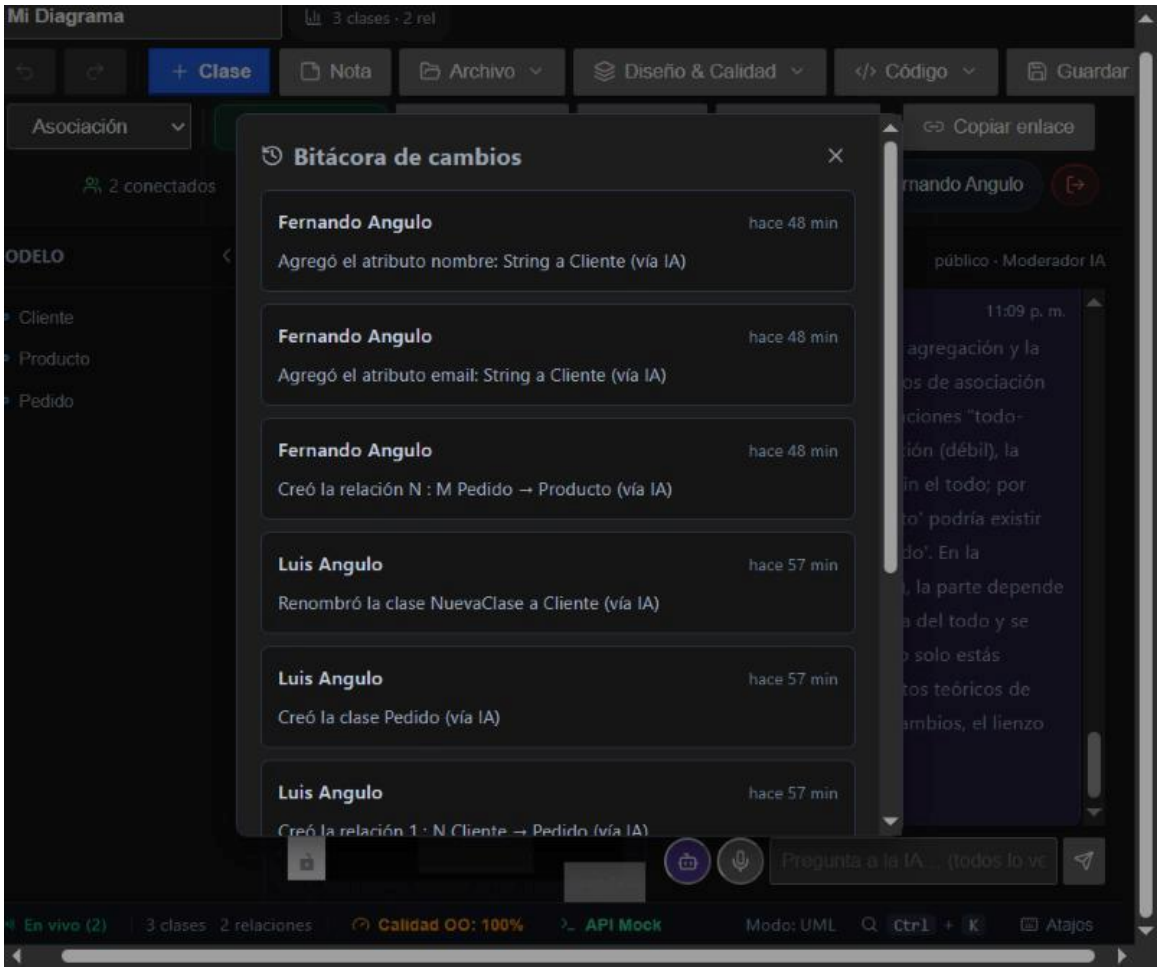


Figura A.12: bitácora de cambios en producción, con cada cambio atribuido a su autor e indicando los hechos «(vía IA)».

A.9 Chat grupal público con el Moderador IA (2do Parcial)

- Todos los conectados ven cada pregunta y respuesta; la IA nombra a quien pregunta y muestra qué proveedor respondió (🏠 Ollama Local / ☁ Gemini Cloud).
- Prueba con dos usuarios simultáneos:
    - indicador *“La IA está pensando la respuesta para Luis”*;
    - cambio aplicado y sincronizado en ambos lienzos;
    - autoría *“Creada por Luis · vía IA”*.

Evidencia: Capturas 6 y 7 del capítulo 7 (chat público en producción e inspector con la autoría “vía IA”).

A.10 Motor de IA híbrido (2do Parcial)

Mediciones reales en la laptop del equipo (CPU):

Escenario	Respondió	Tiempo
Gemini primero (por defecto)	Gemini Cloud	4,3 s
Falla Gemini → respaldo automático	Ollama Local	13,5 s
Ollama primero, modelo precalentado	Ollama Local	5,0 s

Evidencia: Capturas 4 y 5 del capítulo 7 (log de Ollama y badge «🏠 Ollama Local»).

A.11 Visor UML en tiempo real en Flutter (2do Parcial)

- El móvil ve en vivo (solo lectura) el diagrama y el chat del equipo; un cambio pedido a la IA desde la web aparece sin recargar.
- Toca una clase para ver su autoría.
- Notificaciones locales de mensajes y cambios.
- APK de release compilado con los cambios.

Evidencia: Captura 8 del capítulo 7 (pestañas Diagrama y Chat del visor).

A.12 Mecanismo de aprendizaje del usuario

Evidencia: Capturas 9 a 12 de la sección 3.3 (estado vacío, recorrido guiado, leyenda y atajos, y respuesta teórica del Moderador IA).

Anexo B — Estándar de Codificación

1. Introducción

El presente estándar define las convenciones, herramientas y prácticas aplicadas durante la construcción del proyecto, con el fin de garantizar un código legible, uniforme y mantenible a través de las distintas tecnologías que lo componen: el backend en TypeScript (NestJS), el frontend en TypeScript (React), el cliente móvil en Dart (Flutter) y el código Java (Spring Boot) producido por el motor generador. La adopción de un estándar común reduce los errores de transcripción, facilita el trabajo colaborativo y asegura la coherencia del producto final.

2. Estándares Internacionales Implementados

- **UML 2.5 (OMG):** notación estándar para el modelado de los diagramas de clases, base conceptual de toda la herramienta.
- **XMI 2.1 (OMG):** formato estándar de intercambio de modelos para la importación y exportación con herramientas externas como Enterprise Architect.
- **OpenAPI 3 / REST:** los endpoints generados siguen el estilo arquitectónico REST y se documentan con la especificación OpenAPI 3 (Swagger UI).
- **ISO/IEC 25010:** atributos de calidad (mantenibilidad, portabilidad y fiabilidad) tomados como referencia del producto software.

- **Convenciones de nomenclatura:** TypeScript y Dart en camelCase (variables/funciones) y PascalCase (clases, tipos, componentes); Java en PascalCase para clases y camelCase para métodos, según Spring Boot; tablas de base de datos en snake_case.
- **Arquitectura en capas:** el backend generado respeta el patrón Controlador → Servicio → Repositorio → Entidad.

3. Stack Tecnológico y Herramientas de Calidad

El backend de la herramienta CASE está construido en NestJS + TypeScript (modo strict) con TypeORM y PostgreSQL; el frontend en React + Vite + TypeScript con React Flow; el cliente móvil en Flutter/Dart; y la salida generada en Spring Boot 3.3.4 con Java 17 y Maven. El aseguramiento de la calidad se apoya en herramientas automáticas:

- **oxlint:** analizador estático (linter) del código TypeScript en backend y frontend, que detecta errores y malas prácticas antes de la ejecución.
- **Prettier:** formateador automático con configuración uniforme (comillas simples y coma final en estructuras multilínea), evitando diffs ruidosos.
- **TypeScript strict:** verificación estricta de tipos en tiempo de compilación en backend y frontend.
- **vitest:** pruebas unitarias y de integración del backend, con medición de cobertura y pruebas end-to-end.
- **flutter_lints y flutter analyze:** reglas de análisis estático oficiales para el código Dart del cliente móvil.

4. Configuración del Entorno de Desarrollo

- **Requisitos:** Node.js 22+, PostgreSQL en localhost:5432 y el SDK de Flutter.
- **Variables de entorno (.env, nunca versionadas):** credenciales de base de datos, GEMINI_API_KEY y AI_PROVIDER (para alternar entre IA en la nube y la IA local).
- **Comandos estandarizados:** npm run start:dev (desarrollo con recarga en caliente), npm run lint (oxlint), npm run format (Prettier), npm test y npm run test:cov (vitest), npm run test:e2e (end-to-end).
- **Formateo automático al guardar** habilitado en el editor, para mantener el estilo sin intervención manual.

5. Flujo de Trabajo y Métricas

- **Control de versiones con Git:** cada avance se registra en commits con mensajes descriptivos y prefijos convencionales (feat, fix, docs), trazando la evolución del proyecto.
- **Verificación previa a la integración:** antes de validar un cambio se ejecutan el linter (oxlint / flutter analyze), el formateador y las pruebas automatizadas.

- **Métricas de calidad:** suite de pruebas del backend en verde con cobertura medida; verificación estricta de tipos (TypeScript strict); y pruebas específicas del motor generador que validan el pom.xml y las plantillas producidas.
- **Revisión de código:** el conocimiento del código por parte de los autores se verifica en la defensa presencial de autoría.

6. Conclusión

La aplicación de este estándar —soportado por herramientas automáticas de análisis estático, formateo, verificación de tipos y pruebas— asegura un nivel de calidad consistente a pesar de combinar cuatro lenguajes y plataformas distintas. El estándar no es solo un conjunto de recomendaciones: está respaldado por herramientas que lo hacen cumplir de forma automática, reduciendo la deuda técnica y facilitando la mantenibilidad y evolución futura del sistema.

Anexo C. Información del grupo y recursos del proyecto

Campo	Valor
Documento	**Segundo Examen Parcial**
Materia	Ingeniería de Software 1 (SW1)
Docente	Ing. Rolando Martínez
Grupo	**43**
Integrantes	**Elizabeth Rivero Peña** (registro ⟨…⟩) · **Luis Fernando Angulo Heredia** (registro ⟨…⟩)
Fecha de entrega	Martes 10 de noviembre de 2026: documento hasta las 08:00, defensa a las 16:00
Repositorio (QR 1)	`https://github.com/luisfernandoAngulo28/generador_diagrama_clases_uml`
Aplicación en producción (QR 2)	`https://diagramasw1pracial100.duckdns.org`
Aplicación móvil	`ExamenSW1.apk` (incluye el Visor UML en vivo)

Anexo D. Metodología de investigación y de desarrollo

D.1 Metodología de desarrollo: PUDS (Proceso Unificado, versión simplificada)

Seguimos el **Proceso Unificado de Desarrollo de Software** (Jacobson, Booch y Rumbaugh, 1999). Sus tres principios se ven en el proyecto:

- **Dirigido por casos de uso:** los requisitos se modelaron como actores y casos de uso por ciclo (`docs/diagrams/actores.*`, `emcu_ciclo1..3.*`).
- **Centrado en la arquitectura:** las cuatro arquitecturas (backend, frontend, datos y despliegue) se definieron antes de construir; por ejemplo, `PLAN_SEMANA1_MONGODB.md` diseñó la persistencia antes de escribir código.

- **Iterativo e incremental:** el 1er Parcial fue la base y el 2do Parcial se construyó en cuatro iteraciones, cada una con su commit y su verificación.

Fase PUDS	Qué se hizo en el 2do Parcial
Inicio	Requisitos del docente, actores y EMCU, plan de 4 semanas (`PLAN_EVOLUCION_2DO_PARCIAL.md`)
Elaboración	Arquitectura de datos políglota, protocolo WebSocket, diseño del motor IA híbrido
Construcción	Iteración 1: MongoDB y autoría · 2: IA híbrida · 3: chat grupal · 4: visor Flutter
Transición	Pruebas de regresión, documentación SDD, despliegue en el VPS y simulacro

D.2 Métodos de investigación empleados

- **Investigación documental:** especificaciones del OMG (UML 2.5.1, XMI 2.1), bibliografía de ingeniería de software y documentación técnica oficial (NestJS, MongoDB, Socket.IO, Ollama, Gemini).
- **Investigación experimental:** mediciones controladas para decidir el diseño. Por ejemplo, la latencia del modelo local con el prompt completo (36–79 s) frente al prompt lite con caché (5–18 s) determinó que Gemini responda primero por defecto y Ollama quede como respaldo sin internet.
- **Prueba y verificación:** pruebas unitarias automatizadas, pruebas de punta a punta y pruebas con varios usuarios simultáneos. Los defectos encontrados y sus causas se registran en la sección 7.5.

Bibliografía

Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Google. (2024). Gemini API documentation. https://ai.google.dev/docs

Jacobson, I., Booch, G., & Rumbaugh, J. (1999). The unified software development process. Addison-Wesley.

Martin, R. C. (2017). Clean architecture: A craftsman's guide to software structure and design. Prentice Hall.

MongoDB, Inc. (s.f.). *MongoDB manual*. https://www.mongodb.com/docs/manual/

NestJS. (s.f.). *NestJS documentation*. https://docs.nestjs.com/

Object Management Group. (2005). *MOF 2.0/XMI mapping specification, version 2.1*. https://www.omg.org/spec/XMI/

Object Management Group. (2017). OMG Unified Modeling Language (UML), versión 2.5.1. https://www.omg.org/spec/UML/2.5.1/

Ollama. (s.f.). *Ollama documentation*. https://github.com/ollama/ollama/tree/main/docs

Pressman, R. S., & Maxim, B. R. (2015). Ingeniería de software: Un enfoque práctico (8.a ed.). McGraw-Hill.

Ries, E. (2011). The lean startup. Crown Business.

Rumbaugh, J., Blaha, M., Premerlani, W., Eddy, F., & Lorensen, W. (1991). *Object-oriented modeling and design*. Prentice Hall.

Rumbaugh, J., Jacobson, I., & Booch, G. (2005). The Unified Modeling Language reference manual (2.a ed.). Addison-Wesley.

Sadalage, P. J., & Fowler, M. (2012). *NoSQL distilled: A brief guide to the emerging world of polyglot persistence*. Addison-Wesley.

Socket.IO. (s.f.). *Socket.IO documentation*. https://socket.io/docs/v4/

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Spring Framework Documentation. (2024). Spring Boot reference documentation. https://docs.spring.io/spring-boot/docs/current/reference/html/

Szyperski, C. (2002). Component software: Beyond object-oriented programming (2.a ed.). Addison-Wesley.